

Normalization & Optimization Schemes

Table of Contents

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift	2
Layer Normalization	11
Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks	23
Normalization and effective learning rates in reinforcement learning	32
Scaling Off-Policy Reinforcement Learning with Batch and Weight Normalization	54
On the importance of initialization and momentum in deep learning	65
Understanding and Improving Layer Normalization	73
Adam: A Method for Stochastic Optimization	90
END	103

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Sergey Ioffe
Google Inc., sioffe@google.com

Christian Szegedy
Google Inc., szegedy@google.com

Abstract

Training Deep Neural Networks is complicated by the fact that the distribution of each layer’s inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities. We refer to this phenomenon as *internal covariate shift*, and address the problem by normalizing layer inputs. Our method draws its strength from making normalization a part of the model architecture and performing the normalization *for each training mini-batch*. Batch Normalization allows us to use much higher learning rates and be less careful about initialization. It also acts as a regularizer, in some cases eliminating the need for Dropout. Applied to a state-of-the-art image classification model, Batch Normalization achieves the same accuracy with 14 times fewer training steps, and beats the original model by a significant margin. Using an ensemble of batch-normalized networks, we improve upon the best published result on ImageNet classification: reaching 4.9% top-5 validation error (and 4.8% test error), exceeding the accuracy of human raters.

1 Introduction

Deep learning has dramatically advanced the state of the art in vision, speech, and many other areas. Stochastic gradient descent (SGD) has proved to be an effective way of training deep networks, and SGD variants such as momentum (Sutskever et al., 2013) and Adagrad (Duchi et al., 2011) have been used to achieve state of the art performance. SGD optimizes the parameters Θ of the network, so as to minimize the loss

$$\Theta = \arg \min_{\Theta} \frac{1}{N} \sum_{i=1}^N \ell(x_i, \Theta)$$

where $x_{1\dots N}$ is the training data set. With SGD, the training proceeds in steps, and at each step we consider a *mini-batch* $x_{1\dots m}$ of size m . The mini-batch is used to approximate the gradient of the loss function with respect to the parameters, by computing

$$\frac{1}{m} \frac{\partial \ell(x_i, \Theta)}{\partial \Theta}.$$

Using mini-batches of examples, as opposed to one example at a time, is helpful in several ways. First, the gradient of the loss over a mini-batch is an estimate of the gradient over the training set, whose quality improves as the batch size increases. Second, computation over a batch can be much more efficient than m computations for individual examples, due to the parallelism afforded by the modern computing platforms.

While stochastic gradient is simple and effective, it requires careful tuning of the model hyper-parameters, specifically the learning rate used in optimization, as well as the initial values for the model parameters. The training is complicated by the fact that the inputs to each layer are affected by the parameters of all preceding layers – so that small changes to the network parameters amplify as the network becomes deeper.

The change in the distributions of layers’ inputs presents a problem because the layers need to continuously adapt to the new distribution. When the input distribution to a learning system changes, it is said to experience *covariate shift* (Shimodaira, 2000). This is typically handled via domain adaptation (Jiang, 2008). However, the notion of covariate shift can be extended beyond the learning system as a whole, to apply to its parts, such as a sub-network or a layer. Consider a network computing

$$\ell = F_2(F_1(u, \Theta_1), \Theta_2)$$

where F_1 and F_2 are arbitrary transformations, and the parameters Θ_1, Θ_2 are to be learned so as to minimize the loss ℓ . Learning Θ_2 can be viewed as if the inputs $x = F_1(u, \Theta_1)$ are fed into the sub-network

$$\ell = F_2(x, \Theta_2).$$

For example, a gradient descent step

$$\Theta_2 \leftarrow \Theta_2 - \frac{\alpha}{m} \sum_{i=1}^m \frac{\partial F_2(x_i, \Theta_2)}{\partial \Theta_2}$$

(for batch size m and learning rate α) is exactly equivalent to that for a stand-alone network F_2 with input x . Therefore, the input distribution properties that make training more efficient – such as having the same distribution between the training and test data – apply to training the sub-network as well. As such it is advantageous for the distribution of x to remain fixed over time. Then, Θ_2 does

not have to readjust to compensate for the change in the distribution of x .

Fixed distribution of inputs to a sub-network would have positive consequences for the layers *outside* the sub-network, as well. Consider a layer with a sigmoid activation function $z = g(Wu + b)$ where u is the layer input, the weight matrix W and bias vector b are the layer parameters to be learned, and $g(x) = \frac{1}{1+\exp(-x)}$. As $|x|$ increases, $g'(x)$ tends to zero. This means that for all dimensions of $x = Wu + b$ except those with small absolute values, the gradient flowing down to u will vanish and the model will train slowly. However, since x is affected by W , b and the parameters of all the layers below, changes to those parameters during training will likely move many dimensions of x into the saturated regime of the nonlinearity and slow down the convergence. This effect is amplified as the network depth increases. In practice, the saturation problem and the resulting vanishing gradients are usually addressed by using Rectified Linear Units (Nair & Hinton, 2010) $ReLU(x) = \max(x, 0)$, careful initialization (Bengio & Glorot, 2010; Saxe et al., 2013), and small learning rates. If, however, we could ensure that the distribution of nonlinearity inputs remains more stable as the network trains, then the optimizer would be less likely to get stuck in the saturated regime, and the training would accelerate.

We refer to the change in the distributions of internal nodes of a deep network, in the course of training, as *Internal Covariate Shift*. Eliminating it offers a promise of faster training. We propose a new mechanism, which we call *Batch Normalization*, that takes a step towards reducing internal covariate shift, and in doing so dramatically accelerates the training of deep neural nets. It accomplishes this via a normalization step that fixes the means and variances of layer inputs. Batch Normalization also has a beneficial effect on the gradient flow through the network, by reducing the dependence of gradients on the scale of the parameters or of their initial values. This allows us to use much higher learning rates without the risk of divergence. Furthermore, batch normalization regularizes the model and reduces the need for Dropout (Srivastava et al., 2014). Finally, Batch Normalization makes it possible to use saturating nonlinearities by preventing the network from getting stuck in the saturated modes.

In Sec. 4.2, we apply Batch Normalization to the best-performing ImageNet classification network, and show that we can match its performance using only 7% of the training steps, and can further exceed its accuracy by a substantial margin. Using an ensemble of such networks trained with Batch Normalization, we achieve the top-5 error rate that improves upon the best known results on ImageNet classification.

2 Towards Reducing Internal Covariate Shift

We define *Internal Covariate Shift* as the change in the distribution of network activations due to the change in network parameters during training. To improve the training, we seek to reduce the internal covariate shift. By fixing the distribution of the layer inputs x as the training progresses, we expect to improve the training speed. It has been long known (LeCun et al., 1998b; Wiesler & Ney, 2011) that the network training converges faster if its inputs are whitened – i.e., linearly transformed to have zero means and unit variances, and decorrelated. As each layer observes the inputs produced by the layers below, it would be advantageous to achieve the same whitening of the inputs of each layer. By whitening the inputs to each layer, we would take a step towards achieving the fixed distributions of inputs that would remove the ill effects of the internal covariate shift.

We could consider whitening activations at every training step or at some interval, either by modifying the network directly or by changing the parameters of the optimization algorithm to depend on the network activation values (Wiesler et al., 2014; Raiko et al., 2012; Povey et al., 2014; Desjardins & Kavukcuoglu). However, if these modifications are interspersed with the optimization steps, then the gradient descent step may attempt to update the parameters in a way that requires the normalization to be updated, which reduces the effect of the gradient step. For example, consider a layer with the input u that adds the learned bias b , and normalizes the result by subtracting the mean of the activation computed over the training data: $\hat{x} = x - E[x]$ where $x = u + b$, $\mathcal{X} = \{x_1 \dots x_N\}$ is the set of values of x over the training set, and $E[x] = \frac{1}{N} \sum_{i=1}^N x_i$. If a gradient descent step ignores the dependence of $E[x]$ on b , then it will update $b \leftarrow b + \Delta b$, where $\Delta b \propto -\partial \ell / \partial \hat{x}$. Then $u + (b + \Delta b) - E[u + (b + \Delta b)] = u + b - E[u + b]$. Thus, the combination of the update to b and subsequent change in normalization led to no change in the output of the layer nor, consequently, the loss. As the training continues, b will grow indefinitely while the loss remains fixed. This problem can get worse if the normalization not only centers but also scales the activations. We have observed this empirically in initial experiments, where the model blows up when the normalization parameters are computed outside the gradient descent step.

The issue with the above approach is that the gradient descent optimization does not take into account the fact that the normalization takes place. To address this issue, we would like to ensure that, for any parameter values, the network *always* produces activations with the desired distribution. Doing so would allow the gradient of the loss with respect to the model parameters to account for the normalization, and for its dependence on the model parameters Θ . Let again x be a layer input, treated as a

vector, and $\mathcal{X}$ be the set of these inputs over the training data set. The normalization can then be written as a transformation

$$\hat{x} = \text{Norm}(x, \mathcal{X})$$

which depends not only on the given training example x but on all examples $\mathcal{X}$ – each of which depends on Θ if x is generated by another layer. For backpropagation, we would need to compute the Jacobians

$$\frac{\partial \text{Norm}(x, \mathcal{X})}{\partial x} \text{ and } \frac{\partial \text{Norm}(x, \mathcal{X})}{\partial \mathcal{X}};$$

ignoring the latter term would lead to the explosion described above. Within this framework, whitening the layer inputs is expensive, as it requires computing the covariance matrix $\text{Cov}[x] = E_{x \in \mathcal{X}}[xx^T] - E[x]E[x]^T$ and its inverse square root, to produce the whitened activations $\text{Cov}[x]^{-1/2}(x - E[x])$, as well as the derivatives of these transforms for backpropagation. This motivates us to seek an alternative that performs input normalization in a way that is differentiable and does not require the analysis of the entire training set after every parameter update.

Some of the previous approaches (e.g. (Lyu & Simoncelli, 2008)) use statistics computed over a single training example, or, in the case of image networks, over different feature maps at a given location. However, this changes the representation ability of a network by discarding the absolute scale of activations. We want to preserve the information in the network, by normalizing the activations in a training example relative to the statistics of the entire training data.

3 Normalization via Mini-Batch Statistics

Since the full whitening of each layer's inputs is costly and not everywhere differentiable, we make two necessary simplifications. The first is that instead of whitening the features in layer inputs and outputs jointly, we will normalize each scalar feature independently, by making it have the mean of zero and the variance of 1. For a layer with d -dimensional input $x = (x^{(1)} \dots x^{(d)})$, we will normalize each dimension

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

where the expectation and variance are computed over the training data set. As shown in (LeCun et al., 1998b), such normalization speeds up convergence, even when the features are not decorrelated.

Note that simply normalizing each input of a layer may change what the layer can represent. For instance, normalizing the inputs of a sigmoid would constrain them to the linear regime of the nonlinearity. To address this, we make sure that the transformation inserted in the network can represent the identity transform. To accomplish this,

we introduce, for each activation $x^{(k)}$, a pair of parameters $\gamma^{(k)}, \beta^{(k)}$, which scale and shift the normalized value:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}.$$

These parameters are learned along with the original model parameters, and restore the representation power of the network. Indeed, by setting $\gamma^{(k)} = \sqrt{\text{Var}[x^{(k)}]}$ and $\beta^{(k)} = E[x^{(k)}]$, we could recover the original activations, if that were the optimal thing to do.

In the batch setting where each training step is based on the entire training set, we would use the whole set to normalize activations. However, this is impractical when using stochastic optimization. Therefore, we make the second simplification: since we use mini-batches in stochastic gradient training, *each mini-batch produces estimates of the mean and variance* of each activation. This way, the statistics used for normalization can fully participate in the gradient backpropagation. Note that the use of mini-batches is enabled by computation of per-dimension variances rather than joint covariances; in the joint case, regularization would be required since the mini-batch size is likely to be smaller than the number of activations being whitened, resulting in singular covariance matrices.

Consider a mini-batch $\mathcal{B}$ of size m . Since the normalization is applied to each activation independently, let us focus on a particular activation $x^{(k)}$ and omit k for clarity. We have m values of this activation in the mini-batch,

$$\mathcal{B} = \{x_{1\dots m}\}.$$

Let the normalized values be $\hat{x}_{1\dots m}$, and their linear transformations be $y_{1\dots m}$. We refer to the transform

$$\text{BN}_{\gamma, \beta} : x_{1\dots m} \rightarrow y_{1\dots m}$$

as the *Batch Normalizing Transform*. We present the BN Transform in Algorithm 1. In the algorithm, ϵ is a constant added to the mini-batch variance for numerical stability.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

The BN transform can be added to a network to manipulate any activation. In the notation $y = \text{BN}_{\gamma, \beta}(x)$, we

indicate that the parameters γ and β are to be learned, but it should be noted that the BN transform does not independently process the activation in each training example. Rather, $\text{BN}_{\gamma,\beta}(x)$ depends both on the training example and the other examples in the mini-batch. The scaled and shifted values y are passed to other network layers. The normalized activations $\hat{x}$ are internal to our transformation, but their presence is crucial. The distributions of values of any $\hat{x}$ has the expected value of 0 and the variance of 1, as long as the elements of each mini-batch are sampled from the same distribution, and if we neglect ϵ . This can be seen by observing that $\sum_{i=1}^m \hat{x}_i = 0$ and $\frac{1}{m} \sum_{i=1}^m \hat{x}_i^2 = 1$, and taking expectations. Each normalized activation $\hat{x}^{(k)}$ can be viewed as an input to a sub-network composed of the linear transform $y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$, followed by the other processing done by the original network. These sub-network inputs all have fixed means and variances, and although the joint distribution of these normalized $\hat{x}^{(k)}$ can change over the course of training, we expect that the introduction of normalized inputs accelerates the training of the sub-network and, consequently, the network as a whole.

During training we need to backpropagate the gradient of loss ℓ through this transformation, as well as compute the gradients with respect to the parameters of the BN transform. We use chain rule, as follows (before simplification):

$$\begin{aligned}\frac{\partial \ell}{\partial \hat{x}_i} &= \frac{\partial \ell}{\partial y_i} \cdot \gamma \\ \frac{\partial \ell}{\partial \sigma_B^2} &= \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_B) \cdot \frac{-1}{2} (\sigma_B^2 + \epsilon)^{-3/2} \\ \frac{\partial \ell}{\partial \mu_B} &= \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_B^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_B)}{m} \\ \frac{\partial \ell}{\partial x_i} &= \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{2(x_i - \mu_B)}{m} + \frac{\partial \ell}{\partial \mu_B} \cdot \frac{1}{m} \\ \frac{\partial \ell}{\partial \gamma} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i \\ \frac{\partial \ell}{\partial \beta} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}\end{aligned}$$

Thus, BN transform is a differentiable transformation that introduces normalized activations into the network. This ensures that as the model is training, layers can continue learning on input distributions that exhibit less internal covariate shift, thus accelerating the training. Furthermore, the learned affine transform applied to these normalized activations allows the BN transform to represent the identity transformation and preserves the network capacity.

3.1 Training and Inference with Batch-Normalized Networks

To *Batch-Normalize* a network, we specify a subset of activations and insert the BN transform for each of them, according to Alg. 1. Any layer that previously received x as the input, now receives $\text{BN}(x)$. A model employing Batch Normalization can be trained using batch gradient descent, or Stochastic Gradient Descent with a mini-batch size $m > 1$, or with any of its variants such as Adagrad

(Duchi et al., 2011). The normalization of activations that depends on the mini-batch allows efficient training, but is neither necessary nor desirable during inference; we want the output to depend only on the input, deterministically. For this, once the network has been trained, we use the normalization

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}$$

using the population, rather than mini-batch, statistics. Neglecting ϵ , these normalized activations have the same mean 0 and variance 1 as during training. We use the unbiased variance estimate $\text{Var}[x] = \frac{m}{m-1} \cdot \mathbb{E}_B[\sigma_B^2]$, where the expectation is over training mini-batches of size m and σ_B^2 are their sample variances. Using moving averages instead, we can track the accuracy of a model as it trains. Since the means and variances are fixed during inference, the normalization is simply a linear transform applied to each activation. It may further be composed with the scaling by γ and shift by β , to yield a single linear transform that replaces $\text{BN}(x)$. Algorithm 2 summarizes the procedure for training batch-normalized networks. $\square$

Input: Network N with trainable parameters Θ ;
subset of activations $\{x^{(k)}\}_{k=1}^K$
Output: Batch-normalized network for inference, $N_{\text{BN}}^{\text{inf}}$

- 1: $N_{\text{BN}}^{\text{tr}} \leftarrow N$ // Training BN network
- 2: **for** $k = 1 \dots K$ **do**
- 3: Add transformation $y^{(k)} = \text{BN}_{\gamma^{(k)}, \beta^{(k)}}(x^{(k)})$ to $N_{\text{BN}}^{\text{tr}}$ (Alg. 1)
- 4: Modify each layer in $N_{\text{BN}}^{\text{tr}}$ with input $x^{(k)}$ to take $y^{(k)}$ instead $\square$
- 5: **end for**
- 6: Train $N_{\text{BN}}^{\text{tr}}$ to optimize the parameters $\Theta \cup \{\gamma^{(k)}, \beta^{(k)}\}_{k=1}^K$
- 7: $N_{\text{BN}}^{\text{inf}} \leftarrow N_{\text{BN}}^{\text{tr}}$ // Inference BN network with frozen parameters
- 8: **for** $k = 1 \dots K$ **do**
- 9: // For clarity, $x \equiv x^{(k)}, \gamma \equiv \gamma^{(k)}, \mu_B \equiv \mu_B^{(k)}$, etc.
- 10: Process multiple training mini-batches $\mathcal{B}$, each of size m , and average over them:
$$\begin{aligned}\mathbb{E}[x] &\leftarrow \mathbb{E}_B[\mu_B] \\ \text{Var}[x] &\leftarrow \frac{m}{m-1} \mathbb{E}_B[\sigma_B^2]\end{aligned}$$
- 11: In $N_{\text{BN}}^{\text{inf}}$, replace the transform $y = \text{BN}_{\gamma, \beta}(x)$ with
$$y = \frac{\gamma}{\sqrt{\text{Var}[x] + \epsilon}} \cdot x + \left(\beta - \frac{\gamma \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \right)$$
- 12: **end for**

Algorithm 2: Training a Batch-Normalized Network

3.2 Batch-Normalized Convolutional Networks

Batch Normalization can be applied to any set of activations in the network. Here, we focus on transforms

that consist of an affine transformation followed by an element-wise nonlinearity:

$$z = g(Wu + b)$$

where W and b are learned parameters of the model, and $g(\cdot)$ is the nonlinearity such as sigmoid or ReLU. This formulation covers both fully-connected and convolutional layers. We add the BN transform immediately before the nonlinearity, by normalizing $x = Wu + b$. We could have also normalized the layer inputs u , but since u is likely the output of another nonlinearity, the shape of its distribution is likely to change during training, and constraining its first and second moments would not eliminate the covariate shift. In contrast, $Wu + b$ is more likely to have a symmetric, non-sparse distribution, that is “more Gaussian” (Hyvärinen & Oja, 2000); normalizing it is likely to produce activations with a stable distribution.

Note that, since we normalize $Wu + b$, the bias b can be ignored since its effect will be canceled by the subsequent mean subtraction (the role of the bias is subsumed by β in Alg. 1). Thus, $z = g(Wu + b)$ is replaced with

$$\square \quad z = g(\text{BN}(Wu))$$

where the BN transform is applied independently to each dimension of $x = Wu$, with a separate pair of learned parameters $\gamma^{(k)}, \beta^{(k)}$ per dimension.

For convolutional layers, we additionally want the normalization to obey the convolutional property – so that different elements of the same feature map, at different locations, are normalized in the same way. To achieve this, we jointly normalize all the activations in a mini-batch, over all locations. In Alg. 1 we let $\mathcal{B}$ be the set of all values in a feature map across both the elements of a mini-batch and spatial locations – so for a mini-batch of size m and feature maps of size $p \times q$, we use the effective mini-batch of size $m' = |\mathcal{B}| = m \cdot p \cdot q$. We learn a pair of parameters $\gamma^{(k)}$ and $\beta^{(k)}$ per feature map, rather than per activation. Alg. 2 is modified similarly, so that during inference the BN transform applies the same linear transformation to each activation in a given feature map.

3.3 Batch Normalization enables higher learning rates

In traditional deep networks, too-high learning rate may result in the gradients that explode or vanish, as well as getting stuck in poor local minima. Batch Normalization helps address these issues. By normalizing activations throughout the network, it prevents small changes to the parameters from amplifying into larger and suboptimal changes in activations in gradients; for instance, it prevents the training from getting stuck in the saturated regimes of nonlinearities.

Batch Normalization also makes training more resilient to the parameter scale. Normally, large learning rates may increase the scale of layer parameters, which then amplify

the gradient during backpropagation and lead to the model explosion. However, with Batch Normalization, backpropagation through a layer is unaffected by the scale of its parameters. Indeed, for a scalar a ,

$$\text{BN}(Wu) = \text{BN}((aW)u)$$

and we can show that

$$\begin{aligned} \frac{\partial \text{BN}((aW)u)}{\partial u} &= \frac{\partial \text{BN}(Wu)}{\partial u} \\ \frac{\partial \text{BN}((aW)u)}{\partial (aW)} &= \frac{1}{a} \cdot \frac{\partial \text{BN}(Wu)}{\partial W} \end{aligned}$$

The scale does not affect the layer Jacobian nor, consequently, the gradient propagation. Moreover, larger weights lead to *smaller* gradients, and Batch Normalization will stabilize the parameter growth.

We further conjecture that Batch Normalization may lead the layer Jacobians to have singular values close to 1, which is known to be beneficial for training (Saxe et al., 2013). Consider two consecutive layers with normalized inputs, and the transformation between these normalized vectors: $\hat{z} = F(\hat{x})$. If we assume that $\hat{x}$ and $\hat{z}$ are Gaussian and uncorrelated, and that $F(\hat{x}) \approx J\hat{x}$ is a linear transformation for the given model parameters, then both $\hat{x}$ and $\hat{z}$ have unit covariances, and $I = \text{Cov}[\hat{z}] = J\text{Cov}[\hat{x}]J^T = JJ^T$. Thus, $JJ^T = I$, and so all singular values of J are equal to 1, which preserves the gradient magnitudes during backpropagation. In reality, the transformation is not linear, and the normalized values are not guaranteed to be Gaussian nor independent, but we nevertheless expect Batch Normalization to help make gradient propagation better behaved. The precise effect of Batch Normalization on gradient propagation remains an area of further study.

3.4 Batch Normalization regularizes the model

When training with Batch Normalization, a training example is seen in conjunction with other examples in the mini-batch, and the training network no longer producing deterministic values for a given training example. In our experiments, we found this effect to be advantageous to the generalization of the network. Whereas Dropout (Srivastava et al., 2014) is typically used to reduce overfitting, in a batch-normalized network we found that it can be either removed or reduced in strength.

4 Experiments

4.1 Activations over time

To verify the effects of internal covariate shift on training, and the ability of Batch Normalization to combat it, we considered the problem of predicting the digit class on the MNIST dataset (LeCun et al., 1998a). We used a very simple network, with a 28x28 binary image as input, and

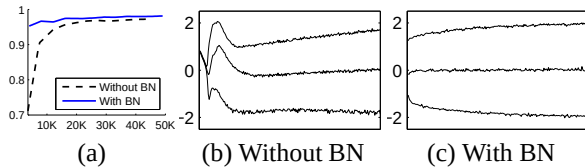


Figure 1: (a) The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy. (b, c) The evolution of input distributions to a typical sigmoid, over the course of training, shown as $\{15, 50, 85\}$ th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.

3 fully-connected hidden layers with 100 activations each. Each hidden layer computes $y = g(Wu + b)$ with sigmoid nonlinearity, and the weights W initialized to small random Gaussian values. The last hidden layer is followed by a fully-connected layer with 10 activations (one per class) and cross-entropy loss. We trained the network for 50000 steps, with 60 examples per mini-batch. We added Batch Normalization to each hidden layer of the network, as in Sec. 3.1. We were interested in the comparison between the baseline and batch-normalized networks, rather than achieving the state of the art performance on MNIST (which the described architecture does not).

Figure 1(a) shows the fraction of correct predictions by the two networks on held-out test data, as training progresses. The batch-normalized network enjoys the higher test accuracy. To investigate why, we studied inputs to the sigmoid, in the original network N and batch-normalized network N_{BN}^{tr} (Alg. 2) over the course of training. In Fig. 1(b,c) we show, for one typical activation from the last hidden layer of each network, how its distribution evolves. The distributions in the original network change significantly over time, both in their mean and the variance, which complicates the training of the subsequent layers. In contrast, the distributions in the batch-normalized network are much more stable as training progresses, which aids the training.

4.2 ImageNet classification

We applied Batch Normalization to a new variant of the Inception network (Szegedy et al., 2014), trained on the ImageNet classification task (Russakovsky et al., 2014). The network has a large number of convolutional and pooling layers, with a softmax layer to predict the image class, out of 1000 possibilities. Convolutional layers use ReLU as the nonlinearity. The main difference to the network described in (Szegedy et al., 2014) is that the 5×5 convolutional layers are replaced by two consecutive layers of 3×3 convolutions with up to 128 filters. The network contains $13.6 \cdot 10^6$ parameters, and, other than the top softmax layer, has no fully-connected layers. More

details are given in the Appendix. We refer to this model as *Inception* in the rest of the text. The model was trained using a version of Stochastic Gradient Descent with momentum (Sutskever et al., 2013), using the mini-batch size of 32. The training was performed using a large-scale, distributed architecture (similar to (Dean et al., 2012)). All networks are evaluated as training progresses by computing the validation accuracy @1, i.e. the probability of predicting the correct label out of 1000 possibilities, on a held-out set, using a single crop per image.

In our experiments, we evaluated several modifications of Inception with Batch Normalization. In all cases, Batch Normalization was applied to the input of each nonlinearity, in a convolutional way, as described in section 3.2 while keeping the rest of the architecture constant.

4.2.1 Accelerating BN Networks

Simply adding Batch Normalization to a network does not take full advantage of our method. To do so, we further changed the network and its training parameters, as follows:

Increase learning rate. In a batch-normalized model, we have been able to achieve a training speedup from higher learning rates, with no ill side effects (Sec. 3.3).

Remove Dropout. As described in Sec. 3.4 Batch Normalization fulfills some of the same goals as Dropout. Removing Dropout from Modified BN-Inception speeds up training, without increasing overfitting.

Reduce the L_2 weight regularization. While in Inception an L_2 loss on the model parameters controls overfitting, in Modified BN-Inception the weight of this loss is reduced by a factor of 5. We find that this improves the accuracy on the held-out validation data.

Accelerate the learning rate decay. In training Inception, learning rate was decayed exponentially. Because our network trains faster than Inception, we lower the learning rate 6 times faster.

Remove Local Response Normalization While Inception and other networks (Srivastava et al., 2014) benefit from it, we found that with Batch Normalization it is not necessary.

Shuffle training examples more thoroughly. We enabled within-shard shuffling of the training data, which prevents the same examples from always appearing in a mini-batch together. This led to about 1% improvements in the validation accuracy, which is consistent with the view of Batch Normalization as a regularizer (Sec. 3.4): the randomization inherent in our method should be most beneficial when it affects an example differently each time it is seen.

Reduce the photometric distortions. Because batch-normalized networks train faster and observe each training example fewer times, we let the trainer focus on more “real” images by distorting them less.

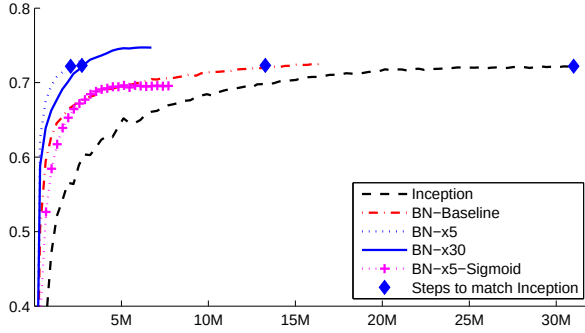


Figure 2: Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.

4.2.2 Single-Network Classification

We evaluated the following networks, all trained on the LSVRC2012 training data, and tested on the validation data:

Inception: the network described at the beginning of Section 4.2, trained with the initial learning rate of 0.0015.

BN-Baseline: Same as Inception with Batch Normalization before each nonlinearity.

BN-x5: Inception with Batch Normalization and the modifications in Sec. 4.2.1. The initial learning rate was increased by a factor of 5, to 0.0075. The same learning rate increase with original Inception caused the model parameters to reach machine infinity.

BN-x30: Like *BN-x5*, but with the initial learning rate 0.045 (30 times that of Inception).

BN-x5-Sigmoid: Like *BN-x5*, but with sigmoid nonlinearity $g(t) = \frac{1}{1+\exp(-x)}$ instead of ReLU. We also attempted to train the original Inception with sigmoid, but the model remained at the accuracy equivalent to chance.

In Figure 2 we show the validation accuracy of the networks, as a function of the number of training steps. Inception reached the accuracy of 72.2% after $31 \cdot 10^6$ training steps. The Figure 3 shows, for each network, the number of training steps required to reach the same 72.2% accuracy, as well as the maximum validation accuracy reached by the network and the number of steps to reach it.

By only using Batch Normalization (*BN-Baseline*), we match the accuracy of Inception in less than half the number of training steps. By applying the modifications in Sec. 4.2.1, we significantly increase the training speed of the network. *BN-x5* needs 14 times fewer steps than Inception to reach the 72.2% accuracy. Interestingly, increasing the learning rate further (*BN-x30*) causes the model to train somewhat slower initially, but allows it to reach a higher final accuracy. It reaches 74.8% after $6 \cdot 10^6$ steps, i.e. 5 times fewer steps than required by Inception to reach 72.2%.

We also verified that the reduction in internal covariate shift allows deep networks with Batch Normalization

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
<i>BN-Baseline</i>	$13.3 \cdot 10^6$	72.7%
<i>BN-x5</i>	$2.1 \cdot 10^6$	73.0%
<i>BN-x30</i>	$2.7 \cdot 10^6$	74.8%
<i>BN-x5-Sigmoid</i>		69.8%

Figure 3: For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.

to be trained when sigmoid is used as the nonlinearity, despite the well-known difficulty of training such networks. Indeed, *BN-x5-Sigmoid* achieves the accuracy of 69.8%. Without Batch Normalization, Inception with sigmoid never achieves better than 1/1000 accuracy.

4.2.3 Ensemble Classification

The current reported best results on the ImageNet Large Scale Visual Recognition Competition are reached by the Deep Image ensemble of traditional models (Wu et al., 2015) and the ensemble model of (He et al., 2015). The latter reports the top-5 error of 4.94%, as evaluated by the ILSVRC server. Here we report a top-5 validation error of 4.9%, and test error of 4.82% (according to the ILSVRC server). This improves upon the previous best result, and exceeds the estimated accuracy of human raters according to (Russakovsky et al., 2014).

For our ensemble, we used 6 networks. Each was based on *BN-x30*, modified via some of the following: increased initial weights in the convolutional layers; using Dropout (with the Dropout probability of 5% or 10%, vs. 40% for the original Inception); and using non-convolutional, per-activation Batch Normalization with last hidden layers of the model. Each network achieved its maximum accuracy after about $6 \cdot 10^6$ training steps. The ensemble prediction was based on the arithmetic average of class probabilities predicted by the constituent networks. The details of ensemble and multicrop inference are similar to (Szegedy et al., 2014).

We demonstrate in Fig. 4 that batch normalization allows us to set new state-of-the-art by a healthy margin on the ImageNet classification benchmarks.

5 Conclusion

We have presented a novel mechanism for dramatically accelerating the training of deep networks. It is based on the premise that covariate shift, which is known to complicate the training of machine learning systems, also ap-

Model	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	variable	-	-	-	5.98%
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	4.9%*

Figure 4: *Batch-Normalized Inception comparison with previous state of the art on the provided validation set comprising 50000 images. *BN-Inception ensemble has reached 4.82% top-5 error on the 100000 images of the test set of the ImageNet as reported by the test server.*

plies to sub-networks and layers, and removing it from internal activations of the network may aid in training. Our proposed method draws its power from normalizing activations, and from incorporating this normalization in the network architecture itself. This ensures that the normalization is appropriately handled by any optimization method that is being used to train the network. To enable stochastic optimization methods commonly used in deep network training, we perform the normalization for each mini-batch, and backpropagate the gradients through the normalization parameters. Batch Normalization adds only two extra parameters per activation, and in doing so preserves the representation ability of the network. We presented an algorithm for constructing, training, and performing inference with batch-normalized networks. The resulting networks can be trained with saturating nonlinearities, are more tolerant to increased training rates, and often do not require Dropout for regularization.

Merely adding Batch Normalization to a state-of-the-art image classification model yields a substantial speedup in training. By further increasing the learning rates, removing Dropout, and applying other modifications afforded by Batch Normalization, we reach the previous state of the art with only a small fraction of training steps – and then beat the state of the art in single-network image classification. Furthermore, by combining multiple models trained with Batch Normalization, we perform better than the best known system on ImageNet, by a significant margin.

Interestingly, our method bears similarity to the standardization layer of (Gülçehre & Bengio, 2013), though the two methods stem from very different goals, and perform different tasks. The goal of Batch Normalization is to achieve a stable distribution of activation values throughout training, and in our experiments we apply it before the nonlinearity since that is where matching the first and second moments is more likely to result in a stable distribution. On the contrary, (Gülçehre & Bengio, 2013) apply the standardization layer to the output of the nonlinearity, which results in sparser activations. In our large-scale image classification experiments, we have not observed the nonlinearity inputs to be sparse, neither with nor without Batch Normalization. Other notable differ-

entiating characteristics of Batch Normalization include the learned scale and shift that allow the BN transform to represent identity (the standardization layer did not require this since it was followed by the learned linear transform that, conceptually, absorbs the necessary scale and shift), handling of convolutional layers, deterministic inference that does not depend on the mini-batch, and batch-normalizing each convolutional layer in the network.

In this work, we have not explored the full range of possibilities that Batch Normalization potentially enables. Our future work includes applications of our method to Recurrent Neural Networks (Pascanu et al., 2013), where the internal covariate shift and the vanishing or exploding gradients may be especially severe, and which would allow us to more thoroughly test the hypothesis that normalization improves gradient propagation (Sec. 3.3). We plan to investigate whether Batch Normalization can help with domain adaptation, in its traditional sense – i.e. whether the normalization performed by the network would allow it to more easily generalize to new data distributions, perhaps with just a recomputation of the population means and variances (Alg. 2). Finally, we believe that further theoretical analysis of the algorithm would allow still more improvements and applications.

References

- Bengio, Yoshua and Glorot, Xavier. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS 2010*, volume 9, pp. 249–256, May 2010.
- Dean, Jeffrey, Corrado, Greg S., Monga, Rajat, Chen, Kai, Devin, Matthieu, Le, Quoc V., Mao, Mark Z., Ranzato, Marc’Aurelio, Senior, Andrew, Tucker, Paul, Yang, Ke, and Ng, Andrew Y. Large scale distributed deep networks. In *NIPS*, 2012.
- Desjardins, Guillaume and Kavukcuoglu, Koray. Natural neural networks. (unpublished).
- Duchi, John, Hazan, Elad, and Singer, Yoram. Adaptive subgradient methods for online learning and stochastic

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	double #3×3 reduce	double #3×3	Pool +proj
convolution*	7×7/2	112×112×64	1						
max pool	3×3/2	56×56×64	0						
convolution	3×3/1	56×56×192	1		64	192			
max pool	3×3/2	28×28×192	0						
inception (3a)		28×28×256	3	64	64	64	64	96	avg + 32
inception (3b)		28×28×320	3	64	64	96	64	96	avg + 64
inception (3c)	stride 2	28×28×576	3	0	128	160	64	96	max + pass through
inception (4a)		14×14×576	3	224	64	96	96	128	avg + 128
inception (4b)		14×14×576	3	192	96	128	96	128	avg + 128
inception (4c)		14×14×576	3	160	128	160	128	160	avg + 128
inception (4d)		14×14×576	3	96	128	192	160	192	avg + 128
inception (4e)	stride 2	14×14×1024	3	0	128	192	192	256	max + pass through
inception (5a)		7×7×1024	3	352	192	320	160	224	avg + 128
inception (5b)		7×7×1024	3	352	192	320	192	224	max + 128
avg pool	7×7/1	1×1×1024	0						

Figure 5: Inception architecture

Layer Normalization

Jimmy Lei Ba
University of Toronto
jimmy@psi.toronto.edu

Jamie Ryan Kiros
University of Toronto
rkiros@cs.toronto.edu

Geoffrey E. Hinton
University of Toronto
and Google Inc.
hinton@cs.toronto.edu

Abstract

Training state-of-the-art, deep neural networks is computationally expensive. One way to reduce the training time is to normalize the activities of the neurons. A recently introduced technique called batch normalization uses the distribution of the summed input to a neuron over a mini-batch of training cases to compute a mean and variance which are then used to normalize the summed input to that neuron on each training case. This significantly reduces the training time in feed-forward neural networks. However, the effect of batch normalization is dependent on the mini-batch size and it is not obvious how to apply it to recurrent neural networks. In this paper, we transpose batch normalization into layer normalization by computing the mean and variance used for normalization from all of the summed inputs to the neurons in a layer on a *single* training case. Like batch normalization, we also give each neuron its own adaptive bias and gain which are applied after the normalization but before the non-linearity. Unlike batch normalization, layer normalization performs exactly the same computation at training and test times. It is also straightforward to apply to recurrent neural networks by computing the normalization statistics separately at each time step. Layer normalization is very effective at stabilizing the hidden state dynamics in recurrent networks. Empirically, we show that layer normalization can substantially reduce the training time compared with previously published techniques.

1 Introduction

Deep neural networks trained with some version of Stochastic Gradient Descent have been shown to substantially outperform previous approaches on various supervised learning tasks in computer vision [Krizhevsky et al., 2012] and speech processing [Hinton et al., 2012]. But state-of-the-art deep neural networks often require many days of training. It is possible to speed-up the learning by computing gradients for different subsets of the training cases on different machines or splitting the neural network itself over many machines [Dean et al., 2012], but this can require a lot of communication and complex software. It also tends to lead to rapidly diminishing returns as the degree of parallelization increases. An orthogonal approach is to modify the computations performed in the forward pass of the neural net to make learning easier. Recently, batch normalization [Ioffe and Szegedy, 2015] has been proposed to reduce training time by including additional normalization stages in deep neural networks. The normalization standardizes each summed input using its mean and its standard deviation across the training data. Feedforward neural networks trained using batch normalization converge faster even with simple SGD. In addition to training time improvement, the stochasticity from the batch statistics serves as a regularizer during training.

Despite its simplicity, batch normalization requires running averages of the summed input statistics. In feed-forward networks with fixed depth, it is straightforward to store the statistics separately for each hidden layer. However, the summed inputs to the recurrent neurons in a recurrent neural network (RNN) often vary with the length of the sequence so applying batch normalization to RNNs appears to require different statistics for different time-steps. Furthermore, batch normaliza-

tion cannot be applied to online learning tasks or to extremely large distributed models where the minibatches have to be small.

This paper introduces layer normalization, a simple normalization method to improve the training speed for various neural network models. Unlike batch normalization, the proposed method directly estimates the normalization statistics from the summed inputs to the neurons within a hidden layer so the normalization does not introduce any new dependencies between training cases. We show that layer normalization works well for RNNs and improves both the training time and the generalization performance of several existing RNN models.

2 Background

A feed-forward neural network is a non-linear mapping from an input pattern $\mathbf{x}$ to an output vector y . Consider the l^{th} hidden layer in a deep feed-forward, neural network, and let a^l be the vector representation of the summed inputs to the neurons in that layer. The summed inputs are computed through a linear projection with the weight matrix W^l and the bottom-up inputs h^l given as follows:

$$a_i^l = w_i^{l\top} h^l \quad h_i^{l+1} = f(a_i^l + b_i^l) \quad (1)$$

where $f(\cdot)$ is an element-wise non-linear function and w_i^l is the incoming weights to the i^{th} hidden units and b_i^l is the scalar bias parameter. The parameters in the neural network are learnt using gradient-based optimization algorithms with the gradients being computed by back-propagation.

One of the challenges of deep learning is that the gradients with respect to the weights in one layer are highly dependent on the outputs of the neurons in the previous layer especially if these outputs change in a highly correlated way. Batch normalization [Ioffe and Szegedy, 2015] was proposed to reduce such undesirable ‘‘covariate shift’’. The method normalizes the summed inputs to each hidden unit over the training cases. Specifically, for the i^{th} summed input in the l^{th} layer, the batch normalization method rescales the summed inputs according to their variances under the distribution of the data

$$\bar{a}_i^l = \frac{g_i^l}{\sigma_i^l} (a_i^l \cup \mu_i^l) \quad \mu_i^l = \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} [a_i^l] \quad \sigma_i^l = \sqrt{\mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} [(a_i^l \cup \mu_i^l)^2]} \quad (2)$$

where $\bar{a}_i^l$ is normalized summed inputs to the i^{th} hidden unit in the l^{th} layer and g_i is a gain parameter scaling the normalized activation before the non-linear activation function. Note the expectation is under the whole training data distribution. It is typically impractical to compute the expectations in Eq. (2) exactly, since it would require forward passes through the whole training dataset with the current set of weights. Instead, μ and σ are estimated using the empirical samples from the current mini-batch. This puts constraints on the size of a mini-batch and it is hard to apply to recurrent neural networks.

3 Layer normalization

We now consider the layer normalization method which is designed to overcome the drawbacks of batch normalization.

Notice that changes in the output of one layer will tend to cause highly correlated changes in the summed inputs to the next layer, especially with ReLU units whose outputs can change by a lot. This suggests the ‘‘covariate shift’’ problem can be reduced by fixing the mean and the variance of the summed inputs within each layer. We, thus, compute the layer normalization statistics over all the hidden units in the same layer as follows:

$$\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l \cup \mu^l)^2} \quad (3)$$

where H denotes the number of hidden units in a layer. The difference between Eq. (2) and Eq. (3) is that under layer normalization, all the hidden units in a layer share the same normalization terms μ and σ , but different training cases have different normalization terms. Unlike batch normalization, layer normalization does not impose any constraint on the size of a mini-batch and it can be used in the pure online regime with batch size 1.

3.1 Layer normalized recurrent neural networks

The recent sequence to sequence models [Sutskever et al., 2014] utilize compact recurrent neural networks to solve sequential prediction problems in natural language processing. It is common among the NLP tasks to have different sentence lengths for different training cases. This is easy to deal with in an RNN because the same weights are used at every time-step. But when we apply batch normalization to an RNN in the obvious way, we need to compute and store separate statistics for each time step in a sequence. This is problematic if a test sequence is longer than any of the training sequences. Layer normalization does not have such problem because its normalization terms depend only on the summed inputs to a layer at the current time-step. It also has only one set of gain and bias parameters shared over all time-steps.

In a standard RNN, the summed inputs in the recurrent layer are computed from the current input $\mathbf{x}^t$ and previous vector of hidden states $\mathbf{h}^{t-1}$ which are computed as $\mathbf{a}^t = W_{hh}\mathbf{h}^{t-1} + W_{xh}\mathbf{x}^t$. The layer normalized recurrent layer re-centers and re-scales its activations using the extra normalization terms similar to Eq. (3):

$$\mathbf{h}^t = f \left[\frac{\mathbf{g}}{\sigma^t} \odot (\mathbf{a}^t \cup \mu^t) + \mathbf{b} \right] \quad \mu^t = \frac{1}{H} \sum_{i=1}^H a_i^t \quad \sigma^t = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^t \cup \mu^t)^2} \quad (4)$$

where W_{hh} is the recurrent hidden to hidden weights and W_{xh} are the bottom up input to hidden weights. $\odot$ is the element-wise multiplication between two vectors. $\mathbf{b}$ and $\mathbf{g}$ are defined as the bias and gain parameters of the same dimension as $\mathbf{h}^t$.

In a standard RNN, there is a tendency for the average magnitude of the summed inputs to the recurrent units to either grow or shrink at every time-step, leading to exploding or vanishing gradients. In a layer normalized RNN, the normalization terms make it invariant to re-scaling all of the summed inputs to a layer, which results in much more stable hidden-to-hidden dynamics.

4 Related work

Batch normalization has been previously extended to recurrent neural networks [Laurent et al., 2015, Amodei et al., 2015, Cooijmans et al., 2016]. The previous work [Cooijmans et al., 2016] suggests the best performance of recurrent batch normalization is obtained by keeping independent normalization statistics for each time step. The authors show that initializing the gain parameter in the recurrent batch normalization layer to 0.1 makes significant difference in the final performance of the model. Our work is also related to weight normalization [Salimans and Kingma, 2016]. In weight normalization, instead of the variance, the L2 norm of the incoming weights is used to normalize the summed inputs to a neuron. Applying either weight normalization or batch normalization using expected statistics is equivalent to have a different parameterization of the original feed-forward neural network. Re-parameterization in the ReLU network was studied in the Path-normalized SGD [Neyshabur et al., 2015]. Our proposed layer normalization method, however, is not a re-parameterization of the original neural network. The layer normalized model, thus, has different invariance properties than the other methods, that we will study in the following section.

5 Analysis

In this section, we investigate the invariance properties of different normalization schemes.

5.1 Invariance under weights and data transformations

The proposed layer normalization is related to batch normalization and weight normalization. Although, their normalization scalars are computed differently, these methods can be summarized as normalizing the summed inputs a_i to a neuron through the two scalars μ and σ . They also learn an adaptive bias b and gain g for each neuron after the normalization.

$$h_i = f \left(\frac{g_i}{\sigma_i} (a_i \cup \mu_i) + b_i \right) \quad (5)$$

Note that for layer normalization and batch normalization, μ and σ is computed according to Eq. 2 and 3. In weight normalization, μ is 0, and $\sigma = \|w\|_2$.

	Weight matrix re-scaling	Weight matrix re-centering	Weight vector re-scaling	Dataset re-scaling	Dataset re-centering	Single training case re-scaling
Batch norm	Invariant	No	Invariant	Invariant	Invariant	No
Weight norm	Invariant	No	Invariant	No	No	No
Layer norm	Invariant	Invariant	No	Invariant	No	Invariant

Table 1: Invariance properties under the normalization methods.

Table 1 highlights the following invariance results for three normalization methods.

Weight re-scaling and re-centering: First, observe that under batch normalization and weight normalization, any re-scaling to the incoming weights w_i of a single neuron has no effect on the normalized summed inputs to a neuron. To be precise, under batch and weight normalization, if the weight vector is scaled by δ , the two scalar μ and σ will also be scaled by δ . The normalized summed inputs stays the same before and after scaling. So the batch and weight normalization are invariant to the re-scaling of the weights. Layer normalization, on the other hand, is not invariant to the individual scaling of the single weight vectors. Instead, layer normalization is invariant to scaling of the entire weight matrix and invariant to a shift to all of the incoming weights in the weight matrix. Let there be two sets of model parameters θ, θ' whose weight matrices W and W' differ by a scaling factor δ and all of the incoming weights in W' are also shifted by a constant vector γ , that is $W' = \delta W + \mathbf{1}\gamma^\top$. Under layer normalization, the two models effectively compute the same output:

$$\begin{aligned} \mathbf{h}' &= f\left(\frac{\mathbf{g}}{\sigma'}(W'\mathbf{x} \cup \mu') + \mathbf{b}\right) = f\left(\frac{\mathbf{g}}{\sigma'}((\delta W + \mathbf{1}\gamma^\top)\mathbf{x} \cup \mu') + \mathbf{b}\right) \\ &= f\left(\frac{\mathbf{g}}{\sigma}(W\mathbf{x} \cup \mu) + \mathbf{b}\right) = \mathbf{h}. \end{aligned} \quad (6)$$

Notice that if normalization is only applied to the input before the weights, the model will not be invariant to re-scaling and re-centering of the weights.

Data re-scaling and re-centering: We can show that all the normalization methods are invariant to re-scaling the dataset by verifying that the summed inputs of neurons stays constant under the changes. Furthermore, layer normalization is invariant to re-scaling of individual training cases, because the normalization scalars μ and σ in Eq. (3) only depend on the current input data. Let $\mathbf{x}'$ be a new data point obtained by re-scaling $\mathbf{x}$ by δ . Then we have,

$$h'_i = f\left(\frac{g_i}{\sigma'}(w_i^\top \mathbf{x}' \cup \mu') + b_i\right) = f\left(\frac{g_i}{\delta\sigma}(\delta w_i^\top \mathbf{x} \cup \delta\mu) + b_i\right) = h_i. \quad (7)$$

It is easy to see re-scaling individual data points does not change the model's prediction under layer normalization. Similar to the re-centering of the weight matrix in layer normalization, we can also show that batch normalization is invariant to re-centering of the dataset.

5.2 Geometry of parameter space during learning

We have investigated the invariance of the model's prediction under re-centering and re-scaling of the parameters. Learning, however, can behave very differently under different parameterizations, even though the models express the same underlying function. In this section, we analyze learning behavior through the geometry and the manifold of the parameter space. We show that the normalization scalar σ can implicitly reduce learning rate and makes learning more stable.

5.2.1 Riemannian metric

The learnable parameters in a statistical model form a smooth manifold that consists of all possible input-output relations of the model. For models whose output is a probability distribution, a natural way to measure the separation of two points on this manifold is the Kullback-Leibler divergence between their model output distributions. Under the KL divergence metric, the parameter space is a Riemannian manifold.

The curvature of a Riemannian manifold is entirely captured by its Riemannian metric, whose quadratic form is denoted as ds^2 . That is the infinitesimal distance in the tangent space at a point in the parameter space. Intuitively, it measures the changes in the model output from the parameter space along a tangent direction. The Riemannian metric under KL was previously studied [Amari, 1998] and was shown to be well approximated under second order Taylor expansion using the Fisher

information matrix:

$$ds^2 = D_{\text{KL}}[P(y | \mathbf{x}; \theta) \| P(y | \mathbf{x}; \theta + \delta)] \approx \frac{1}{2} \delta^\top F(\theta) \delta, \quad (8)$$

$$F(\theta) = \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x}), y \sim P(y | \mathbf{x})} \left[\frac{\partial \log P(y | \mathbf{x}; \theta)}{\partial \theta} \frac{\partial \log P(y | \mathbf{x}; \theta)^\top}{\partial \theta} \right], \quad (9)$$

where, δ is a small change to the parameters. The Riemannian metric above presents a geometric view of parameter spaces. The following analysis of the Riemannian metric provides some insight into how normalization methods could help in training neural networks.

5.2.2 The geometry of normalized generalized linear models

We focus our geometric analysis on the generalized linear model. The results from the following analysis can be easily applied to understand deep neural networks with block-diagonal approximation to the Fisher information matrix, where each block corresponds to the parameters for a single neuron.

A generalized linear model (GLM) can be regarded as parameterizing an output distribution from the exponential family using a weight vector w and bias scalar b . To be consistent with the previous sections, the log likelihood of the GLM can be written using the summed inputs a as the following:

$$\log P(y | \mathbf{x}; w, b) = \frac{(a + b)y \cup \eta(a + b)}{\phi} + c(y, \phi), \quad (10)$$

$$\mathbb{E}[y | \mathbf{x}] = f(a + b) = f(w^\top \mathbf{x} + b), \quad \text{Var}[y | \mathbf{x}] = \phi f'(a + b), \quad (11)$$

where, $f(\cdot)$ is the transfer function that is the analog of the non-linearity in neural networks, $f'(\cdot)$ is the derivative of the transfer function, $\eta(\cdot)$ is a real valued function and $c(\cdot)$ is the log partition function. ϕ is a constant that scales the output variance. Assume a H -dimensional output vector $\mathbf{y} = [y_1, y_2, \dots, y_H]$ is modeled using H independent GLMs and $\log P(\mathbf{y} | \mathbf{x}; W, \mathbf{b}) = \sum_{i=1}^H \log P(y_i | \mathbf{x}; w_i, b_i)$. Let W be the weight matrix whose rows are the weight vectors of the individual GLMs, $\mathbf{b}$ denote the bias vector of length H and $\text{vec}(\cdot)$ denote the Kronecker vector operator. The Fisher information matrix for the multi-dimensional GLM with respect to its parameters $\theta = [w_1^\top, b_1, \dots, w_H^\top, b_H]^\top = \text{vec}([W, \mathbf{b}]^\top)$ is simply the expected Kronecker product of the data features and the output covariance matrix:

$$F(\theta) = \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\frac{\text{Cov}[\mathbf{y} | \mathbf{x}]}{\phi^2} \otimes \begin{bmatrix} \mathbf{x} \mathbf{x}^\top & \mathbf{x} \\ \mathbf{x}^\top & 1 \end{bmatrix} \right]. \quad (12)$$

We obtain normalized GLMs by applying the normalization methods to the summed inputs a in the original model through μ and σ . Without loss of generality, we denote $\bar{F}$ as the Fisher information matrix under the normalized multi-dimensional GLM with the additional gain parameters $\theta = \text{vec}([W, \mathbf{b}, \mathbf{g}]^\top)$:

$$\bar{F}(\theta) = \begin{bmatrix} \bar{F}_{11} & \cdots & \bar{F}_{1H} \\ \vdots & \ddots & \vdots \\ \bar{F}_{H1} & \cdots & \bar{F}_{HH} \end{bmatrix}, \quad \bar{F}_{ij} = \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\frac{\text{Cov}[y_i, y_j | \mathbf{x}]}{\phi^2} \begin{bmatrix} \frac{g_i g_j}{\sigma_i \sigma_j} \chi_i \chi_j^\top & \chi_i \frac{g_i}{\sigma_i} & \chi_i \frac{g_i (a_j | \mu_j)}{\sigma_i \sigma_j} \\ \chi_j^\top \frac{g_j}{\sigma_j} & 1 & \frac{a_j | \mu_j}{\sigma_j} \\ \chi_j^\top \frac{g_j (a_i | \mu_i)}{\sigma_i \sigma_j} & \frac{a_i | \mu_i}{\sigma_i} & \frac{(a_i | \mu_i)(a_j | \mu_j)}{\sigma_i \sigma_j} \end{bmatrix} \right] \quad (13)$$

$$\chi_i = \mathbf{x} \cup \frac{\partial \mu_i}{\partial w_i} \cup \frac{a_i \cup \mu_i}{\sigma_i} \frac{\partial \sigma_i}{\partial w_i}. \quad (14)$$

Implicit learning rate reduction through the growth of the weight vector: Notice that, comparing to standard GLM, the block $\bar{F}_{ij}$ along the weight vector w_i direction is scaled by the gain parameters and the normalization scalar σ_i . If the norm of the weight vector w_i grows twice as large, even though the model's output remains the same, the Fisher information matrix will be different. The curvature along the w_i direction will change by a factor of $\frac{1}{2}$ because the σ_i will also be twice as large. As a result, for the same parameter update in the normalized model, the norm of the weight vector effectively controls the learning rate for the weight vector. During learning, it is harder to change the orientation of the weight vector with large norm. The normalization methods, therefore,

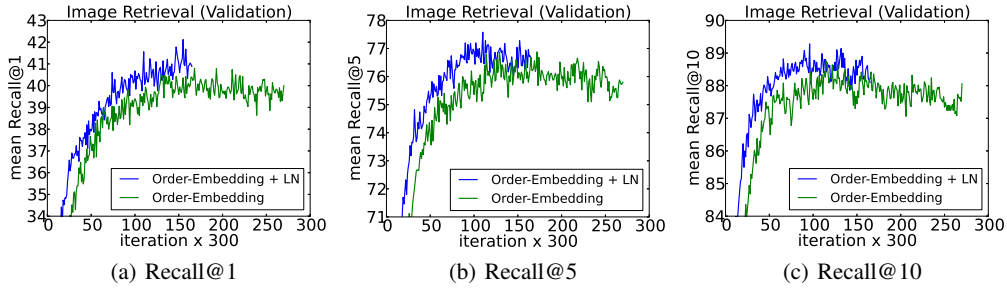


Figure 1: Recall@K curves using order-embeddings with and without layer normalization.

MSCOCO								
Model	Caption Retrieval				Image Retrieval			
	R@1	R@5	R@10	Mean r	R@1	R@5	R@10	Mean r
Sym [Vendrov et al., 2016]	45.4		88.7	5.8	36.3		85.8	9.0
OE [Vendrov et al., 2016]	46.7		88.9	5.7	37.9		85.9	8.1
OE (ours)	46.6	79.3	89.1	5.2	37.8	73.6	85.7	7.9
OE + LN	48.5	80.6	89.8	5.1	38.9	74.3	86.3	7.6

Table 2: Average results across 5 test splits for caption and image retrieval. **R@K** is Recall@K (high is good). **Mean r** is the mean rank (low is good). Sym corresponds to the symmetric baseline while OE indicates order-embeddings.

have an implicit “early stopping” effect on the weight vectors and help to stabilize learning towards convergence.

Learning the magnitude of incoming weights: In normalized models, the magnitude of the incoming weights is explicitly parameterized by the gain parameters. We compare how the model output changes between updating the gain parameters in the normalized GLM and updating the magnitude of the equivalent weights under original parameterization during learning. The direction along the gain parameters in $\bar{F}$ captures the geometry for the magnitude of the incoming weights. We show that Riemannian metric along the magnitude of the incoming weights for the standard GLM is scaled by the norm of its input, whereas learning the gain parameters for the batch normalized and layer normalized models depends only on the magnitude of the prediction error. Learning the magnitude of incoming weights in the normalized model is therefore, more robust to the scaling of the input and its parameters than in the standard model. See Appendix for detailed derivations.

6 Experimental results

We perform experiments with layer normalization on 6 tasks, with a focus on recurrent neural networks: image-sentence ranking, question-answering, contextual language modelling, generative modelling, handwriting sequence generation and MNIST classification. Unless otherwise noted, the default initialization of layer normalization is to set the adaptive gains to 1 and the biases to 0 in the experiments.

6.1 Order embeddings of images and language

In this experiment, we apply layer normalization to the recently proposed order-embeddings model of [Vendrov et al., 2016] for learning a joint embedding space of images and sentences. We follow the same experimental protocol as [Vendrov et al., 2016] and modify their publicly available code to incorporate layer normalization which utilizes Theano [Team et al., 2016]. Images and sentences from the Microsoft COCO dataset [Lin et al., 2014] are embedded into a common vector space, where a GRU [Cho et al., 2014] is used to encode sentences and the outputs of a pre-trained VGG ConvNet [Simonyan and Zisserman, 2015] (10-crop) are used to encode images. The order-embedding model represents images and sentences as a 2-level partial ordering and replaces the cosine similarity scoring function used in [Kiros et al., 2014] with an asymmetric one.

<https://github.com/ivendrov/order-embedding>

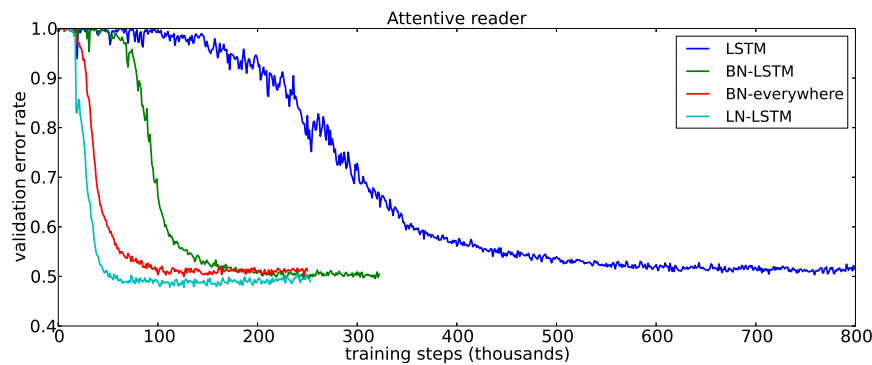


Figure 2: Validation curves for the attentive reader model. BN results are taken from [Cooijmans et al., 2016].

We trained two models: the baseline order-embedding model as well as the same model with layer normalization applied to the GRU. After every 300 iterations, we compute Recall@K (R@K) values on a held out validation set and save the model whenever R@K improves. The best performing models are then evaluated on 5 separate test sets, each containing 1000 images and 5000 captions, for which the mean results are reported. Both models use Adam [Kingma and Ba, 2014] with the same initial hyperparameters and both models are trained using the same architectural choices as used in [Vendrov et al., 2016]. We refer the reader to the appendix for a description of how layer normalization is applied to GRU.

Figure 1 illustrates the validation curves of the models, with and without layer normalization. We plot R@1, R@5 and R@10 for the image retrieval task. We observe that layer normalization offers a per-iteration speedup across all metrics and converges to its best validation model in 60% of the time it takes the baseline model to do so. In Table 2 the test set results are reported from which we observe that layer normalization also results in improved generalization over the original model. The results we report are state-of-the-art for RNN embedding models, with only the structure-preserving model of [Wang et al., 2016] reporting better results on this task. However, they evaluate under different conditions (1 test set instead of the mean over 5) and are thus not directly comparable.

6.2 Teaching machines to read and comprehend

In order to compare layer normalization to the recently proposed recurrent batch normalization [Cooijmans et al., 2016], we train an unidirectional attentive reader model on the CNN corpus both introduced by [Hermann et al., 2015]. This is a question-answering task where a query description about a passage must be answered by filling in a blank. The data is anonymized such that entities are given randomized tokens to prevent degenerate solutions, which are consistently permuted during training and evaluation. We follow the same experimental protocol as [Cooijmans et al., 2016] and modify their public code to incorporate layer normalization² which uses Theano [Team et al., 2016]. We obtained the pre-processed dataset used by [Cooijmans et al., 2016] which differs from the original experiments of [Hermann et al., 2015] in that each passage is limited to 4 sentences. In [Cooijmans et al., 2016], two variants of recurrent batch normalization are used: one where BN is only applied to the LSTM while the other applies BN everywhere throughout the model. In our experiment, we only apply layer normalization within the LSTM.

The results of this experiment are shown in Figure 2. We observe that layer normalization not only trains faster but converges to a better validation result over both the baseline and BN variants. In [Cooijmans et al., 2016], it is argued that the scale parameter in BN must be carefully chosen and is set to 0.1 in their experiments. We experimented with layer normalization for both 1.0 and 0.1 scale initialization and found that the former model performed significantly better. This demonstrates that layer normalization is not sensitive to the initial scale in the same way that recurrent BN is.³

6.3 Skip-thought vectors

Skip-thoughts [Kiros et al., 2015] is a generalization of the skip-gram model [Mikolov et al., 2013] for learning unsupervised distributed sentence representations. Given contiguous text, a sentence is

²https://github.com/cooijmanstim/Attentive_reader/tree/bn

³We only produce results on the validation set, as in the case of [Cooijmans et al., 2016]

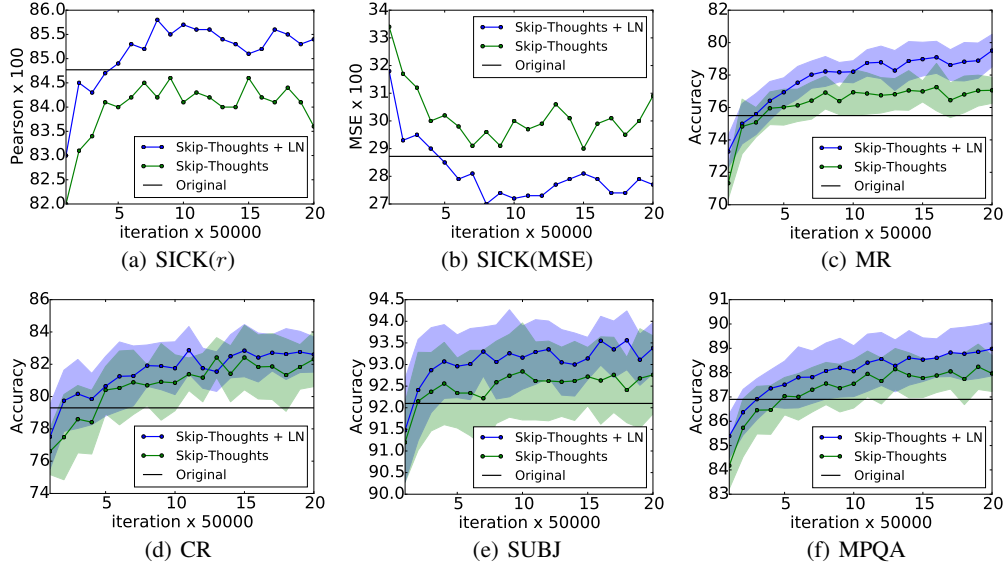


Figure 3: Performance of skip-thought vectors with and without layer normalization on downstream tasks as a function of training iterations. The original lines are the reported results in [Kiros et al., 2015]. Plots with error use 10-fold cross validation. Best seen in color.

Method	SICK(r)	SICK(ρ)	SICK(MSE)	MR	CR	SUBJ	MPQA
Original [Kiros et al., 2015]	0.848	0.778	0.287	75.5	79.3	92.1	86.9
Ours	0.842	0.767	0.298	77.3	81.8	92.6	87.9
Ours + LN	0.854	0.785	0.277	79.5	82.6	93.4	89.0
Ours + LN [†]	0.858	0.788	0.270	79.4	83.1	93.7	89.3

Table 3: Skip-thoughts results. The first two evaluation columns indicate Pearson and Spearman correlation, the third is mean squared error and the remaining indicate classification accuracy. Higher is better for all evaluations except MSE. Our models were trained for 1M iterations with the exception of ([†]) which was trained for 1 month (approximately 1.7M iterations)

encoded with a encoder RNN and decoder RNNs are used to predict the surrounding sentences. [Kiros et al., 2015] showed that this model could produce generic sentence representations that perform well on several tasks without being fine-tuned. However, training this model is time-consuming, requiring several days of training in order to produce meaningful results.

In this experiment we determine to what effect layer normalization can speed up training. Using the publicly available code of [Kiros et al., 2015]⁴, we train two models on the BookCorpus dataset [Zhu et al., 2015]: one with and one without layer normalization. These experiments are performed with Theano [Team et al., 2016]. We adhere to the experimental setup used in [Kiros et al., 2015], training a 2400-dimensional sentence encoder with the same hyperparameters. Given the size of the states used, it is conceivable layer normalization would produce slower per-iteration updates than without. However, we found that provided CNMeM⁵ is used, there was no significant difference between the two models. We checkpoint both models after every 50,000 iterations and evaluate their performance on five tasks: semantic-relatedness (SICK) [Marelli et al., 2014], movie review sentiment (MR) [Pang and Lee, 2005], customer product reviews (CR) [Hu and Liu, 2004], subjectivity/objectivity classification (SUBJ) [Pang and Lee, 2004] and opinion polarity (MPQA) [Wiebe et al., 2005]. We plot the performance of both models for each checkpoint on all tasks to determine whether the performance rate can be improved with LN.

The experimental results are illustrated in Figure 3. We observe that applying layer normalization results both in speedup over the baseline as well as better final results after 1M iterations are performed as shown in Table 3. We also let the model with layer normalization train for a total of a month, resulting in further performance gains across all but one task. We note that the performance

⁴<https://github.com/ryankiros/skip-thoughts>

⁵<https://github.com/NVIDIA/cnmem>

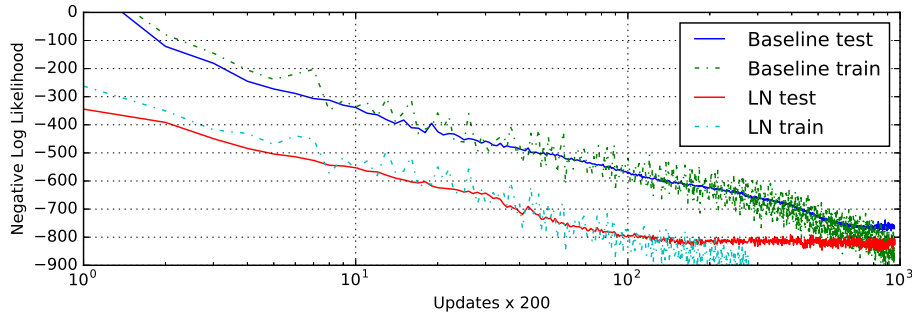


Figure 5: Handwriting sequence generation model negative log likelihood with and without layer normalization. The models are trained with mini-batch size of 8 and sequence length of 500,

differences between the original reported results and ours are likely due to the fact that the publicly available code does not condition at each timestep of the decoder, where the original model does.

6.4 Modeling binarized MNIST using DRAW

We also experimented with the generative modeling on the MNIST dataset. Deep Recurrent Attention Writer (DRAW) [Gregor et al., 2015] has previously achieved the state-of-the-art performance on modeling the distribution of MNIST digits. The model uses a differential attention mechanism and a recurrent neural network to sequentially generate pieces of an image. We evaluate the effect of layer normalization on a DRAW model using 64 glimpses and 256 LSTM hidden units. The model is trained with the default setting of Adam [Kingma and Ba, 2014] optimizer and the minibatch size of 128. Previous publications on binarized MNIST have used various training protocols to generate their datasets. In this experiment, we used the fixed binarization from [Larochelle and Murray, 2011]. The dataset has been split into 50,000 training, 10,000 validation and 10,000 test images.

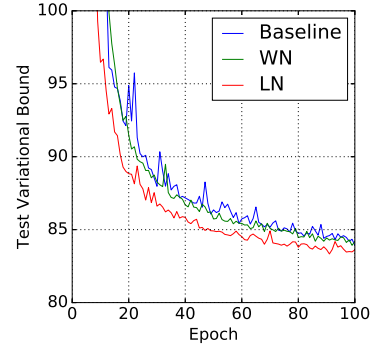


Figure 4: DRAW model test negative log likelihood with and without layer normalization.

Figure 4 shows the test variational bound for the first 100 epoch. It highlights the speedup benefit of applying layer normalization that the layer normalized DRAW converges almost twice as fast than the baseline model. After 200 epoches, the baseline model converges to a variational log likelihood of 82.36 nats on the test data and the layer normalization model obtains 82.09 nats.

6.5 Handwriting sequence generation

The previous experiments mostly examine RNNs on NLP tasks whose lengths are in the range of 10 to 40. To show the effectiveness of layer normalization on longer sequences, we performed handwriting generation tasks using the IAM Online Handwriting Database [Liwicki and Bunke, 2005]. IAM-OnDB consists of handwritten lines collected from 221 different writers. When given the input character string, the goal is to predict a sequence of x and y pen co-ordinates of the corresponding handwriting line on the whiteboard. There are, in total, 12179 handwriting line sequences. The input string is typically more than 25 characters and the average handwriting line has a length around 700.

We used the same model architecture as in Section (5.2) of [Graves, 2013]. The model architecture consists of three hidden layers of 400 LSTM cells, which produce 20 bivariate Gaussian mixture components at the output layer, and a size 3 input layer. The character sequence was encoded with one-hot vectors, and hence the window vectors were size 57. A mixture of 10 Gaussian functions was used for the window parameters, requiring a size 30 parameter vector. The total number of weights was increased to approximately 3.7M. The model is trained using mini-batches of size 8 and the Adam [Kingma and Ba, 2014] optimizer.

The combination of small mini-batch size and very long sequences makes it important to have very stable hidden dynamics. Figure 5 shows that layer normalization converges to a comparable log likelihood as the baseline model but is much faster.

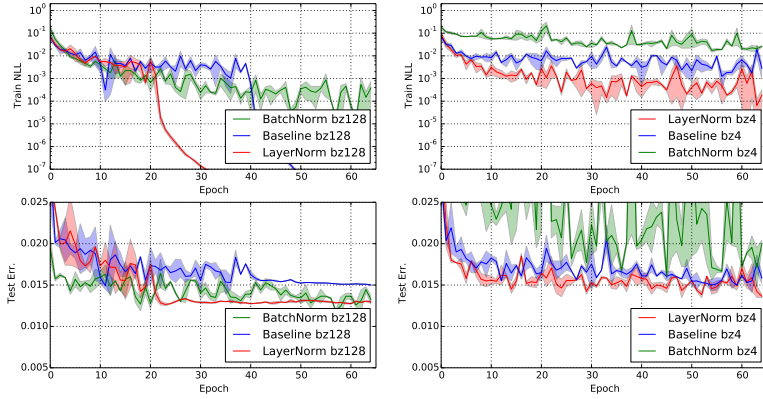


Figure 6: Permutation invariant MNIST 784-1000-1000-10 model negative log likelihood and test error with layer normalization and batch normalization. (Left) The models are trained with batch-size of 128. (Right) The models are trained with batch-size of 4.

6.6 Permutation invariant MNIST

In addition to RNNs, we investigated layer normalization in feed-forward networks. We show how layer normalization compares with batch normalization on the well-studied permutation invariant MNIST classification problem. From the previous analysis, layer normalization is invariant to input re-scaling which is desirable for the internal hidden layers. But this is unnecessary for the logit outputs where the prediction confidence is determined by the scale of the logits. We only apply layer normalization to the fully-connected hidden layers that excludes the last softmax layer.

All the models were trained using 55000 training data points and the Adam [Kingma and Ba, 2014] optimizer. For the smaller batch-size, the variance term for batch normalization is computed using the unbiased estimator. The experimental results from Figure 6 highlight that layer normalization is robust to the batch-sizes and exhibits a faster training convergence comparing to batch normalization that is applied to all layers.

□

6.7 Convolutional Networks

We have also experimented with convolutional neural networks. In our preliminary experiments, we observed that layer normalization offers a speedup over the baseline model without normalization, but batch normalization outperforms the other methods. With fully connected layers, all the hidden units in a layer tend to make similar contributions to the final prediction and re-centering and re-scaling the summed inputs to a layer works well. However, the assumption of similar contributions is no longer true for convolutional neural networks. The large number of the hidden units whose receptive fields lie near the boundary of the image are rarely turned on and thus have very different statistics from the rest of the hidden units within the same layer. We think further research is needed to make layer normalization work well in ConvNets.

7 Conclusion

In this paper, we introduced layer normalization to speed-up the training of neural networks. We provided a theoretical analysis that compared the invariance properties of layer normalization with batch normalization and weight normalization. We showed that layer normalization is invariant to per training-case feature shifting and scaling.

Empirically, we showed that recurrent neural networks benefit the most from the proposed method especially for long sequences and small mini-batches.

Acknowledgments

This research was funded by grants from NSERC, CFI, and Google.

Supplementary Material

Application of layer normalization to each experiment

This section describes how layer normalization is applied to each of the papers' experiments. For notation convenience, we define layer normalization as a function mapping $LN : \mathbb{R}^D \rightarrow \mathbb{R}^D$ with two set of adaptive parameters, gains α and biases β :

$$LN(\mathbf{z}; \alpha, \beta) = \frac{(\mathbf{z} \cup \mu)}{\sigma} \odot \alpha + \beta, \quad (15)$$

$$\mu = \frac{1}{D} \sum_{i=1}^D z_i, \quad \sigma = \sqrt{\frac{1}{D} \sum_{i=1}^D (z_i \cup \mu)^2}, \quad (16)$$

where, z_i is the i^{th} element of the vector $\mathbf{z}$.

Teaching machines to read and comprehend and handwriting sequence generation

The basic LSTM equations used for these experiment are given by:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + b \quad (17)$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \quad (18)$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(\mathbf{c}_t) \quad (19)$$

The version that incorporates layer normalization is modified as follows:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \alpha_1, \beta_1) + LN(\mathbf{W}_x \mathbf{x}_t; \alpha_2, \beta_2) + b \quad (20)$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \quad (21)$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(LN(\mathbf{c}_t; \alpha_3, \beta_3)) \quad (22)$$

where α_i, β_i are the additive and multiplicative parameters, respectively. Each α_i is initialized to a vector of zeros and each β_i is initialized to a vector of ones.

Order embeddings and skip-thoughts

These experiments utilize a variant of gated recurrent unit which is defined as follows:

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t \quad (23)$$

$$\hat{\mathbf{h}}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \sigma(\mathbf{r}_t) \odot (\mathbf{U} \mathbf{h}_{t-1})) \quad (24)$$

$$\mathbf{h}_t = (1 \cup \sigma(\mathbf{z}_t)) \mathbf{h}_{t-1} + \sigma(\mathbf{z}_t) \hat{\mathbf{h}}_t \quad (25)$$

Layer normalization is applied as follows:

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \alpha_1, \beta_1) + LN(\mathbf{W}_x \mathbf{x}_t; \alpha_2, \beta_2) \quad (26)$$

$$\hat{\mathbf{h}}_t = \tanh(LN(\mathbf{W}_x \mathbf{x}_t; \alpha_3, \beta_3) + \sigma(\mathbf{r}_t) \odot LN(\mathbf{U} \mathbf{h}_{t-1}; \alpha_4, \beta_4)) \quad (27)$$

$$\mathbf{h}_t = (1 \cup \sigma(\mathbf{z}_t)) \mathbf{h}_{t-1} + \sigma(\mathbf{z}_t) \hat{\mathbf{h}}_t \quad (28)$$

just as before, α_i is initialized to a vector of zeros and each β_i is initialized to a vector of ones.

Modeling binarized MNIST using DRAW

The layer norm is only applied to the output of the LSTM hidden states in this experiment:

The version that incorporates layer normalization is modified as follows:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + b \quad (29)$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \quad (30)$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(LN(\mathbf{c}_t; \boldsymbol{\alpha}, \boldsymbol{\beta})) \quad (31)$$

where $\boldsymbol{\alpha}, \boldsymbol{\beta}$ are the additive and multiplicative parameters, respectively. $\boldsymbol{\alpha}$ is initialized to a vector of zeros and $\boldsymbol{\beta}$ is initialized to a vector of ones.

Learning the magnitude of incoming weights

We now compare how gradient descent updates changing magnitude of the equivalent weights between the normalized GLM and original parameterization. The magnitude of the weights are explicitly parameterized using the gain parameter in the normalized model. Assume there is a gradient update that changes norm of the weight vectors by δ_g . We can project the gradient updates to the weight vector for the normal GLM. The KL metric, ie how much the gradient update changes the model prediction, for the normalized model depends only on the magnitude of the prediction error. Specifically,

under batch normalization:

$$ds^2 = \frac{1}{2} \text{vec}([0, 0, \delta_g]^\top)^\top \bar{F}(\text{vec}([W, \mathbf{b}, \mathbf{g}]^\top) \text{vec}([0, 0, \delta_g]^\top) = \frac{1}{2} \delta_g^\top \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\frac{\text{Cov}[\mathbf{y} | \mathbf{x}]}{\phi^2} \right] \delta_g. \quad (32)$$

Under layer normalization:

$$\begin{aligned} ds^2 &= \frac{1}{2} \text{vec}([0, 0, \delta_g]^\top)^\top \bar{F}(\text{vec}([W, \mathbf{b}, \mathbf{g}]^\top) \text{vec}([0, 0, \delta_g]^\top) \\ &= \frac{1}{2} \delta_g^\top \frac{1}{\phi^2} \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} \begin{bmatrix} \text{Cov}(y_1, y_1 | \mathbf{x}) \frac{(a_1 \lfloor \mu \rfloor)^2}{\sigma^2} & \cdots & \text{Cov}(y_1, y_H | \mathbf{x}) \frac{(a_1 \lfloor \mu \rfloor)(a_H \lfloor \mu \rfloor)}{\sigma^2} \\ \vdots & \ddots & \vdots \\ \text{Cov}(y_H, y_1 | \mathbf{x}) \frac{(a_H \lfloor \mu \rfloor)(a_1 \lfloor \mu \rfloor)}{\sigma^2} & \cdots & \text{Cov}(y_H, y_H | \mathbf{x}) \frac{(a_H \lfloor \mu \rfloor)^2}{\sigma^2} \end{bmatrix} \delta_g \end{aligned} \quad (33)$$

Under weight normalization:

$$\begin{aligned} ds^2 &= \frac{1}{2} \text{vec}([0, 0, \delta_g]^\top)^\top \bar{F}(\text{vec}([W, \mathbf{b}, \mathbf{g}]^\top) \text{vec}([0, 0, \delta_g]^\top) \\ &= \frac{1}{2} \delta_g^\top \frac{1}{\phi^2} \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} \begin{bmatrix} \text{Cov}(y_1, y_1 | \mathbf{x}) \frac{a_1^2}{\|w_1\|_2^2} & \cdots & \text{Cov}(y_1, y_H | \mathbf{x}) \frac{a_1 a_H}{\|w_1\|_2 \|w_H\|_2} \\ \vdots & \ddots & \vdots \\ \text{Cov}(y_H, y_1 | \mathbf{x}) \frac{a_H a_1}{\|w_H\|_2 \|w_1\|_2} & \cdots & \text{Cov}(y_H, y_H | \mathbf{x}) \frac{a_H^2}{\|w_H\|_2^2} \end{bmatrix} \delta_g. \end{aligned} \quad (34)$$

Whereas, the KL metric in the standard GLM is related to its activities $a_i = w_i^\top \mathbf{x}$, that is depended on both its current weights and input data. We project the gradient updates to the gain parameter δ_{gi} of the i^{th} neuron to its weight vector as $\delta_{gi} \frac{w_i}{\|w_i\|_2}$ in the standard GLM model:

$$\begin{aligned} & \frac{1}{2} \text{vec}([\delta_{gi} \frac{w_i}{\|w_i\|_2}, 0, \delta_{gj} \frac{w_j}{\|w_j\|_2}, 0]^\top)^\top F([w_i^\top, b_i, w_j^\top, b_j]^\top) \text{vec}([\delta_{gi} \frac{w_i}{\|w_i\|_2}, 0, \delta_{gj} \frac{w_j}{\|w_j\|_2}, 0]^\top) \\ &= \frac{\delta_{gi} \delta_{gj}}{2\phi^2} \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\text{Cov}(y_i, y_j | \mathbf{x}) \frac{a_i a_j}{\|w_i\|_2 \|w_j\|_2} \right] \end{aligned} \quad (35)$$

The batch normalized and layer normalized models are therefore more robust to the scaling of the input and its parameters than the standard model.

Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks

Tim Salimans
OpenAI
tim@openai.com

Diederik P. Kingma
OpenAI
dpkingma@openai.com

Abstract

We present *weight normalization*: a reparameterization of the weight vectors in a neural network that decouples the length of those weight vectors from their direction. By reparameterizing the weights in this way we improve the conditioning of the optimization problem and we speed up convergence of stochastic gradient descent. Our reparameterization is inspired by batch normalization but does not introduce any dependencies between the examples in a minibatch. This means that our method can also be applied successfully to recurrent models such as LSTMs and to noise-sensitive applications such as deep reinforcement learning or generative models, for which batch normalization is less well suited. Although our method is much simpler, it still provides much of the speed-up of full batch normalization. In addition, the computational overhead of our method is lower, permitting more optimization steps to be taken in the same amount of time. We demonstrate the usefulness of our method on applications in supervised image recognition, generative modelling, and deep reinforcement learning.

1 Introduction

Recent successes in deep learning have shown that neural networks trained by first-order gradient based optimization are capable of achieving amazing results in diverse domains like computer vision, speech recognition, and language modelling [5]. However, it is also well known that the practical success of first-order gradient based optimization is highly dependent on the curvature of the objective that is optimized. If the condition number of the Hessian matrix of the objective at the optimum is low, the problem is said to exhibit *pathological curvature*, and first-order gradient descent will have trouble making progress [18, 28]. The amount of curvature, and thus the success of our optimization, is not invariant to reparameterization [1]: there may be multiple equivalent ways of parameterizing the same model, some of which are much easier to optimize than others. Finding good ways of parameterizing neural networks is thus an important problem in deep learning.

While the architectures of neural networks differ widely across applications, they are typically mostly composed of conceptually simple computational building blocks sometimes called neurons: each such neuron computes a *weighted sum* over its inputs and adds a *bias* term, followed by the application of an elementwise nonlinear transformation. Improving the general optimizability of deep networks is a challenging task [4], but since many neural architectures share these basic building blocks, improving these building blocks improves the performance of a very wide range of model architectures and could thus be very useful.

Several authors have recently developed methods to improve the conditioning of the cost gradient for general neural network architectures. One approach is to explicitly left multiply the cost gradient with an approximate inverse of the Fisher information matrix, thereby obtaining an approximately whitened *natural gradient*. Such an approximate inverse can for example be obtained by using a Kronecker factored approximation to the Fisher matrix and inverting it (KFAC, [19]), by using an



approximate Cholesky factorization of the inverse Fisher matrix (FANG, [8]), or by whitening the input of each layer in the neural network (PRONG, [3]).

Alternatively, we can use standard first order gradient descent without preconditioning, but change the parameterization of our model to give gradients that are more like the whitened natural gradients of these methods. For example, Raiko et al. [23] propose to transform the outputs of each neuron to have zero output and zero slope on average. They show that this transformation approximately diagonalizes the Fisher information matrix, thereby whitening the gradient, and that this leads to improved optimization performance. Another approach in this direction is *batch normalization* [11], a method where the output of each neuron (before application of the nonlinearity) is normalized by the mean and standard deviation of the outputs calculated over the examples in the minibatch. This reduces covariate shift of the neuron outputs and the authors suggest it also brings the Fisher matrix closer to the identity matrix.

Following this second approach to approximate natural gradient optimization, we propose a simple but general method, called *weight normalization*, for improving the optimizability of the weights of neural network models. The method is inspired by batch normalization, but it is a deterministic method that does not share batch normalization’s property of adding noise to the gradients. In addition, the overhead imposed by our method is lower: no additional memory is required and the additional computation is negligible. The method show encouraging results on a wide range of deep learning applications.

2 Weight Normalization

We consider standard artificial neural networks where the computation of each neuron consists in taking a weighted sum of input features, followed by an elementwise nonlinearity:

$$y = \phi(\mathbf{w} \cdot \mathbf{x} + b), \quad (1)$$

where $\mathbf{w}$ is a k -dimensional weight vector, b is a scalar bias term, $\mathbf{x}$ is a k -dimensional vector of input features, $\phi(\cdot)$ denotes an elementwise nonlinearity such as the rectifier $\max(\cdot, 0)$, and y denotes the scalar output of the neuron.

After associating a loss function to one or more neuron outputs, such a neural network is commonly trained by stochastic gradient descent in the parameters $\mathbf{w}$, b of each neuron. In an effort to speed up the convergence of this optimization procedure, we propose to reparameterize each weight vector $\mathbf{w}$ in terms of a parameter vector $\mathbf{v}$ and a scalar parameter g and to perform stochastic gradient descent with respect to those parameters instead. We do so by expressing the weight vectors in terms of the new parameters using

$$\mathbf{w} = \frac{g}{\|\mathbf{v}\|} \mathbf{v} \quad (2)$$

where $\mathbf{v}$ is a k -dimensional vector, g is a scalar, and $\|\mathbf{v}\|$ denotes the Euclidean norm of $\mathbf{v}$. This reparameterization has the effect of fixing the Euclidean norm of the weight vector $\mathbf{w}$: we now have $\|\mathbf{w}\| = g$, independent of the parameters $\mathbf{v}$. We therefore call this reparameterization *weight normalization*.

The idea of normalizing the weight vector has been proposed before (e.g. [27]) but earlier work typically still performed optimization in the $\mathbf{w}$ -parameterization, only applying the normalization after each step of stochastic gradient descent. This is fundamentally different from our approach: we propose to explicitly reparameterize the model and to perform stochastic gradient descent in the new parameters $\mathbf{v}$, g directly. Doing so improves the conditioning of the gradient and leads to improved convergence of the optimization procedure: By decoupling the norm of the weight vector (g) from the direction of the weight vector ($\mathbf{v}/\|\mathbf{v}\|$), we speed up convergence of our stochastic gradient descent optimization, as we show experimentally in section 5.

Instead of working with g directly, we may also use an exponential parameterization for the scale, i.e. $g = e^s$, where s is a log-scale parameter to learn by stochastic gradient descent. Parameterizing the g parameter in the log-scale is more intuitive and more easily allows g to span a wide range of different magnitudes. Empirically, however, we did not find this to be an advantage. In our experiments, the eventual test-set performance was not significantly better or worse than the results with directly learning g in its original parameterization, and optimization was slightly slower.

2.1 Gradients

Training a neural network in the new parameterization is done using standard stochastic gradient descent methods. Here we differentiate through (2) to obtain the gradient of a loss function L with respect to the new parameters $\mathbf{v}, g$. Doing so gives

$$\nabla_g L = \frac{\nabla_{\mathbf{w}} L \cdot \mathbf{v}}{\|\mathbf{v}\|}, \quad \nabla_{\mathbf{v}} L = \frac{g}{\|\mathbf{v}\|} \nabla_{\mathbf{w}} L - \frac{g \nabla_g L}{\|\mathbf{v}\|^2} \mathbf{v}, \quad (3)$$

where $\nabla_{\mathbf{w}} L$ is the gradient with respect to the weights $\mathbf{w}$ as used normally.

Backpropagation using weight normalization thus only requires a minor modification to the usual backpropagation equations, and is easily implemented using standard neural network software. We provide reference implementations for Theano at https://github.com/TimSalimans/weight_norm. Unlike with batch normalization, the expressions above are independent of the minibatch size and thus cause only minimal computational overhead.

An alternative way to write the gradient is

$$\nabla_{\mathbf{v}} L = \frac{g}{\|\mathbf{v}\|} M_{\mathbf{w}} \nabla_{\mathbf{w}} L, \quad \text{with} \quad M_{\mathbf{w}} = \mathbf{I} - \frac{\mathbf{w} \mathbf{w}'}{\|\mathbf{w}\|^2}, \quad (4)$$

where $M_{\mathbf{w}}$ is a projection matrix that projects onto the complement of the $\mathbf{w}$ vector. This shows that weight normalization accomplishes two things: it *scales* the weight gradient by $g/\|\mathbf{v}\|$, and it *projects* the gradient away from the current weight vector. Both effects help to bring the covariance matrix of the gradient closer to identity and benefit optimization, as we explain below.

Due to projecting away from $\mathbf{w}$, the norm of $\mathbf{v}$ grows monotonically with the number of weight updates when learning a neural network with weight normalization using standard gradient descent without momentum: Let $\mathbf{v}' = \mathbf{v} + \Delta \mathbf{v}$ denote our parameter update, with $\Delta \mathbf{v} \propto \nabla_{\mathbf{v}} L$ (steepest ascent/descent), then $\Delta \mathbf{v}$ is necessarily orthogonal to the current weight vector $\mathbf{w}$ since we project away from it when calculating $\nabla_{\mathbf{v}} L$ (equation 4). Since $\mathbf{v}$ is proportional to $\mathbf{w}$, the update is thus also orthogonal to $\mathbf{v}$ and increases its norm by the Pythagorean theorem. Specifically, if $\|\Delta \mathbf{v}\|/\|\mathbf{v}\| = c$ the new weight vector will have norm $\|\mathbf{v}'\| = \sqrt{\|\mathbf{v}\|^2 + c^2 \|\mathbf{v}\|^2} = \sqrt{1 + c^2} \|\mathbf{v}\| \geq \|\mathbf{v}\|$. The rate of increase will depend on the variance of the weight gradient. If our gradients are noisy, c will be high and the norm of $\mathbf{v}$ will quickly increase, which in turn will decrease the scaling factor $g/\|\mathbf{v}\|$. If the norm of the gradients is small, we get $\sqrt{1 + c^2} \approx 1$, and the norm of $\mathbf{v}$ will stop increasing. Using this mechanism, the scaled gradient self-stabilizes its norm. This property does not strictly hold for optimizers that use separate learning rates for individual parameters, like Adam [12] which we use in experiments, or when using momentum. However, qualitatively we still find the same effect to hold.

Empirically, we find that the ability to grow the norm $\|\mathbf{v}\|$ makes optimization of neural networks with weight normalization very robust to the value of the learning rate: If the learning rate is too large, the norm of the unnormalized weights grows quickly until an appropriate effective learning rate is reached. Once the norm of the weights has grown large with respect to the norm of the updates, the effective learning rate stabilizes. Neural networks with weight normalization therefore work well with a much wider range of learning rates than when using the normal parameterization. It has been observed that neural networks with batch normalization also have this property [11], which can also be explained by this analysis.

By projecting the gradient away from the weight vector $\mathbf{w}$, we also eliminate the noise in that direction. If the covariance matrix of the gradient with respect to $\mathbf{w}$ is given by $\mathbf{C}$, the covariance matrix of the gradient in $\mathbf{v}$ is given by $\mathbf{D} = (g^2/\|\mathbf{v}\|^2) M_{\mathbf{w}} \mathbf{C} M_{\mathbf{w}}$. Empirically, we find that $\mathbf{w}$ is often (close to) a dominant eigenvector of the covariance matrix $\mathbf{C}$: removing that eigenvector then gives a new covariance matrix $\mathbf{D}$ that is closer to the identity matrix, which may further speed up learning.

2.2 Relation to batch normalization

An important source of inspiration for this reparameterization is *batch normalization* [11], which normalizes the statistics of the pre-activation t for each minibatch as

$$t' = \frac{t - \mu[t]}{\sigma[t]},$$

with $\mu[t], \sigma[t]$ the mean and standard deviation of the pre-activations $t = \mathbf{v} \cdot \mathbf{x}$. For the special case where our network only has a single layer, and the input features $\mathbf{x}$ for that layer are whitened (independently distributed with zero mean and unit variance), these statistics are given by $\mu[t] = 0$ and $\sigma[t] = \|\mathbf{v}\|$. In that case, normalizing the pre-activations using batch normalization is equivalent to normalizing the weights using weight normalization.

Convolutional neural networks usually have much fewer weights than pre-activations, so normalizing the weights is often much cheaper computationally. In addition, the norm of $\mathbf{v}$ is non-stochastic, while the minibatch mean $\mu[t]$ and variance $\sigma^2[t]$ can in general have high variance for small minibatch size. Weight normalization can thus be viewed as a cheaper and less noisy approximation to batch normalization. Although exact equivalence does not usually hold for deeper architectures, we still find that our weight normalization method provides much of the speed-up of full batch normalization. In addition, its deterministic nature and independence on the minibatch input also means that our method can be applied more easily to models like RNNs and LSTMs, as well as noise-sensitive applications like reinforcement learning.

3 Data-Dependent Initialization of Parameters

Besides a reparameterization effect, batch normalization also has the benefit of fixing the scale of the features generated by each layer of the neural network. This makes the optimization robust against parameter initializations for which these scales vary across layers. Since weight normalization lacks this property, we find it is important to properly initialize our parameters. We propose to sample the elements of $\mathbf{v}$ from a simple distribution with a fixed scale, which is in our experiments a normal distribution with mean zero and standard deviation 0.05. Before starting training, we then initialize the b and g parameters to fix the minibatch statistics of all pre-activations in our network, just like in batch normalization, but only for a single minibatch of data and only during initialization. This can be done efficiently by performing an initial feedforward pass through our network for a single minibatch of data $\mathbf{X}$, using the following computation at each neuron:

$$t = \frac{\mathbf{v} \cdot \mathbf{x}}{\|\mathbf{v}\|}, \quad \text{and} \quad y = \phi\left(\frac{t - \mu[t]}{\sigma[t]}\right), \quad (5)$$

where $\mu[t]$ and $\sigma[t]$ are the mean and standard deviation of the pre-activation t over the examples in the minibatch. We can then initialize the neuron's bias b and scale g as

$$g \leftarrow \frac{1}{\sigma[t]}, \quad b \leftarrow \frac{-\mu[t]}{\sigma[t]}, \quad (6)$$

so that $y = \phi(\mathbf{w} \cdot \mathbf{x} + b)$. Like batch normalization, this method ensures that all features initially have zero mean and unit variance before application of the nonlinearity. With our method this only holds for the minibatch we use for initialization, and subsequent minibatches may have slightly different statistics, but experimentally we find this initialization method to work well. The method can also be applied to networks without weight normalization, simply by doing stochastic gradient optimization on the parameters $\mathbf{w}$ directly, after initialization in terms of $\mathbf{v}$ and g : this is what we compare to in section 5. Independently from our work, this type of initialization was recently proposed by different authors [20, 14] who found such data-based initialization to work well for use with the standard parameterization in terms of $\mathbf{w}$.

The downside of this initialization method is that it can only be applied in similar cases as where batch normalization is applicable. For models with recursion, such as RNNs and LSTMs, we will have to resort to standard initialization methods.

4 Mean-only Batch Normalization

Weight normalization, as introduced in section 2, makes the scale of neuron activations approximately independent of the parameters $\mathbf{v}$. Unlike with batch normalization, however, the means of the neuron activations still depend on $\mathbf{v}$. We therefore also explore the idea of combining weight normalization with a special version of batch normalization, which we call *mean-only batch normalization*: With this normalization method, we subtract out the minibatch means like with full batch normalization,

but we do not divide by the minibatch standard deviations. That is, we compute neuron activations using

$$t = \mathbf{w} \cdot \mathbf{x}, \quad \tilde{t} = t - \mu[t] + b, \quad y = \phi(\tilde{t}) \quad (7)$$

where $\mathbf{w}$ is the weight vector, parameterized using weight normalization, and $\mu[t]$ is the minibatch mean of the pre-activation t . During training, we keep a running average of the minibatch mean which we substitute in for $\mu[t]$ at test time.

The gradient of the loss with respect to the pre-activation t is calculated as

$$\nabla_t L = \nabla_{\tilde{t}} L - \mu[\nabla_{\tilde{t}} L], \quad (8)$$

where $\mu[\cdot]$ denotes once again the operation of taking the minibatch mean. Mean-only batch normalization thus has the effect of centering the gradients that are backpropagated. This is a comparatively cheap operation, and the computational overhead of mean-only batch normalization is thus lower than for full batch normalization. In addition, this method causes less noise during training, and the noise that is caused is more gentle as the law of large numbers ensures that $\mu[t]$ and $\mu[\nabla_{\tilde{t}} L]$ are approximately normally distributed. Thus, the added noise has much lighter tails than the highly kurtotic noise caused by the minibatch estimate of the variance used in full batch normalization. As we show in section 5.1, this leads to improved accuracy at test time.

5 Experiments

We experimentally validate the usefulness of our method using four different models for varied applications in supervised image recognition, generative modelling, and deep reinforcement learning.

5.1 Supervised Classification: CIFAR-10

To test our reparameterization method for the application of supervised classification, we consider the CIFAR-10 data set of natural images [15]. The model we are using is based on the ConvPool-CNN-C architecture of [26], with some small modifications: we replace the first dropout layer by a layer that adds Gaussian noise, we expand the last hidden layer from 10 units to 192 units, and we use 2×2 max-pooling rather than 3×3 . The only hyperparameter that we actively optimized (the standard deviation of the Gaussian noise) was chosen to maximize the performance of the network on a holdout set of 10000 examples, using the standard parameterization (no weight normalization or batch normalization). A full description of the resulting architecture is given in table A in the supplementary material.

We train our network for CIFAR-10 using Adam [12] for 200 epochs, with a fixed learning rate and momentum of 0.9 for the first 100 epochs. For the last 100 epochs we set the momentum to 0.5 and linearly decay the learning rate to zero. We use a minibatch size of 100. We evaluate 5 different parameterizations of the network: 1) the standard parameterization, 2) using batch normalization, 3) using weight normalization, 4) using weight normalization combined with *mean-only* batch normalization, 5) using mean-only batch normalization with the normal parameterization. The network parameters are initialized using the scheme of section 3 such that all four cases have identical parameters starting out. For each case we pick the optimal learning rate in $\{0.0003, 0.001, 0.003, 0.01\}$. The resulting error curves during training can be found in figure 1: both weight normalization and batch normalization provide a significant speed-up over the standard parameterization. Batch normalization makes slightly more progress per epoch than weight normalization early on, although this is partly offset by the higher computational cost: with our implementation, training with batch normalization was about 16% slower compared to the standard parameterization. In contrast, weight normalization was not noticeably slower. During the later stage of training, weight normalization and batch normalization seem to optimize at about the same speed, with the normal parameterization (with or without mean-only batch normalization) still lagging behind.

After optimizing the network for 200 epochs using the different parameterizations, we evaluate their performance on the CIFAR-10 test set. The results are summarized in table 2: weight normalization, the normal parameterization, and mean-only batch normalization have similar test accuracy ($\approx 8.5\%$ error). Batch normalization does significantly better at 8.05% error. Mean-only batch normalization combined with weight normalization has the best performance at 7.31% test error, and interestingly does much better than mean-only batch normalization combined with the normal parameterization: This suggests that the noise added by batch normalization can be useful for regularizing the network,

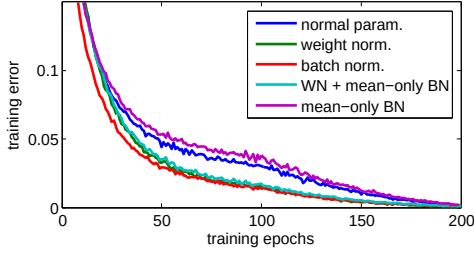


Figure 1: Training error for CIFAR-10 using different network parameterizations. For *weight normalization*, *batch normalization*, and *mean-only batch normalization* we show results using Adam with a learning rate of 0.003. For the normal parameterization we instead use 0.0003 which works best in this case. For the last 100 epochs the learning rate is linearly decayed to zero.

Model	Test Error
Maxout [6]	11.68%
Network in Network [17]	10.41%
Deeply Supervised [16]	9.6%
ConvPool-CNN-C [26]	9.31%
ALL-CNN-C [26]	9.08%
our CNN, mean-only B.N.	8.52%
our CNN, weight norm.	8.46%
our CNN, normal param.	8.43%
our CNN, batch norm.	8.05%
ours, W.N. + mean-only B.N.	7.31%

Figure 2: Classification results on CIFAR-10 without data augmentation.

but that the reparameterization provided by weight normalization or full batch normalization is also needed for optimal results. We hypothesize that the substantial improvement by mean-only B.N. with weight normalization over regular batch normalization is due to the distribution of the noise caused by the normalization method during training: for mean-only batch normalization the minibatch mean has a distribution that is approximately Gaussian, while the noise added by full batch normalization during training has much higher kurtosis. As far as we are aware, the result with mean-only batch normalization combined with weight normalization represents the state-of-the-art for CIFAR-10 among methods that do not use data augmentation.

5.2 Generative Modelling: Convolutional VAE

Next, we test the effect of weight normalization applied to deep convolutional variational auto-encoders (CVAEs) [13, 24, 25], trained on the MNIST data set of images of handwritten digits and the CIFAR-10 data set of small natural images.

Variational auto-encoders are generative models that explain the data vector $\mathbf{x}$ as arising from a set of latent variables $\mathbf{z}$, through a joint distribution of the form $p(\mathbf{z}, \mathbf{x}) = p(\mathbf{z})p(\mathbf{x}|\mathbf{z})$, where the *decoder* $p(\mathbf{x}|\mathbf{z})$ is specified using a neural network. A lower bound on the log marginal likelihood $\log p(\mathbf{x})$ can be obtained by approximately inferring the latent variables $\mathbf{z}$ from the observed data $\mathbf{x}$ using an *encoder* distribution $q(\mathbf{z}|\mathbf{x})$ that is also specified as a neural network. This lower bound is then optimized to fit the model to the data.

We follow a similar implementation of the CVAE as in [25] with some modifications, mainly that the encoder and decoder are parameterized with ResNet [9] blocks, and that the diagonal posterior is replaced with auto-regressive variational inference¹. For MNIST, the encoder consists of 3 sequences of two ResNet blocks each, the first sequence acting on 16 feature maps, the others on 32 feature maps. The first two sequences are followed by a 2-times subsampling operation implemented using 2×2 stride, while the third sequence is followed by a fully connected layer with 450 units. The decoder has a similar architecture, but with reversed direction. For CIFAR-10, we used a neural architecture with ResNet units and multiple intermediate stochastic layers⁴. We used Adamax [12] with $\alpha = 0.002$ for optimization, in combination with Polyak averaging [22] in the form of an exponential moving average that averages parameters over approximately 10 epochs.

In figure 3 we plot the test-set lower bound as a function of number of training epochs, including error bars based on multiple different random seeds for initializing parameters. As can be seen, the parameterization with weight normalization has lower variance and converges to a better optimum. We observe similar results across different hyper-parameter settings.

¹Manuscript in preparation

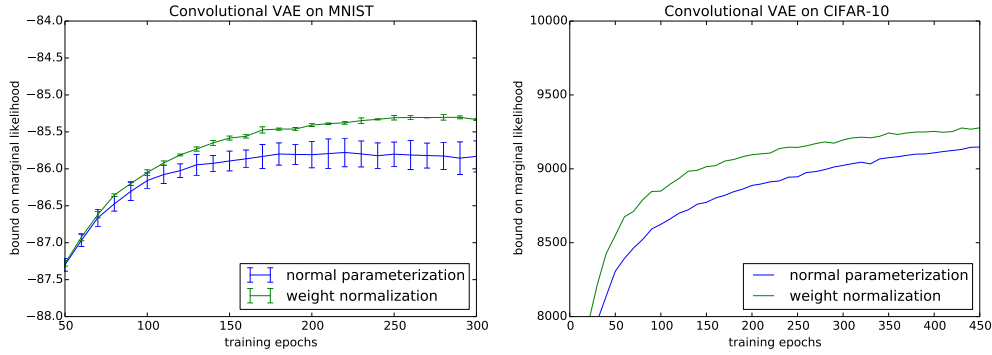


Figure 3: Marginal log likelihood lower bound on the MNIST (top) and CIFAR-10 (bottom) test sets for a convolutional VAE during training, for both the *standard* implementation as well as our modification with *weight normalization*. For MNIST, we provide standard error bars to indicate variance based on different initial random seeds.

5.3 Generative Modelling: DRAW

Next, we consider DRAW, a recurrent generative model by [17]. DRAW is a variational auto-encoder with generative model $p(\mathbf{z})p(\mathbf{x}|\mathbf{z})$ and encoder $q(\mathbf{z}|\mathbf{x})$, similar to the model in section 5.2, but with both the encoder and decoder consisting of a recurrent neural network comprised of Long Short-Term Memory (LSTM) [10] units. LSTM units consist of a memory cell with additive dynamics, combined with input, forget, and output gates that determine which information flows in and out of the memory. The additive dynamics enables learning of long-range dependencies in the data.

At each time step of the model, DRAW uses the same set of weight vectors to update the *cell states* of the LSTM units in its encoder and decoder. Because of the recurrent nature of this process it is not clear how batch normalization could be applied to this model: Normalizing the cell states diminishes their ability to pass through information. Fortunately, weight normalization can be applied trivially to the weight vectors of each LSTM unit, and we find this to work well empirically.

We take the Theano implementation of DRAW provided at <https://github.com/jbornschein/draw> and use it to model the MNIST data set of handwritten digits. We then make a single modification to the model: we apply weight normalization to all weight vectors. As can be seen in figure 4, this significantly speeds up convergence of the optimization procedure, even without modifying the initialization method and learning rate that were tuned for use with the normal parameterization.

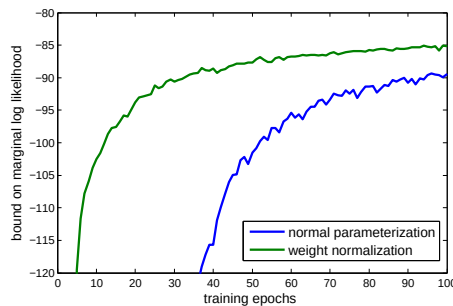


Figure 4: Marginal log likelihood lower bound on the MNIST test set for DRAW during training, for both the *standard* implementation as well as our modification with *weight normalization*. 100 epochs is not sufficient for convergence for this model, but the implementation using weight normalization clearly makes progress much more quickly than with the standard parameterization.

5.4 Reinforcement Learning: DQN

Next we apply weight normalization to the problem of Reinforcement Learning for playing games on the Atari Learning Environment [2]. The approach we use is the Deep Q-Network (DQN) proposed by [21]. This is an application for which batch normalization is not well suited: the noise introduced by estimating the minibatch statistics destabilizes the learning process. We were not able to get batch normalization to work for DQN without using an impractically large minibatch size. In contrast, weight normalization is easy to apply in this context, as is the initialization method of section 3. Stochastic gradient learning is performed using Adamax [12] with momentum of 0.5. We search for optimal learning rates in $\{0.0001, 0.0003, 0.001, 0.003\}$, generally finding 0.0003 to work well with weight normalization and 0.0001 to work well for the normal parameterization. We also use a larger minibatch size (64) which we found to be more efficient on our hardware (Amazon Elastic Compute Cloud g2.2xlarge GPU instance). Apart from these changes we follow [21] as closely as possible in terms of parameter settings and evaluation methods. However, we use a Python/Theano/Lasagne reimplementation of their work, adapted from the implementation available at https://github.com/spragunr/deep_q_rl, so there may be small additional differences in implementation.

Figure 5 shows the training curves obtained using DQN with the standard parameterization and with weight normalization on Space Invaders. Using weight normalization the algorithm progresses more quickly and reaches a better final result. Table 6 shows the final evaluation scores obtained by DQN with weight normalization for four games: on average weight normalization improves the performance of DQN.

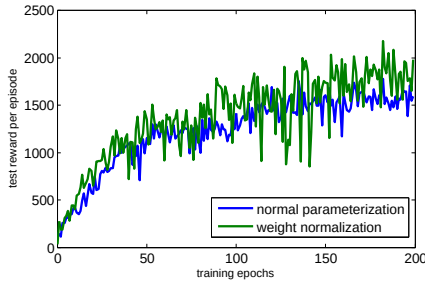


Figure 5: Evaluation scores for Space Invaders obtained by DQN after each epoch of training, for both the standard parameterization and using weight normalization. Learning rates for both cases were selected to maximize the highest achieved test score.

Game	normal	weightnorm	Mnih
Breakout	410	403	401
Enduro	1,250	1,448	302
Seaquest	7,188	7,375	5,286
Space Invaders	1,779	2,179	1,975

Figure 6: Maximum evaluation scores obtained by DQN, using either the normal parameterization or using weight normalization. The scores indicated by *Mnih et al.* are those reported by [21]: Our normal parameterization is approximately equivalent to their method. Differences in scores may be caused by small differences in our implementation. Specifically, the difference in our score on Enduro and that reported by [21] might be due to us not using a play-time limit during evaluation.

6 Conclusion

We have presented *weight normalization*, a simple reparameterization of the weight vectors in a neural network that accelerates the convergence of stochastic gradient descent optimization. Weight normalization was applied to four different models in supervised image recognition, generative modelling, and deep reinforcement learning, showing a consistent advantage across applications. The reparameterization method is easy to apply, has low computational overhead, and does not introduce dependencies between the examples in a minibatch, making it our default choice in the development of new deep learning architectures.

Acknowledgments

We thank John Schulman for helpful comments on an earlier draft of this paper.

A Neural network architecure for CIFAR-10 experiments

Layer type	# channels	x, y dimension
raw RGB input	3	32
ZCA whitening	3	32
Gaussian noise $\sigma = 0.15$	3	32
3×3 conv leaky ReLU	96	32
3×3 conv leaky ReLU	96	32
3×3 conv leaky ReLU	96	32
2×2 max pool, str. 2	96	16
dropout with $p = 0.5$	96	16
3×3 conv leaky ReLU	192	16
3×3 conv leaky ReLU	192	16
3×3 conv leaky ReLU	192	16
2×2 max pool, str. 2	192	8
dropout with $p = 0.5$	192	8
3×3 conv leaky ReLU	192	6
1×1 conv leaky ReLU	192	6
1×1 conv leaky ReLU	192	6
global average pool	192	1
softmax output	10	1

Table 1: Neural network architecture for CIFAR-10.

Normalization and effective learning rates in reinforcement learning

Clare Lyle[†] Zeyu Zheng[†] Khimya Khetarpal[†] James Martens[†] Hado van Hasselt[†]

Razvan Pascanu[†]

Will Dabney[†]

Abstract

Normalization layers have recently experienced a renaissance in the deep reinforcement learning and continual learning literature, with several works highlighting diverse benefits such as improving loss landscape conditioning and combatting overestimation bias. However, normalization brings with it a subtle but important side effect: an equivalence between growth in the norm of the network parameters and decay in the effective learning rate. This becomes problematic in continual learning settings, where the resulting effective learning rate schedule may decay to near zero too quickly relative to the timescale of the learning problem. We propose to make the learning rate schedule explicit with a simple re-parameterization which we call Normalize-and-Project (NaP), which couples the insertion of normalization layers with weight projection, ensuring that the effective learning rate remains constant throughout training. This technique reveals itself as a powerful analytical tool to better understand learning rate schedules in deep reinforcement learning, and as a means of improving robustness to nonstationarity in synthetic plasticity loss benchmarks along with both the single-task and sequential variants of the Arcade Learning Environment. We also show that our approach can be easily applied to popular architectures such as ResNets and transformers while recovering and in some cases even slightly improving the performance of the base model in common stationary benchmarks.

1 Introduction

Many of the most promising application areas of deep learning, in particular reinforcement learning (RL), require training on a problem which is in some way nonstationary. In order for this type of training to be effective, the neural network must maintain its ability to adapt to new information as it becomes available, i.e. it must remain *plastic*. Several recent works have shown that loss of plasticity can present a major barrier to performance improvement in RL and in continual learning [Dohare et al., 2021, Lyle et al., 2021, Nikishin et al., 2022]. These works have proposed a variety of explanations for plasticity loss such as the accumulation of saturated ReLU unit and increased sharpness of the loss landscape [Lyle et al., 2023], along with mitigation strategies, such as resetting dead units [Sokar et al., 2023] and regularizing the parameters towards their initial values [Kumar et al., 2023]. Many of these explanations and their corresponding mitigation strategies center around reducing drift in the distribution of pre-activations [Lyle et al., 2024], a problem which has historically been resolved in the supervised learning setting by incorporating normalization layers into the network architecture. Indeed, normalization layers have been shown to be highly effective at stabilizing optimization in both continual learning and RL [Hussing et al., 2024, Ball et al., 2023].

[†]Google DeepMind. Correspondence to clarelyle@google.com

While effective, normalization on its own is insufficient to avoid loss of plasticity [Lee et al., 2023]. Part of the reason for this lies in a subtle property of normalization: a normalization layer causes the subnetwork preceding it to become scale-invariant, which means that the layer’s *effective learning rate* (ELR) now depends on the norm of its parameters [Van Laarhoven, 2017]. In particular, when the norm of the parameters grows, as it typically does in neural networks trained without regularization, the effective learning rate shrinks. While this property has been studied extensively in idealized supervised learning settings [Arora et al., 2018], its practical implications on optimization in nonstationary problems have previously only been noted in the form of ‘saturation’ of normalization layers [Lyle et al., 2024].

This work begins with an analysis of the effect of layer normalization on a network’s plasticity, revealing two surprising benefits. We first show that when coupled with adaptive optimizers such as Adam and RMSProp whose step size is independent of gradient norm, saturated units still receive sufficient gradient signal to make non-trivial changes to their parameters, providing the opportunity for them to recover as a natural byproduct of optimization. We go on to study the implicit learning rate schedule that layer normalization induces, arriving at the striking observation that even without an explicit learning rate schedule, the implicit learning rate decay induced by parameter norm growth in value-based deep RL algorithms is in fact critical to the optimization process. This observation yields an explanation for the dramatic performance degradations often induced by weight decay in value-based deep RL agents: certain components of the value function require a sufficiently small ELR in order to be learned, and an optimization process which does not reach this value will therefore underfit the value function in ways that can inhibit performance improvement. We further show that, while often beneficial in single-task settings, this implicit schedule can harm performance in nonstationary regimes, where the natural effective learning rate schedule drives the learning rate to trivial values too quickly relative to the task duration.

Leveraging this insight, we propose Normalize-and-Project (NaP), a simple protocol which ensures that the learning rate used by the optimizer reflects the true effective learning rate on the network. NaP avoids implicit learning rate decay, allowing us to attain impressive performance in a variety of nonstationary learning problems even without explicit regularization intended to prevent loss of plasticity. We conduct an empirical evaluation of NaP, confirming that it can be applied to a variety of architectures and datasets without interfering with (indeed, in some cases even improving) performance, including 400M transformer models trained on the C4 dataset and vision models trained on CIFAR-10 and ImageNet. We conclude with a study on the Sequential Arcade Learning Environment, where NaP demonstrates remarkable robustness to task changes and outperforms a baseline Rainbow agent with freshly initialized parameters after 400M training frames (100M optimizer steps).

2 Background and related work

We begin by providing background on trainability and its loss in nonstationary learning problems. We additionally give an overview of neural network training dynamics and effective learning rates.

2.1 Training dynamics and plasticity in neural networks

Early work on neural network initialization centered around the idea of controlling the norm of the activation vectors [LeCun et al., 2002, Glorot and Bengio, 2010, He et al., 2015] using informal arguments. More recently, this perspective has been formalized and expanded [Poole et al., 2016, Daniely et al., 2016, Martens et al., 2021] to include the inner-products between pairs of activation vectors (for different inputs to the network). The function that describes the evolution of these inner-products determines the network’s gradients at initialization up to rotation, and this in turn determines trainability (which was shown formally in the Neural Tangent Kernel regime by Xiao et al., [2020] and [Martens et al., 2021]). A variety of initialization methods have been developed to ensure the network avoids “shattering” [Balduzzi et al., 2017] or collapsing gradients [Poole et al., 2016, Martens et al., 2021, Zhang et al., 2021b].

Once training begins, learning dynamics can be well-characterized in the infinite-width limit by the neural tangent kernel and related quantities [Jacot et al., 2018, Yang, 2019], although in practice optimization dynamics diverge significantly from the infinite-width limit [Fort et al., 2020]. A complementary line of work has empirically and theoretically characterized self-stabilization properties

[Lewkowycz et al., 2020, Cohen et al., 2021, Agarwala et al., 2022], along with implicit regularization effects [Barrett and Dherin, 2020, Smith et al., 2020] from using non-infinitesimal learning rate. A variety of architectural choices can accelerate the training of extremely deep networks, including residual connections [He et al., 2016] and normalization layers [Ioffe and Szegedy, 2015, Ba et al., 2016]. Some additional works have aimed to replicate the benefits of normalization layers via normalization of the network parameters [Salimans and Kingma, 2016, Arpit et al., 2016], though layer normalization remains standard practice.

Maintaining trainability not only at initialization, but also, over the course of optimization is of critical importance in reinforcement and continual learning, and failure to do so has been extensively documented throughout the literature under the term *loss of plasticity* [Abbas et al., 2023, Lyle et al., 2021, Dohare et al., 2021, Sodhani et al., 2020, Nikishin et al., 2022]. This phenomenon has been shown to present a limiting factor to performance in a number of RL tasks [Nikishin et al., 2023], along with continual learning and warm-starting neural network training [Berariu et al., 2021, Ash and Adams, 2020]. Plasticity loss can be further decomposed into two distinct components [Lee et al., 2023]: loss of trainability and reduced generalization performance, of which we focus on the former.

2.2 Effective learning rates

As noted by several prior works [Van Laarhoven, 2017, Li and Arora, 2020, Li et al., 2020b], normalization introduces scale-invariance into the layers to which it is applied, where by a scale-invariant function f we mean $f(c\theta, \mathbf{x}) = f(\theta, \mathbf{x})$ for any positive scalar $c > 0$. This leads to the gradient scaling inversely with the parameter norm, whereby $\nabla f(c\theta) = \frac{1}{c} \nabla f(\theta)$. The intuition behind this property is simple: changing the direction of a large vector requires a greater perturbation than changing the direction of a small vector. This motivates the concept of an ‘effective learning rate’, which provides a scale-invariant notion of optimizer step size. In the following definition, we take the approach of [Kodryan et al., 2022] and assume an implicit ‘reference norm’ of size 1 for the parameters.

Definition 1 (Effective learning rate). *Consider a scale-invariant function f , parameters θ and update function $\theta_{t+1} \leftarrow \theta_t + \eta g(\theta_t)$. Letting $\rho = \frac{1}{\|\theta\|}$, we then define the effective learning rate $\tilde{\eta}$ as follows:*

$$\tilde{\eta} = \begin{cases} \eta\rho^2, & \text{if } g(\theta_t) = \nabla_{\theta} f(\theta_t) \\ \eta\rho, & \text{if } g(\theta_t) = \frac{\nabla_{\theta} f(\theta_t)}{\|\nabla_{\theta} f(\theta_t)\|} \end{cases} \quad (1)$$

where, letting $\tilde{\theta} = \theta \frac{1}{\|\theta\|}$ we then have $f(\tilde{\theta} + \tilde{\eta}g(\tilde{\theta})) = f(\theta + \eta g(\theta))$

This suggests that regularization of the network norm can have the dual effect of increasing the effective learning rate, as noted by prior work [Van Laarhoven, 2017, Hoffer et al., 2018]. Several works have since offered more fine-grained theoretical analyses of such implicit learning rate schedules [Arora et al., 2018], and suggested means of translating these implicit schedules into explicit ones [Li and Arora, 2020, Li et al., 2020a]. More recently, the work of [Lobacheva et al., 2021] and [Kodryan et al., 2022] has studied the training properties of scale-invariant networks trained with parameters constrained to the unit sphere, a training regime we expand upon in this work.

3 Analysis of normalization layers and plasticity

Although widely used and studied, the precise reasons behind the effectiveness of layer normalization remain mysterious. In this section, we provide some new insights into how normalization can help neural networks to maintain plasticity by facilitating the recovery of saturated nonlinearities, and highlight the importance of controlling the parameter norm in networks which incorporate normalization layers. We leverage these insights to propose Normalize-and-Project, a simple training protocol to maintain important statistics of the layers and gradients throughout training.

3.1 Layer normalization

It is widely accepted that achieving approximately mean-zero, unit-variance pre-activations (assuming suitable choices of activation functions) is useful to ensure a network is trainable at initialization [e.g. [Martens et al., 2021]], and many neural network initialization schemes aim to maintain this property

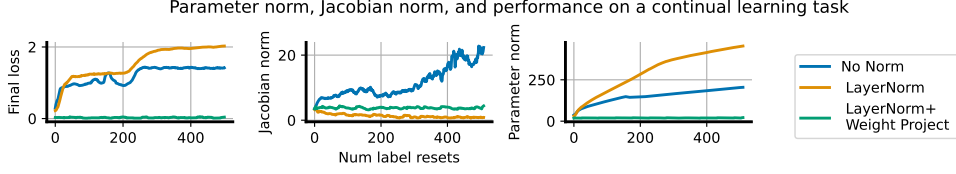


Figure 1: Continual random-labels CIFAR training: simple feedforward network architecture (No Norm) exhibits rapid growth in its parameter norm and the norm of its gradients, whereas the otherwise-identical network with layer normalization sees parameter norm growth coupled with a *reduction* in the norm of its gradients and reduced performance on later tasks. Constraining the parameter norm of this network maintains the performance of a random initialization.

as the depth of the network grows. Indeed, it is easy to show that in extreme cases large deviations of these statistics from their initial values can lead to a variety of network pathologies including saturated units and low numerical rank of the empirical neural tangent kernel [Xiao et al., 2020]. Layer normalization not only guarantees that activations are unit-norm, mean-zero at initialization, but also that they stay that way over the course of training even if the data distribution changes, assuming no scale or offset parameters. This property not only improves robustness to a variety of pathologies that cause loss of plasticity [Lyle et al., 2024], but also helps to improve the conditioning of the network’s gradients in RL [Ball et al., 2023].

Beyond re-normalizing the pre-activation statistics, layer normalization also introduces a dependency between units in a given layer via the mean subtraction and division by standard deviation transformations, which translates to correlations in the gradients of their corresponding weights. This mixing step allows gradients to propagate through a pre-activation even if the unit is saturated, provided layer normalization is applied prior to the nonlinearity, a property we highlight in Proposition 1. We will use the notation f_{RMS} to refer to the transform $h \mapsto \frac{h}{\|h\|}$, and $f'(x) \equiv \frac{\partial}{\partial x} f(x)$ to refer to the scalar derivative of any f at scalar x .

Proposition 1. Consider two indices i and j of a feature embedding $\phi(f_{\text{RMS}}(h))$ such that $\phi'(f_{\text{RMS}}(h))_j \neq 0$, and $h_i, h_j \neq 0$. Then we have

$$\frac{d}{dh_i} \phi(f_{\text{RMS}}(h))_j = \phi'(f_{\text{RMS}}(h))_j \frac{1}{\|h\|^3} h_i h_j \neq 0.$$

In contrast, for post-activation normalization the gradient is zero whenever $\phi'(h_i) = 0$, taking the form $\partial_{h_i} f_{\text{RMS}}(\phi(h))_j = \phi'(h_i) \frac{1}{\|\phi(h)\|^3} \phi(h_i) \phi(h_j)$.

In other words, normalization effectively gives dead ReLU units a second chance at life – rather than immediately decaying to zero, the gradients flowing through a dead ReLU unit will instead take small, non-zero values, which depends on the gradients of the mean and variance of that particular layer. These gradients will be much smaller than those that would typically backpropagate to the unit, but if an optimizer such as Adam or RMSProp is used to correct for the gradient norm, then the unit may still be able to take nontrivial steps, which have a chance at propelling it back into the activated regime. We include the full derivation in Appendix A.2, and we demonstrate the effect this can have on both a theoretical model and empirical neural network training in Figure 10 in Appendix C.4. This property is also naturally inherited by layer normalization, which can be viewed as the composition of RMSNorm with a centering transform. While layer normalization can help the network to recover from saturated nonlinearities, it introduces a new source of potential saturation which must be carefully considered, which is something we will do in the next section.

3.2 Parameter norm and effective learning rate decay

While the *output* of a scale-invariant function is insensitive to scalar multiplication of the parameters, its *gradient* magnitude scales inversely with the parameter norm. This results in the opposite behaviour of what we would expect in an unnormalized network: in networks with layer normalization, growth in the norm of the parameters corresponds to a decline in the network’s sensitivity to changes in these parameters. In a sense this is preferable, as the glacially slow but stable regime of vanishing gradients is easier to recover from than the unstable exploding gradient regime. However, if the parameter

Algorithm 1 NaP: Normalize-and-Project

Input: network $\mathcal{N}$, input x for nonlinearity ϕ_l in network do if ϕ_l not already normalized then $\phi_l \leftarrow \phi_l \circ \text{LayerNorm}$ end if end for compute $\theta' = \text{update}(\theta)$ for parameter W_l in network do compute $W_l \leftarrow \text{WeightProject}(W_l)$ end for	$\text{WeightProject}(W_l, \rho_l) :$ if W_l is a weight parameter then $W_l \leftarrow \frac{\rho_l W_l}{\ W_l\ }$ else $\sigma_l, \mu_l \leftarrow W_l, d \leftarrow \text{len}(\sigma_l)$ $\sigma_l \leftarrow \sigma_l \sqrt{\frac{d}{\ \sigma_l\ ^2 + \ \mu_l\ ^2}}$ $\mu_l \leftarrow \mu_l \sqrt{\frac{d}{\ \sigma_l\ ^2 + \ \mu_l\ ^2}}$ end if
---	--

norm grows indefinitely then the corresponding reduction in the effective learning rate will eventually cause noticeable slowdowns in learning – indeed, it is quite easy to induce this type of situation as we show in Figure 1. To do so, we take a small base convolutional neural network architecture (detailed in Appendix B.4) and train it on random labels of the CIFAR-10 dataset, akin to the classic setting of Zhang et al. (2021a). We then re-randomize these labels and continue training, repeating this process 500 times. When we apply this process to a network without normalization layers, the Jacobian norm grows to unstable values as the parameter norm increases; in contrast, an equivalent architecture with normalization layers sees a sharp decline in the Jacobian norm as the parameter norm increases. In both cases, the end result is similar: increased parameter norm accompanies reduced performance.

While this particular problem is artificial, it is a real and widely observed underlying phenomenon that the magnitudes of neural network parameters tend to increase over the course of training (Nikishin et al., 2022; Abbas et al., 2023). In a supervised learning problem, where one is using a fixed training budget, the ELR decay induced by growing parameter norms might be desirable and help to protect against too-large learning rates (Arora et al., 2018; Salimans and Kingma, 2016). Allowed to continue to extremes, however, ELR decay becomes problematic (Lyle et al., 2024). Instead, if we re-normalize the parameters after every task (which does not affect the output of our scale-invariant model) so they recover the norm they had at initialization (but do not change their direction), we see in Figure 1 that we can maintain the performance of a random initialization even after hundreds of training iterations.

3.3 Normalize-and-Project

We conclude from the above investigation that normalizing a network’s pre-activations and fixing the parameter norm presents a simple but effective defense against loss of plasticity. In this section, we propose a principled approach to combine these two steps which we call NaP. Our goal for NaP is to provide a flexible recipe which can be applied to essentially any architecture, and which improves the stability of training, motivated by but not limited to non-stationary problems. Our approach can be decomposed into two steps: the **insertion of normalization layers** prior to nonlinearities in the network architecture, and the **periodic projection** of the network’s weights onto a fixed-norm radius throughout training, along with a corresponding update to the per-layer learning rates into the optimization process. Algorithm 1 provides an overview of NaP. While some design choices, such as the use of scale and offset terms, can be tailored to a particular problem setting, we will aim to provide principled guidance on how to reason about the effects of these choices.

Layer normalization: Introducing layer normalization allows us to benefit from the properties discussed in Section 3.1, though fully benefiting from these properties depend on normalization being applied each time a linear transform precedes a nonlinearity. While it might seem extreme, this proposal is in line with most popular language and vision architectures. For example, Vaswani et al. (2017) apply normalization after every two fully-connected layers, and recent results suggesting that adding normalization to the key and query matrices in attention heads (Henry et al., 2020) can provide further benefits.

Weight projection: As discussed in Section 3.2, we must then take care in controlling the network’s effective learning rate. We propose disentangling the parameter norm from the effective learning rate by enforcing a constant norm on the weight parameters of the network, allowing scaling of the layer outputs to depend only on the learnable scale and offset parameters. This approach is similar to that proposed by Kodryan et al. (2022), but importantly takes care to treat the scale and offset parameters

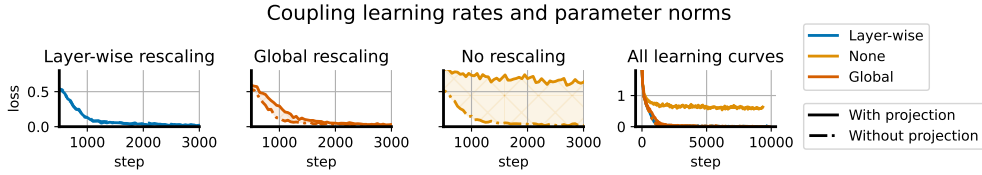


Figure 2: We run a ‘coupled networks’ experiment as described in the text. All networks exhibit similar learning curves, as seen by the rightmost subplot, however there is small but visible gap between the learning curves obtained by NaP and an unconstrained network with fixed learning rates. Using a global learning rate schedule almost entirely closes this gap, but does not induce a precise equivalence in the dynamics as obtained by layer-wise rescaling (leftmost).

separately from weights. For simplicity, we remove bias terms as these are made redundant by the learnable offset parameters. In order to maintain constant parameter norm, we rescale the parameters of each layer to match their initial norm periodically throughout training – the precise frequency is not important as long as the parameter norm does not meaningfully grow to a point of slowing optimization between projections. For example, we find that in Rainbow agents an interval of 1000 steps and 1 step produce nearly identical empirical results.

We find it is absolutely critical to normalize the weight parameters, as these represent the bulk of trainable parameters in the network. The learnable scale and offset parameters, assuming they are included in the network³, permit more flexibility and can be dealt with in one of three ways. The strictly correct version in the case of homogeneous activations such as ReLUs is the approach we outline in Appendix D, which projects the concatenated scale-offset vector. This can require some effort to implement due to the dependency between the scale and offset parameters and may not be worthwhile for small training runs – indeed, most of our empirical results did not require this step, though we include it in our analysis of smaller networks in Figure 2. A softer version of this normalization which requires less implementation overhead and which generalizes to non-homogeneous activations is to regularize the scale and offset parameters to their initial values, a strategy which we employ in our continual learning evaluations. Finally, they can be allowed to drift unconstrained from their initialization, a choice we find unproblematic in most shorter, stationary learning problems. For a more detailed discussion on this choice, we refer to Appendix D.

4 Lessons on the effective learning rate

NaP constrains the network’s effective learning rate to follow an explicit rather than implicit schedule. In this section, we explore how this property affects network training dynamics, demonstrating how implicit learning rate schedules due to parameter norm growth can be made explicit and be leveraged to improve the performance of NaP in deep RL domains.

4.1 Replicating the dynamics of parameter norm growth in NaP

We begin our study of effective learning rates by illustrating how the implicit learning rate schedule induced by the evolution of the parameter norm can be translated to an explicit schedule in NaP. We study a small CNN described in Appendix B.4 with layer normalization prior to each nonlinearity trained on CIFAR-10 with the usual label set. We train two ‘twin’ scale-invariant networks with the Adam optimizer in tandem: both networks see the exact same data stream and start from the same initialization, but the per-layer weights of one are projected after every gradient step to have constant norm, while the other is allowed to vary the norms of the weights. We then consider three experimental settings: in the first, we re-scale the per-layer learning rates of the projected network so that the explicit learning rate is equal to the effective learning rate of its twin. In the second, we re-scale the global learning rate based on the ratio of parameter norms between the projected and unprojected network, but do not tune per-layer. In the third, we do no learning rate re-scaling. We see in Figure 2 that the shapes of the learning curves for all networks except for the constant-ELR

³We observed in many of our experiments that removing trainable scale and offset parameters often has little effect on network performance. In Rainbow agents, for example, removing the trainable offset parameter even improves performance in several environments.

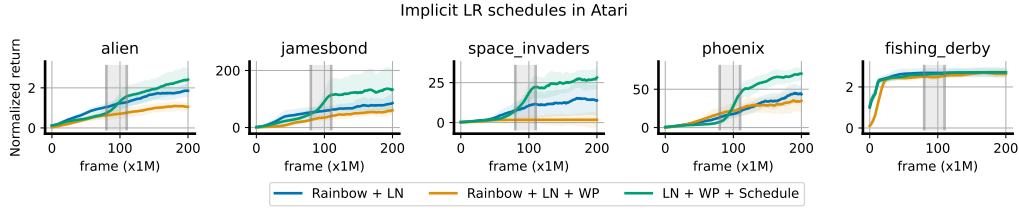


Figure 3: Without an explicit learning rate schedule, a Rainbow trained with NaP may fail to make any performance improvement; while the implicit schedule induced by the parameter norm is clearly important to performance, in several games this is significantly outperformed by a simple linear schedule terminating halfway through training. Intriguingly, we see a characteristic sharp improvement near the end of the decay schedule in several (though not all, e.g. fishing derby) games.

variant are quite similar, with the global learning rate scaling strategy producing a smaller gap than the no-rescaling strategy. By construction, the dynamics of the per-layer rescaling network and its twin are identical. Because global learning rate schedules are standard practice and induce dynamics that are quite close to those obtained by parameter norm growth in Figure 2, we take this approach in the remainder of the paper, leaving layer-specific learning rates and schedules for future work.

□

4.2 Implicit learning rate schedules in deep RL

When taken to extremes, learning rate decay will eventually prevent the network from making nontrivial learning progress. However, learning rate decay plays an integral role in the training of many modern architectures, and is required to achieve convergence for stochastic training objectives (unless the interpolation applies or Polyak averaging is employed). In this section we will show that, perhaps unsurprisingly, naive application of NaP with a constant effective learning rate can sometimes harm performance in settings where the implicit learning rate schedule induced by parameter norm growth was in fact critical to the optimization process. More surprising is that the domain where this phenomenon is most apparent is one where common wisdom would suggest learning rate decay would be undesirable: deep RL.

RL involves a high degree of nonstationarity. As a result, deep RL algorithms such as DQN and Rainbow often use a constant learning rate schedule. Given that layer normalization has been widely observed to improve performance in value-based RL agents on the arcade learning environment, and that parameter norm tends to increase significantly in these agents, one might at first believe that the performance improvement offered by layer normalization is happening in *spite* of the resulting implicit learning rate decay. A closer look at the literature, however, reveals that several well-known algorithms such as AlphaZero [Schrittwieser et al., 2020], along with many implementations of popular methods such as Proximal Policy Optimization [Schulman et al., 2017], incorporate some form of learning rate decay, suggesting that a constant learning rate is not always desirable. Indeed, Figure 3 shows that constraining the parameter norm to induce a fixed ELR in the Rainbow agent frequently results in *worse* performance compared to unconstrained parameters. This is particularly striking given that many of the benefits supposedly provided by layer normalization, such as better conditioning of the loss landscape [Lyle et al., 2023] and mitigation of overestimation bias [Ball et al., 2023], should be independent of the effective learning rate. Instead, these properties appear to either be irrelevant for optimization or dependent on reductions in the effective learning rate.

We can close this gap by introducing a learning rate schedule (linear decay from the default $6.25 \cdot 10^{-5}$ to 10^{-6} , roughly proportional to the average parameter norm growth across games). We further observe in Appendix C.1 that when we vary the endpoint of the learning rate schedule, we often obtain a corresponding x-axis shift in the learning curves, suggesting that reaching a particular learning rate was necessary to master some aspect of the game. We conclude that, while beneficial, the implicit schedule induced by the parameter norm is not necessarily *optimal* for deep RL agents, and it is possible that a more principled adaptive approach could provide still further improvements.

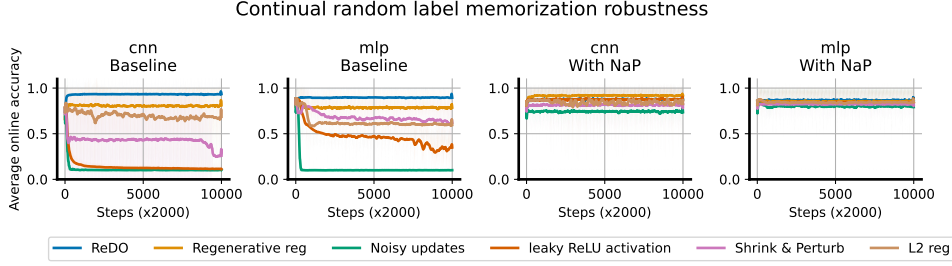


Figure 4: Robustness to nonstationarity: we see that without NaP, there is a wide spread in the effectiveness of various plasticity-preserving methods across two architectures. Once we incorporate NaP, however, the gaps between these methods shrink significantly and almost uniformly improves over the unconstrained baseline.

	CIFAR-10	ImageNet-1k	C4	Pile	WikiText	Lambada	SIQA	PIQA
NaP	94.64	77.26	45.7	47.9	45.4	56.6	44.2	68.8
Baseline	94.65	77.08	44.8	47.4	44.2	54.1	43.5	67.3
Norm only	94.47	77.45	44.9	47.6	44.3	53.6	43.8	67.1

Table 1: Left: Top-1 prediction accuracy on the test sets of CIFAR-10 and ImageNet-1k. **Right:** per-token accuracy of a 400M transformer model pretrained on the C4 dataset, evaluated on a variety of language benchmarks. See Appendix C.5 for more results with variation measures.

5 Experiments

We now validate the utility of NaP empirically. Our goal in this section is to validate two key properties: first, that NaP does not hurt performance on stationary tasks; second, that NaP can mitigate plasticity loss under a variety of both synthetic and natural nonstationarities.

5.1 Robustness to nonstationarity

We begin with the continual classification problem described in Appendix B.4. We evaluate our approach on a variety of sources of nonstationarity, using two architectures: a small CNN, and a fully-connected MLP (see Appendix B.4 for details). We first evaluate a number of methods designed to maintain plasticity including Regenerative regularization [Kumar et al., 2023], Shrink and Perturb [Ash and Adams, 2020], ReDO [Sokar et al., 2023], along with leaky ReLU units (inspired by the CReLU trick of Abbas et al. [2023]), L2 regularization, and random Gaussian perturbations to the optimizer update, a heuristic form of Langevin Dynamics. We track the average online accuracy over the course of training for 20M steps, equivalent to 200 data relabelings, using a constant learning rate. We find varying degrees of efficacy in these approaches in the base network architectures, with regenerative regularization and ReDO tending to perform the best. When we apply the same suite of methods to networks with NaP, in Figure 4 we observe near-monotonic improvements (with the exception of ReDO, which we conjecture is because the reset method designed for unnormalized networks) in performance and a significant reduction in the gaps between methods, with the performance curves of the different methods nearly indistinguishable in the MLP. Further, we observe constant or increasing slopes in the online accuracy, suggesting that the difference between methods has more to do with their effect on within-task performance than on plasticity loss once the parameter and layer norms have been constrained.

5.2 Stationary supervised benchmarks

Having observed remarkable improvements in synthetic tasks, we now confirm that NaP does not interfere with learning on more widely-studied, natural datasets.

Large-scale image classification. We begin by studying the effect of NaP on two well-established benchmarks: a VGG16-like network [Simonyan and Zisserman, 2014] on CIFAR-10, and a ResNet-50 [He et al., 2016] on the ImageNet-1k dataset. We provide full details in Appendix B.4. In Table 1 we obtain comparable performance in both cases using the same learning rate schedule as the baseline.

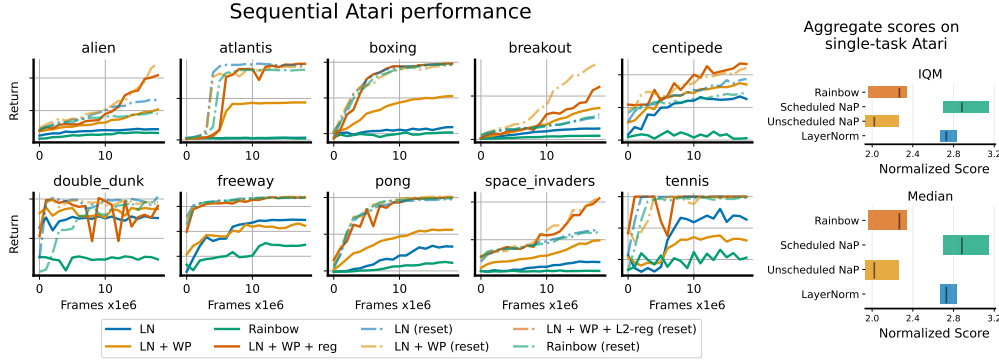


Figure 5: Left: We visualize the learning curves of continual atari agents on sequential ALE training (i.e. 200M frames). Each game is played for 20M frames, and agents pass sequentially from one to another, repeating all ten games twice for a total of 400M training frames. Solid lines indicate performance on the second visit to each game, and dotted lines indicate performance of a randomly initialized network on the game. Even in its second visit to each game, NaP performs comparably the randomly initialized networks, whereas the standard rainbow agent exhibits poor performance on all games in the sequential training regime. **Right:** aggregate effects of normalization on single-task atari, computed via the approach of Agarwal et al. [2021]. Bars indicate 95% confidence intervals over 4 seeds and 57 environments.

Natural language: we now turn our attention to language tasks, training a 400M-parameter transformer architecture (details in Appendix B.3) on the C4 benchmark [Raffel et al., 2020]. Table 1 shows that our approach does not interfere with performance on this task, where we match final performance in terms of training accuracy. When evaluating the pre-trained network on a variety of other datasets, we find that NaP slightly outperforms baselines in terms of performance on a variety of benchmarks, including WikiText-103, Lambada [Paperno et al., 2016], piqa [Bisk et al., 2020], SocialQA [Sap et al., 2019], and Pile [Gao et al., 2020].

5.3 Deep reinforcement learning

Finally, we evaluate our approach on a setting where maintaining plasticity is critical to performance: RL on the Arcade Learning Environment. We conduct a full sweep over 57 Atari 2600 games comparing the effects of normalization, weight projection, and learning rate schedules on a Rainbow agent [Hessel et al., 2018]. In the RHS of Figure 5 we plot the spread of scores, along with estimates of the Mean and IQM of four agents: standard Rainbow, Rainbow + LayerNorm, Rainbow + NaP without an explicit LR schedule, and Rainbow + NaP with the LR schedule described in Section 4.2. We find that NaP with a linear schedule outperforms the other methods.

We also consider the sequential setting of Abbas et al. [2023]. In this case, we consider an idealized setup where we reset the optimizer state and schedule every time the environment changes, using a cosine schedule with warmup described in Appendix B.2. To evaluate NaP on this regime, we train on each of 10 games for 20M frames, going through this cycle twice. We do not reset parameters of the continual agents between games, but do reset the optimizer. We plot learning curves for the second round of games in the LHS of Figure 5, finding that NaP significantly outperforms a baseline Rainbow agent with and without layer normalization. Indeed, even after 200M steps the networks trained with NaP make similar learning progress to a random initialization.

6 Discussion

This paper has shown that loss of plasticity can be substantially mitigated by constraining the norms of the network’s layers and parameters. This finding was made possible by two key insights: first, that in addition to the obvious benefits relating to maintaining constant (pre-)activation norms, layer normalization can also protect against saturated units, and second, that it introduces a correspondence between the parameter norm and the effective learning rate which has significant consequences on performance. We proposed NaP as a means of making the effective learning rate explicit in the optimization process, and leveraged this to gain new insights into the importance of learning rate decay schedules in deep reinforcement learning. In particular, we showed that Rainbow agents trained

on the Arcade Learning Environment in fact under-perform their unconstrained counterparts when a constant effective learning rate is enforced. Provided a suitable decay schedule is used, however, a network’s performance and robustness to nonstationarity can both be improved by using NaP. Our analysis suggests a number of exciting directions for future work; in particular, the use of adaptive learning rate schedules in reinforcement learning, and the possibility of tuning not only the global learning rate but also per-layer learning rates in networks which use normalization layers.

Broader Impact

This work concerns basic properties of optimization of neural networks. We do not anticipate any broader societal impacts as a direct consequence of our findings.

References

- Zaheer Abbas, Rosie Zhao, Joseph Modayil, Adam White, and Marlos C Machado. Loss of plasticity in continual deep reinforcement learning. *ICML*, 2023.
- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems*, 34:29304–29320, 2021.
- Atish Agarwala, Fabian Pedregosa, and Jeffrey Pennington. Second-order regression models exhibit progressive sharpening to the edge of stability. *arXiv preprint arXiv:2210.04860*, 2022.
- Sanjeev Arora, Zhiyuan Li, and Kaifeng Lyu. Theoretical analysis of auto rate-tuning by batch normalization. In *International Conference on Learning Representations*, 2018.
- Devansh Arpit, Yingbo Zhou, Bhargava Kota, and Venu Govindaraju. Normalization propagation: A parametric technique for removing internal covariate shift in deep networks. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1168–1176, New York, New York, USA, 20–22 Jun 2016. PMLR.
- Jordan Ash and Ryan P Adams. On warm-starting neural network training. *Advances in Neural Information Processing Systems*, 33:3884–3894, 2020.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- David Balduzzi, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, and Brian McWilliams. The shattered gradients problem: If resnets are the answer, then what is the question? In *International Conference on Machine Learning*, pages 342–350. PMLR, 2017.
- Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 1577–1594. PMLR, 23–29 Jul 2023.
- David GT Barrett and Benoit Dherin. Implicit gradient regularization. *arXiv preprint arXiv:2009.11162*, 2020.
- Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47: 253–279, 2013.
- Tudor Berariu, Wojciech Czarnecki, Soham De, Jorg Bornschein, Samuel Smith, Razvan Pascanu, and Claudia Clopath. A study on the plasticity of neural networks. *arXiv preprint arXiv:2106.00042*, 2021.
- Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

A Derivations

A.1 Notation

Analysis of a network’s training dynamics depends on characterizing the evolution of (pre-)activations and gradients in the forward and backward passes respectively. We lay out basic notation for fully-connected layers first, and then note additional details which must be considered for convolutional, skip connection, and attention layers. We will write the parameters of a layer as θ_l , and use f to denote a neural network.

Fully-connected layers: We write the forward pass through a network $f : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_L}$ as a composition of layer-wise computations $f^l : \mathbb{R}^{d_{l-1}} \rightarrow \mathbb{R}^{d_l}$ of the form:

$$a^l = f^l(a^{l-1}) = \phi(\sigma^l f_{\text{LN}}(h^l) + \mu^l), \quad h^l = W^l a^{l-1} \quad W^l = \theta_l \quad (2)$$

where ϕ denotes a nonlinearity and f_{LN} is a (possibly absent) normalization operator. We let a^0 denote the network inputs. We will refer to a forward pass through a subset of a network with the notation $f^{l_1:l_2} = f^{l_2} \circ \dots \circ f^{l_1}$, and use interchangeably $f = f^{1:L}$. We will refer to the full set of network parameters by $\theta = \text{vec}(\{\theta^l\}_{l=1}^L)$.

Convolutional layers: since a convolution can be viewed as a parameterization of a matrix with a particular symmetry, we express these layers identically to the fully-connected layers, with a change in semantics such that W^l is the matrix representation of the convolutional parameters θ_l , where we write the embedding of θ_l into a matrix as $W_{\text{conv}}(\theta_l)$. For simplicity, we ignore the choice of padding.

$$\phi(\sigma^l f_{\text{LN}}(h^l) + \mu^l), \quad h^l = W^l a^{l-1} \quad \text{and} \quad W^l = W_{\text{conv}}(\theta_l) \quad (3)$$

Skip-connect layers: in some network architectures, a nonlinearity is placed on the outputs of two subnetworks to produce a function of the form

$$f^l = \phi\left(\sum_{i \in L} a_{l_i}\right) \quad \text{or} \quad f^l = \phi\left(f_{\text{LN}}\left(\sum_{i \in L} a_{l_i}\right)\right) \quad (4)$$

for some index set L . The choice of whether to apply normalization to the sum of the subnetwork outputs or to each output individually depends on the desired signal propagation properties of the network [De and Smith, 2020].

A.2 Derivation of Proposition 1

The result described in proposition 1 follows straightforwardly from the chain rule. For pre-activation RMSNorm we have

$$\frac{d}{dh_i} \phi(f_{\text{RMS}}(h))_j = \phi'(f_{\text{RMS}}(h))_j \frac{d}{dh_i} f_{\text{RMS}}(h)_j \quad (5)$$

$$\frac{d}{dh_i} f_{\text{RMS}}(h)_j = \frac{d}{dh_i} \frac{h_j}{\left(\sum h_k^2\right)^{1/2}} \quad (6)$$

$$= \cup \frac{1}{2} \frac{h_j}{\left(\sum h_k^2\right)^{3/2}} \frac{d}{dh_i} \sum h_k^2 \quad (7)$$

$$= \cup \frac{h_j}{\left(\sum h_k^2\right)^{3/2}} h_i \quad (8)$$

$$\frac{d}{dh_i} \phi(f_{\text{RMS}}(h))_j = \cup \frac{\phi'(f_{\text{RMS}}(h))_j h_j h_i}{\|h\|^3} \quad (9)$$

A.3 Gradients and signal propagation

One perspective which can provide some additional insight into our approach is to consider the following decomposition of the gradient being backpropagated through a layer.

$$\nabla_{W_k} \mathcal{L}(\theta) = \partial_{a_k} \mathcal{L}(\theta) \cdot \partial_{W_k} \phi(f_{\text{LN}}(W_k a_{k-1})) \quad (10)$$

$$= \partial_{a_k} \mathcal{L}(\theta) \cdot D_{\phi} \nabla_{h_k} f_{\text{LN}}(h_k) a_{k-1}^\top \quad (11)$$

With this decomposition, we obtain the following interpretation of some common pathologies.

Saturated nonlinearities: a saturated nonlinearity implies that D_{ϕ} , and the gradient is exactly (in the case of ReLU) or very close to (e.g. tanh) zero. As a result, u_t will be zero for the affected coordinates and the corresponding parameters will remain frozen. NaP addresses this problem by using Layer or RMSNorm prior to any nonlinearity in the network.

Saturated normalization layers: the normalization transformation $x \mapsto \frac{x}{\|x\|}$ is vulnerable to saturation as $\|x\|$ grows, in the sense that for a fixed-norm update u we will have

$$\lim_{\|x\| \rightarrow \infty} \frac{x+u}{\|x+u\|} \cup \frac{x}{\|x\|} = 0 \quad (12)$$

and so the term $\nabla_{h_k} f_{\text{LN}}(h_k)$ will vanish. In this case, while the optimizer will be able to update the parameters, these updates will have a diminishing effect on the network’s output.

Vanishing and exploding gradients: divergence and disappearance of the activations a_{k-1} and backpropagated gradients $\partial_{a_k} \mathcal{L}(\theta)$ are well-known pathologies which can make networks untrainable. However, even networks which start training from a well-tuned initialization may still encounter exploding gradients due to parameter norm growth over time [Dohare et al., 2021, Wortsman et al., 2023], or vanishing gradients and activations, such as in the case of saturated (i.e. dead/dormant) ReLU units [Sokar et al., 2023].

A.4 Details of NaP

Guiding principles: in general, the goal of NaP is to avoid dramatic distribution shifts in the pre-activation and parameter norms, and to ensure that the network can perform updates to parameters even if a nonlinearity is saturated. With these in mind, there are two key properties that a network designer should aim to maintain:

1. All **parametric** functions entering a nonlinearity should have a normalization layer that ensures the gradients of all units’ parameters are correlated. If there are no parameters between nonlinearities (as is sometimes the case in e.g. resnets) normalization is not essential.
2. Based on our investigations in Appendix C.4, *L2 normalization* of the pre-activations is crucial to obtain the positive benefits of layer normalization, while centering does not have noticeable effects on the network’s robustness to unit saturation. As a result, applying at least RMSNorm is crucial prior to nonlinearities, but the choice of whether or not to incorporate centering is up to the designer’s discretion.

Batch normalization layers: by default, we put layer normalization prior to batch normalization if an architecture already incorporates batch normalization prior to a nonlinearity. This preserves the property of batch norm that individual units have mean zero across the batch, which may not be the case if layer normalization is applied after. We also always omit offset parameters if layernorm is succeeded by batchnorm, as these offset parameters will be zeroed out by batchnorm.

Skip-connect layers: provided that layer normalization is applied to the outputs of a linear transformation prior to a nonlinearity, NaP is agnostic to whether normalization is applied prior to or after a residual connection’s outputs are added to the output of a layer. In particular, if we have a layer of the form $\phi(a_1 + a_2)$ and a_1 and a_2 are the outputs of some subnetwork of the form $\phi_1(f_{\text{LN}}(h_1))$ and $\phi_2(f_{\text{LN}}(h_2))$ where ϕ_1 and ϕ_2 are (possibly trivial) activation functions, then the relevant parameters will already benefit from Proposition 1 and it is not necessary to add an additional normalization layer prior to the activation ϕ .

Attention layers: unlike linear layers, attention layers do not typically include bias terms. To analogize this common practice in NaP, we omit the offset and mean subtraction components of the LayerNorm transform, obtaining

$$\text{MHA}(W_Q X, W_K X) \mapsto \text{MHA}(\sigma_Q f_{\text{RMS}}(W_Q X), \sigma_K f_{\text{RMS}}(W_K X)) \quad (13)$$

where crucially f_{RMS} is not applied along the token axis as this can lead to leakage of information during training on next-token-prediction objectives. A similar problem also prevents us from using normalization directly on the QK product matrix, along with the observation that the intuition of normalizing vectors so that their dot product is equal to the cosine similarity is lost once the dot products have already occurred [Henry et al., 2020]. Empirically, we find that the scale parameters σ_Q and σ_K don't seem to be strictly necessary for expressivity, and that networks can even form selective attention masks for in-context learning without using these parameters to further saturate the softmax.

A.5 Dynamics of NaP

Weight projection non-interference: NaP incorporates a projection onto the ball of constant norm after each update step. A natural question is whether this projection step might simply be the inverse of the update step, leaving the parameters of the network constant. Fortunately, we note that the normalization layers have the effect of projecting gradients onto a subspace which is orthogonal to the current parameter values, i.e.

$$\left(\nabla_{\mathbf{x}} f_{\text{RMS}}(\mathbf{x}) \right) (\mathbf{x}) = \mathbf{0} . \quad (14)$$

We also note that except for extreme situations such as Neural Collapse [Papayan et al., 2020], real-world gradient updates are almost never colinear with the parameters, meaning that even without normalization layers this problem would be unlikely.

Another concern that arises from the constraints we place on the weights and features is the possibility that these constraints will limit the network's expressivity. Normalization does remove the ability to distinguish colinear inputs of differing norms, meaning that the inputs $\mathbf{x}$ and $\alpha \mathbf{x}$ will map to the same output for all α ; however, since many data preprocessing pipelines already normalize inputs, we argue this is not a significant limitation. Indeed, under a more widely-used notion of expressivity, the number of *activation patterns* [Raghu et al., 2017], NaP does not limit expressivity at all. While straightforward, we provide a formal statement and proof of this claim in Appendix A.7.

Layer normalization and parameter growth: In fact, if we incorporate normalization layers into the network we might expect an even more aggressive decay schedule. Recall that in a scale invariant network, we have $\langle \nabla_{\theta} f(\theta), \theta \rangle = 0$. Thus we know that the gradient at each time step will be orthogonal to the current parameters. In an idealized setting where we use the update rule $\theta_{t+1} \leftarrow \theta_t + \alpha \frac{\nabla_{\theta} \ell(\theta_t)}{\|\nabla_{\theta} \ell(\theta_t)\|}$, this would result in the parameter norm growing at a rate $\Theta(t)$, corresponding to an effectively linear learning rate decay.

A.6 Scale-invariance and layer-wise gradient norms

One benefit of NaP is that, because we normalize layer outputs, we limit the extent to which divergence in the norm of one layer's parameters can propagate to the gradients of other layers. For e.g. linear homogeneous activations such as ReLUs, the gradient of some objective function for some input with respect to the parameters of a particular layer contains a sum of matrix products whose norm will depend multilinearly on the norm of each matrix. In particular, in the simplified setting of a deep linear network where $f(\theta, \mathbf{x}) = \prod W^l \mathbf{x}$, we recall [Saxe et al., 2013]

$$\nabla_{W^l} f(\theta; \mathbf{x}) = \left[\prod_{k>l} W^k \right]^{\top} \mathbf{x}^{\top} \left[\prod_{k<l} W^k \right]^{\top} \quad (15)$$

In particular, with $\theta' = W^1, \dots, cW^k, \dots, W^L$, for $k \neq l$ we would have

$$\implies \nabla_{W^l} f(\theta'; \mathbf{x}) = c \nabla_{W^l} f(\theta; \mathbf{x}) \quad (16)$$

The situation changes little if we add ReLU nonlinearities to the network. In this case, we use the notation $\mathbf{D}_{\phi_l}(\mathbf{x})$ to denote the diagonal matrix indicating whether $a^l[i] > 0$

$$\nabla_{W^l} f(\theta; \mathbf{x}) = \left[\prod_{k>l} \mathbf{D}_{\phi_k}(\mathbf{x}) W^k \right]^\top \mathbf{x}^\top \left[\prod_{k<l} \mathbf{D}_{\phi_k}(\mathbf{x}) W^k \right]^\top \quad (17)$$

$$\implies \nabla_{W^l} f(\theta'; \mathbf{x}) = c \nabla_{W^l} f(\theta; \mathbf{x}) \quad (18)$$

If we incorporate a normalization layer at the end of the network (for simplicity we consider RMSNorm here, but a similar argument applies to standard LayerNorm), the scale-invariance of the resulting output means that the norms of each layer's gradients are independent of the norms of the other layers' parameters, i.e.

$$\nabla_{W^l} f_{\text{RMS}} \circ f(\theta; \mathbf{x}) = \nabla_{W^l} f_{\text{RMS}} \circ f(\theta'; \mathbf{x}) \text{ whenever } \theta' = (W_1, \dots, cW_k, \dots, W_L), k \neq l \quad (19)$$

This property is appealing as it means that growth or decay of the norm of a single layer will not interfere with the dynamics of the others. However, it does mean that a layer's effective learning rate will still be sensitive to scaling, which motivates our use of renormalization. It also does not help to avoid saturated units, motivating our use of layer normalization prior to nonlinearities.

A.7 Expressivity of NaP

Finally, we discuss the effect of normalization and weight projection on a notion of expressivity known as the number of activation patterns [Raghu et al., 2017] exhibited by a neural network. This quantity relates to the complexity of the function class a network can compute, giving the following result the corollary that NaP doesn't interfere with this notion of expressivity.

Proposition 2. *Let f be a fully-connected network with ReLU nonlinearities. Let $\tilde{f}$ be the function computed by f after applying NaP. Then the activation pattern of a particular architecture f and parameter θ be $\mathcal{A}_\theta$, we have*

$$\mathcal{A}_f(\theta, \mathbf{x}) = \mathcal{A}_{\tilde{f}}(N(\theta), \mathbf{x}). \quad (20)$$

Further, the decision boundary $\max_{i \in d_{\text{out}}} f_i(\mathbf{x})$ is preserved under NaP.

Proof. We apply an inductive argument on each layer. In particular, when ϕ is a ReLU nonlinearity we have

$$\begin{aligned} \mathcal{A}(\phi(\mathbf{h})) &= \mathcal{A}(\phi(f_{\text{RMS}}(\mathbf{h}))) \\ \phi(f_{\text{RMS}}(\mathbf{h})) &= \frac{\phi(\mathbf{h})}{\|\mathbf{h}\|} \\ \tilde{f}(\mathbf{x}) &= \frac{1}{\prod_{l=1}^L \|\mathbf{h}_l(\mathbf{x})\|} f(\mathbf{x}) \end{aligned}$$

which trivially results in identical activation patterns in the normalized and unnormalized networks. It is worth noting that one distinguishing factor from a standard ReLU network is that the resulting scaling factor will be different for each $\mathbf{x}$. Thus while the activation patterns will be the same, the two different inputs $\mathbf{x}$ $\mathbf{y}$ might have different scaling factors, which will be a nonlinear function of the input. NaP networks, even with ReLU activations, thus do not have the property of being piecewise linear. $\square$

A.8 Rescaling scale/offset parameters (linear homogeneous networks)

We observe that for any $c > 0$, letting $\phi(x) = \max(x, 0)$ we have:

$$f_{\text{LN}}(\phi(W\sigma\mathbf{x} + \mu)) = f_{\text{LN}}(c\phi(W\sigma\mathbf{x} + \mu)) \quad (21)$$

$$= f_{\text{LN}}(\phi(cW(\sigma\mathbf{x} + \mu))) \text{ by homogeneity of ReLU} \quad (22)$$

$$= f_{\text{LN}}(\phi(W(c\sigma\mathbf{x} + c\mu))) \quad (23)$$

Then we obtain analogous effective learning rates, letting

$$g_{c\mu} = \nabla_{\mu} f(c\sigma, c\mu; \mathbf{x}) \quad g_{c\sigma} = \nabla_{\sigma} f(c\sigma, c\mu; \mathbf{x}) \quad (24)$$

we then have equivalent updates for $\|\sigma^2 + \mu^2\| = 1$ and $\eta_c = c^2$

$$f_{\text{LN}}((c\sigma + \eta_c g_{c\sigma})\mathbf{x} + c\mu + \eta_c g_{c\mu}) = f_{\text{LN}}((c\sigma + c^2 g_{c\sigma})\mathbf{x} + c\mu + c^2 g_{c\mu}) \quad (25)$$

$$= f_{\text{LN}}((c\sigma + c^2 \frac{1}{c} g_{\sigma} + c\mu + c^2 \frac{1}{c} g_{\mu}) \quad (26)$$

$$= f_{\text{LN}}(c((\sigma + g_{\sigma})\mathbf{x} + \mu + g_{\mu})) = f_{\text{LN}}((\sigma + g_{\sigma})\mathbf{x} + \mu + g_{\mu}) \quad (27)$$

Finally, we note that the above also holds if we apply a linear transformation W to the output of the scale-offset transform, since

$$f_{\text{LN}}(W(c\sigma\mathbf{x} + c\mu)) = f_{\text{LN}}(cW(\sigma\mathbf{x} + \mu)) = f_{\text{LN}}(W(\sigma\mathbf{x} + \mu)) \quad (28)$$

and so the effective learning rate scales precisely as we had previously for linear layers, but now with respect to the joint norm of the scale and offset parameters μ, σ .

B Experiment details

B.1 Toy experiment details

We conduct a variety of illustrative experiments on toy problem settings and small networks.

Network: in Figures 1 and 2, we use a DQN-style network which consists of two sets of two convolutional layers with 32 and 64 channels respectively. We then apply max pooling and flatten the output, feeding through a 512-unit hidden linear layer before applying a final linear transform to obtain the output logits. The network uses ReLU nonlinearities. When NaP is applied, we add layer normalization prior to each nonlinearity.

B.2 RL details

Single-task atari: We base our RL experiments off of the publicly available implementation of the Rainbow agent [Hessel et al., 2018] in DQN Zoo [Quan and Ostrovski, 2020]. We follow the default hyperparameters detailed in this codebase. In our implementation, we add normalization layers prior to each nonlinearity except for the final softmax. We train for 200M frames on the Atari 57 suite [Bellemare et al., 2013]. We also allow for a learning rate schedule, which we explicitly detail in cases where non-constant learning rates are used.

Sequential ALE: we use the same rainbow implementation as for the single-task results, using a cosine decay learning rate for all variants. We restart the cosine decay schedule at every task change for all agents. Our cosine decay schedule uses an init value of 10^{-8} , a peak value of the default LR for Rainbow (0.000625), 1000 warmup steps after the optimizer is reset, and end-value equal to 10^{-6} as in the single-task settings. We choose cosine decay due to its popularity in supervised learning, and to highlight the versatility of NaP to different LR schedules. We follow the game sequence used by Abbas et al. [2023], training for 20M frames per game.

B.3 Language details

Sequence memorization: we set a dataset size of 1024 and a sequence length of 512. We use a vocabulary size of 256, equivalent to ASCII tokenization. We use the adam optimizer, and train all networks for a minimum of 10 000 steps. We reset the dataset every 1000 optimizer steps, generating a new set of 1024 random strings of length 512. We use as a baseline a transformer architecture [Vaswani et al., 2017, Raffel et al., 2020] consisting of 4 attention blocks, with 8 heads and d_{model} equal to 256. We use a batch size of 128.

In-context learning: our in-context learning experiments use the same overall setup as the sequence memorization experiments, with identical architectures and baseline optimization algorithm. In this

case we train on a dataset consisting of 4096 randomly generated strings, in which the final 100 tokens are a contiguous subsequence of the first 412, selected uniformly at random from indices in [1, 312].

Natural language: we run our natural language experiments on a 400M parameter transformer architecture based on the same backbone as the previous two tasks, this time consisting of 12 blocks with 12 heads and model dimension 1536. We use the standard practice of learning rate warmup followed by cosine decay, setting a peak learning rate of $2 \cdot 10^{-4}$ which is reached after a linear warmup of 1000 steps. We use a batch size of 128 and train for 30,000 steps. We use a weight decay parameter of 0.1 with the adamw optimizer.

B.4 Vision details

CIFAR-10 Memorization: We consider three classes of network architecture: a fully-connected multilayer perceptron (MLP), for which we default to a width of 512 and depth of 4 in our evaluations; a convolutional network with k convolutional layers followed by two fully connected layers, for which we default to depth four, 32 channels, and fully-connected hidden layer width of 256. In all networks, we apply layer normalization before batch normalization if both are used at the same time. By default, we typically do not use batch normalization. We use ReLU activations

Our continual supervised learning domain is constructed from the CIFAR-10 dataset, from which we construct a family of continual classification problems. Each continual classification problem is characterized by a transformation function on the labels. For label transformations, we permute classes (for example, all images with the label 5 will be re-assigned the label 2), and random label assignment, where each input is uniformly at random assigned a new label independent of its class in the underlying classification dataset. Figures in the main paper concern random label assignments, as this is a more challenging task which produces more pronounced effects.

In our figures in the main paper, we run a total of 20M steps and a total of 200 random target resets. All networks are trained using the Adam optimizer. We conducted a sweep over learning rates for the different architectures, settling on 10^{-4} as this provided a reasonable balance between convergence speed and stability in all architectures, to ensure that all networks could at least solve the single-task version of each label and target transformation.

VGGNet and ResNet-50 baselines: we use the standard data augmentation policies for the CIFAR-10 and ImageNet-1k experiments. In our ImageNet-1k experiments, we use the ResNetv2 architecture variant, with a label smoothing parameter of 0.1, weight decay 10^{-4} , and as an optimizer we use SGD with a cosine annealing learning rate schedule, and Nesterov momentum with decay rate 0.9. We use a batch size of 256. Our CIFAR-10 experiments use a VGG-Net architecture. We use the sgd optimizer with a batch size of 32, and set a learning rate schedule which starts at 0.025 and decays by a factor of 0.1 iteratively through training. We use Nesterov momentum with decay rate 0.9. We train for a total of 400 000 steps.

C Additional experimental results

C.1 Arcade learning environment

Our choice of learning rate schedule in the paper is motivated by an attempt to roughly approximate the shape of the implicit schedule obtained by parameter growth on average across games in the suite, with a slightly smaller terminal point than would typically be achieved by the parameter norm alone. We consider learning rate schedules which linearly interpolate between the initial learning rate of 0.000625 and a learning rate of 10^{-6} , which is roughly equivalent to increasing the parameter norm by a factor of 60. We explore the importance of the duration of this decay in Figure 6, where we conduct a sweep over a subset of the full Atari 57 suite (selected by sorting alphabetically and selecting every third environment). We observe that faster decay schedules result in better performance initially, but often plateau at a lower value. Decaying linearly over the entire course of training, in contrast, exhibits slow initial progress but often picks up significantly towards the end of training. We conclude that in many games, reducing the learning rate is necessary for performance improvements in this agent, and the linear decay over the entire training period doesn't give the agent sufficient time to take advantage of the finer-grained updates to its predictions that a lower learning rate affords.

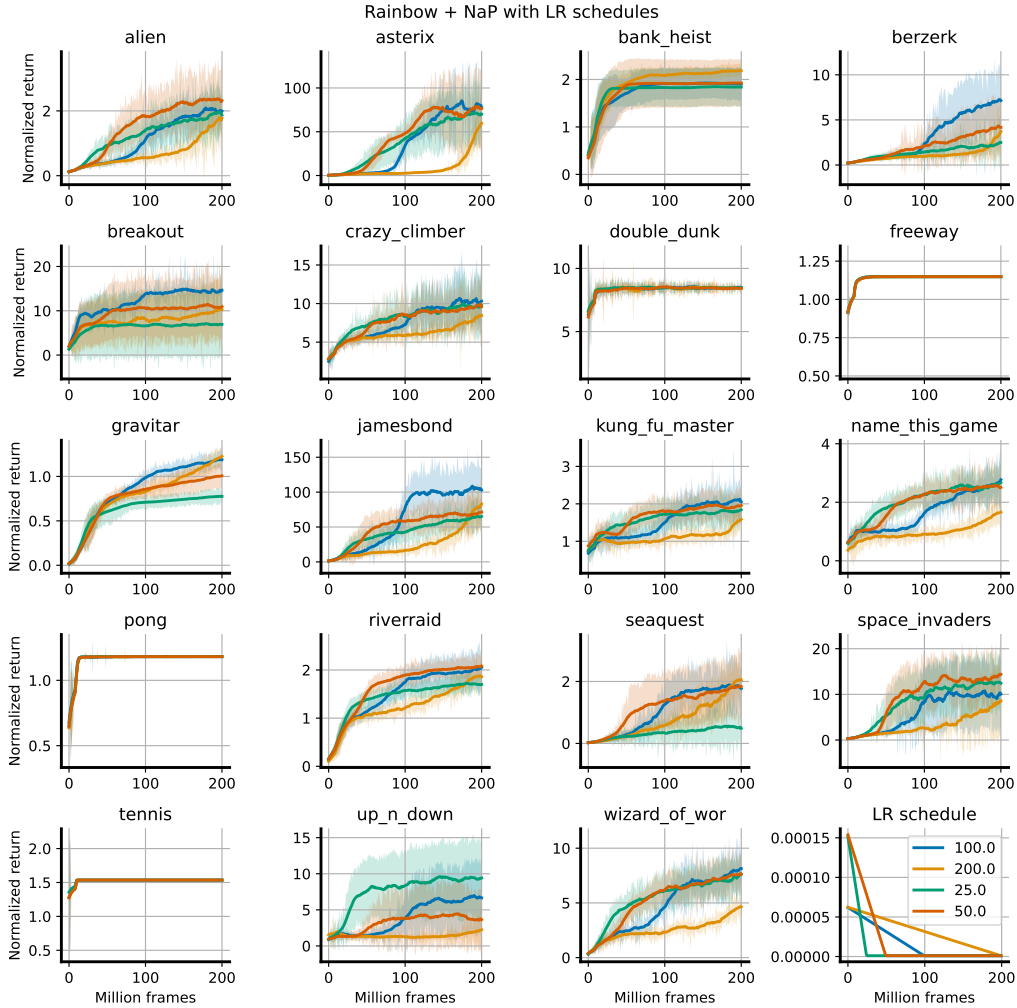


Figure 6: We see that faster LR decays typically accompany fast initial progress followed by plateaus. Terminating the linear schedule halfway through training strikes the best balance of the four settings considered for overall progress.

C.2 Non-stationary MNIST

We include additional network statistics from the experiments shown in Figure 4 in Figure 7

Linearized units: given a large batch, what fraction of the ReLU units in the penultimate layer are either 0 for all units or nonzero for all units. This gives a slightly more nuanced take on the amount of computation performed in the penultimate layer than the feature rank.

Feature rank: we compute the numerical rank of the penultimate layer features by sampling a batch of data and computing the singular values of the $b \times d$ matrix of $d \cup$ dimensional feature vectors. $\sigma_1, \dots, \sigma_d$. We then compute the numerical rank as $\sum \mathbb{1}(\sigma_i / \sigma_1 > 0.01)$.

Parameter and gradient norm: these are both computed in the standard way by flattening out the set of parameters / per-parameter gradients and computing the 2-norm of this vector.

C.3 Non-stationary sequence modeling

We find additionally that NaP is capable of improving the robustness of sequence models to nonstationarities, while also not interfering with the formation of in-context learning circuits. We demonstrate

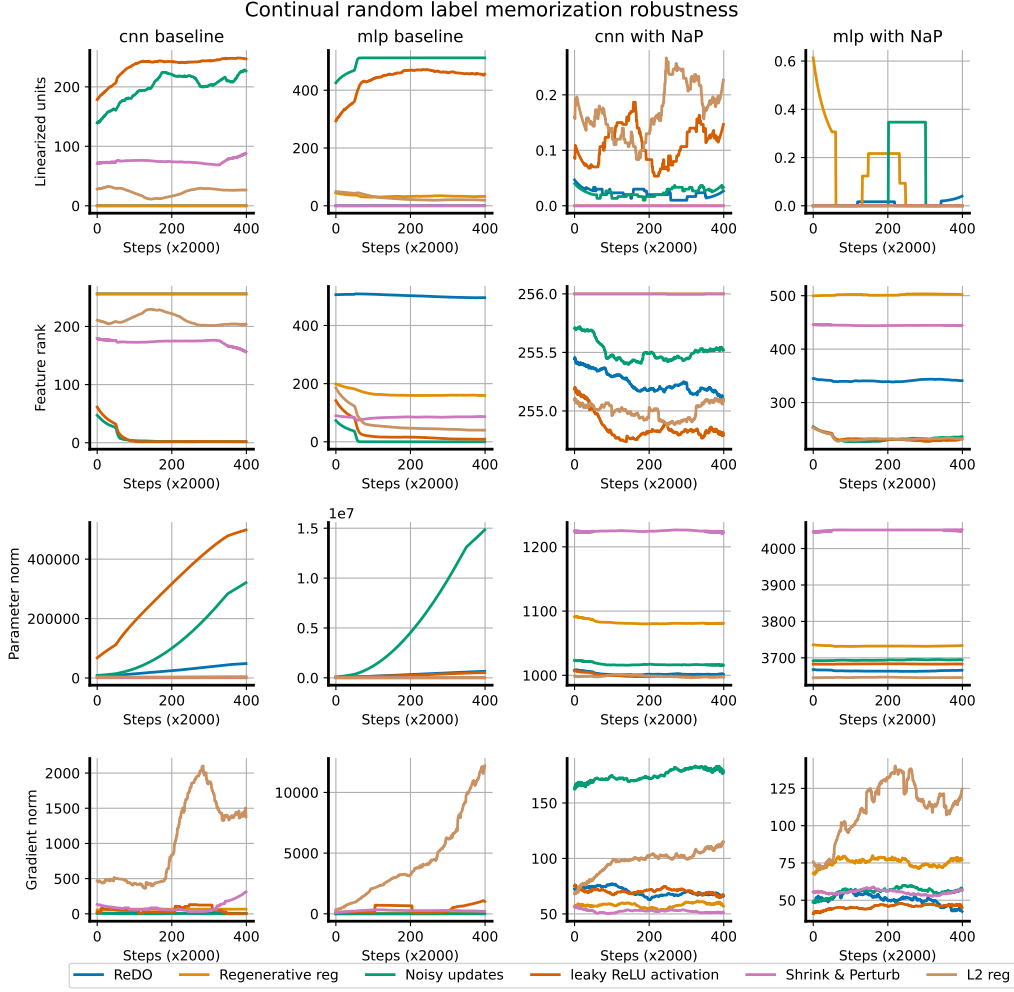


Figure 7: We plot a variety of additional network statistics in the continual CIFAR-10 experiment shown in Figure 4

□

the latter point in Figure 8, where we train a small transformer model on a dataset of the form $s_p \oplus s_s$, where s_p is a prefix string of length 100 and s_s is a suffix string of length 50 which is a contiguous subset of the prefix string. We use a fixed dataset, such that in theory the network could memorize all strings, though we use a sufficiently large dataset that this is not possible to achieve within the training budget. We train four protocols using next-token prediction: with and without weight projection, and with and without normalization (the networks are small enough that normalization is not critical for training stability). We evaluate accuracy on the final 48 tokens of s_s as training progresses, and observe that all networks very quickly learn to identify the starting point of the suffix string and copy the relevant subset of the prefix. We observe that normalization accelerates learning of both the retrieval component of the accuracy and the memorization component of the accuracy, and that the weight projection step in fact also accelerates this process.

In Figure 9, we see that normalization and weight projection can also help to improve robustness in a nonstationary random string memorization task, a sequence-modelling analogue of random label memorization in CIFAR-10. We observe that in this case, allowing a learnable scale to evolve unregularized can somewhat slow down learning compared to omitting this parameter, and that weight projection recovers similar dynamics to learning with a large weight decay factor.

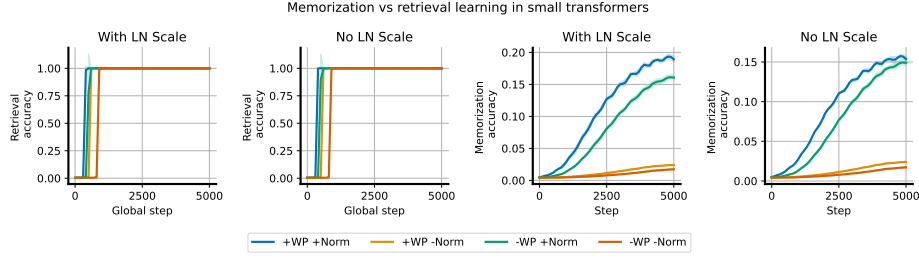


Figure 8: Demonstration that applying normalization and removing the learnable scale parameter does not prevent the network from learning to copy previously-observed subsequences.

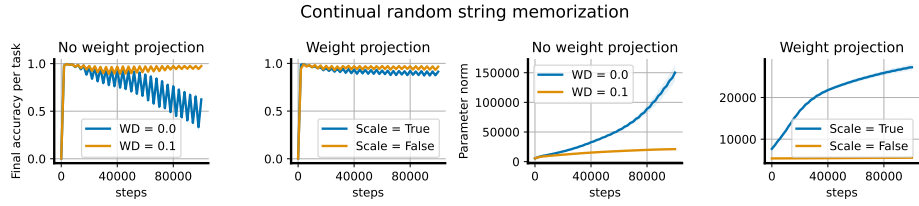


Figure 9: Random string memorization: unregularized networks exhibit plasticity loss when trained to memorize a sequence of random strings, while weight projection and weight decay improve robustness to this nonstationarity.

C.4 ReLU revival experiments

Many previous works have noted that adaptive optimizers are particularly damaging to network plasticity [Dohare et al., 2021, Lyle et al., 2023, Dohare et al., 2023]. The primary mechanism underlying this is due to the sudden distribution shift in gradient moments due to changes in the learning objective – when the gradient norm increases, adaptive optimizers are slow to catch up and can take enormous update steps when this occurs. Layer normalization mitigates this effect due to two facts: first, the gradients of negative preactivations are nonzero, and second, all nonzero gradients are treated essentially the same by adaptive optimizers (up to ϵ). As a result, networks with layer norm can still perform significant updates to parameters feeding into ‘dead’ units, meaning that these networks have a good chance of turning on again later.

We illustrate this with a simple experimental setting, where we model optimizer updates with isotropic Gaussian-distributed gradient signals and perform a (truncated at zero) random walk. Formally we look at the time evolution of the system:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \max(\mathbf{v}, \mathbf{0}) \odot \mathbf{z}_t, \quad \mathbf{z}_t \sim \mathcal{N}(0, I) \quad (29)$$

to model the evolution of features under a gradient descent trajectory. To simulate the steps taken by an adaptive optimizer like RMSProp or Adam, where updates to each parameter have fixed norm, we modify this process slightly as follows:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \text{sign}(\max(\mathbf{v}, \mathbf{0}) \odot \mathbf{z}_t), \quad \mathbf{z}_t \sim \mathcal{N}(0, I). \quad (30)$$

To simulate layer normalization, we compute the dot product between $\mathbf{z}_t$ and the Jacobian $\nabla_{\mathbf{v}} \frac{\mathbf{v}}{\|\mathbf{v}\|}$ to simulate gradient descent:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \mathbf{z}_t^\top \nabla_{\mathbf{v}} \max\left(\frac{\mathbf{v}}{\|\mathbf{v}\|}, \mathbf{0}\right), \quad \mathbf{z}_t \sim \mathcal{N}(0, I) \quad (31)$$

and analogously compute the sign of this update to model RMSProp-style optimizers:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \text{sign}\left(\mathbf{z}_t^\top \nabla_{\mathbf{v}} \max\left(\frac{\mathbf{v}}{\|\mathbf{v}\|}, \mathbf{0}\right)\right), \quad \mathbf{z}_t \sim \mathcal{N}(0, I) \quad (32)$$

In Figure 10 we simulate each of these processes for 1000 steps, and track the number of negative (i.e. ‘dead’) indices. We observe that layer normalization doesn’t avoid dead units in GD, but does reduce

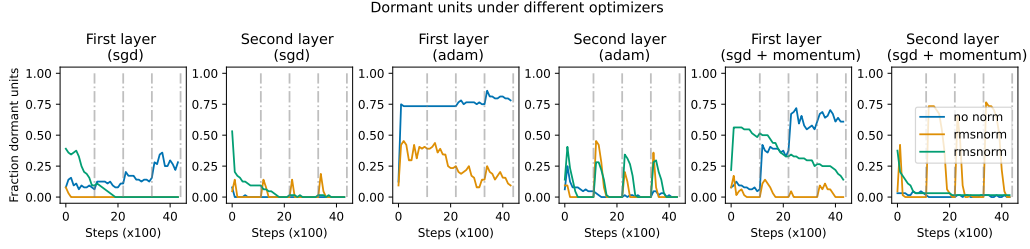


Figure 10: Simple MLP model with dead unit recovery after sudden changes in the classification task. We

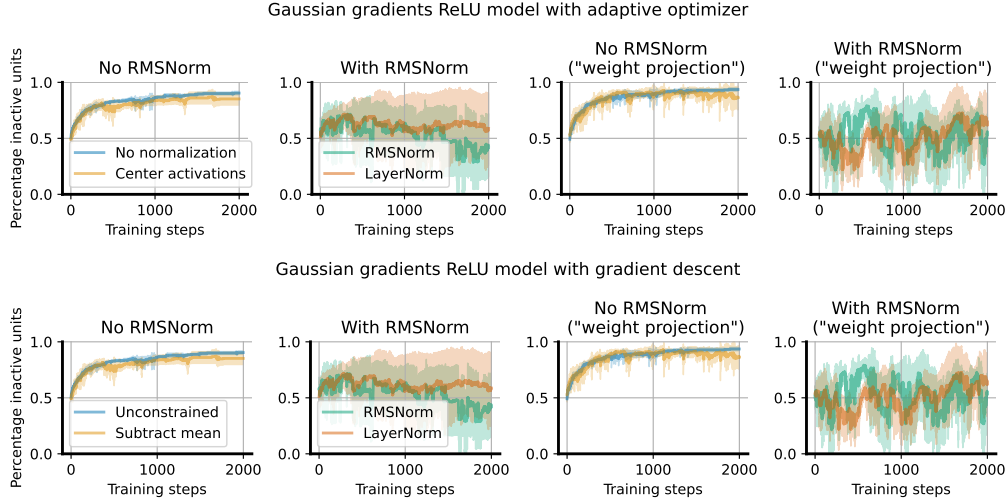


Figure 11: Accumulation of gradients in a random walk model: backpropagated ‘gradients’ are isotropic random Gaussian vectors and updates are computed by taking the product of these vectors with the layer jacobian. We see that the centering transform actually does relatively little to reduce the risk of dead units, and can in fact produce a ‘winner-take-all’ effect wherein one large

the rate at which they accumulate. We also observe that layer normalization does avoid monotonic increases in the number of dead units in a model of RMSProp. Intuitively, this makes sense: rather than freezing once they reach a negative value, parameters continue updating once they become negative with equally large steps as they did before, making escape from the dead zone more probable. One visualization of a few trajectories in each setting is shown in Figure 11.

C.5 Stationary supervised benchmarks

We provide the results with standard deviations in Table 2 and Table 3.

	CIFAR-10	ImageNet-1k
NaP	94.64 (0.12)	77.26 (0.04)
Baseline	94.65 (0.08)	77.08 (0.11)
Norm only	94.47 (0.18)	77.45 (0.08)

Table 2: Top-1 prediction accuracy on the test sets of CIFAR-10 and ImageNet-1k. Numbers in parentheses are standard deviations.

	C4	Pile	WikiText	Lambada	SIQA	PIQA
NaP	45.7 (0.0)	47.9 (0.1)	45.4 (0.1)	56.6 (0.4)	44.2 (0.2)	68.8 (0.7)
Baseline	44.8 (0.0)	47.4 (0.1)	44.2 (0.0)	54.1 (0.2)	43.5 (0.6)	67.3 (0.2)
Norm only	44.9 (0.0)	47.6 (0.1)	44.3 (0.0)	53.6 (0.3)	43.8 (0.6)	67.1 (0.4)

Table 3: Per-token accuracy of a 400M transformer model pretrained on the C4 dataset, evaluated on a variety of language benchmarks. Numbers in parentheses are standard deviations.

D A note on scale and offset parameters

One loose end from our presentation of NaP is what to do with the learnable scale and offset terms, which are not by default projected and so may drift from their initial values. Most supervised tasks are too short for this drift to present problems, and adding layer-specific regularization or normalization adds additional engineering overhead to an experiment. However, in deep RL or in the synthetic continual tasks we present in Figure 4, this is a real concern. In the case of homogeneous activations such as ReLU, the scale and offset parameters can be viewed identically to the weight and bias terms and normalized accordingly, noting that now all that matters is the relative ratio of the scale and the offset. To account for this, we propose to treat the joint set σ, μ as a single parameter to be normalized. This resolves the issues involved with normalizing a parameter to an initial value of zero, and can be shown not to change the network output (see Appendix A.8 for a derivation of this fact). With non-homogeneous nonlinearities, however, this property will not hold, and we suggest in the general case to use mild weight decay towards the initial values of 1 and 0 for the scale and offset terms respectively. These two approaches can be summarized in the following two update rules:

$$\mathcal{U}_{\text{norm}}(\sigma, \mu) = \frac{(\sigma, \mu)}{\sqrt{\|\sigma\|^2 + \|\mu\|^2}} \quad \text{and} \quad \mathcal{U}_{\text{decay}}^\alpha(\sigma, \mu) = (\alpha\sigma + (1 - \alpha)\mathbb{1}, \alpha\mu). \quad (33)$$

Most of our evaluations on single tasks do not use any regularization or projection of the scale and offset parameters, but we do include a regularization-based approach in our evaluations on the sequential ALE in Section 5. In general, if it did not appear that the scale/offset drift was causing problems, we did not introduce additional complexity by adding regularization or normalization. Indeed, in many cases a simpler solution was to omit these parameters entirely; for example we observed that removing offsets was beneficial in several, though not all, games in the Arcade Learning Environment.

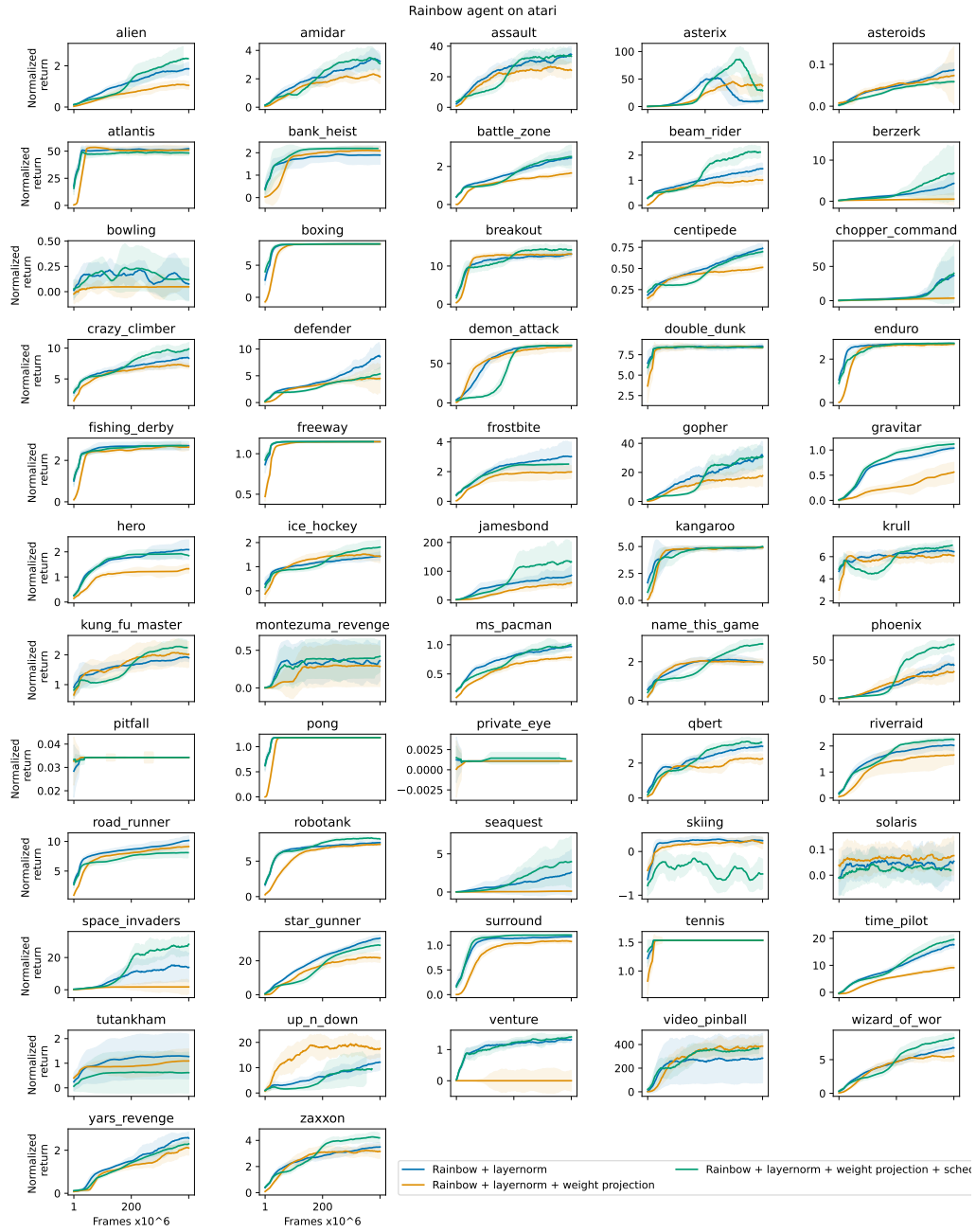


Figure 12: Rainbow agents on atari.

Scaling Off-Policy Reinforcement Learning with Batch and Weight Normalization

Daniel Palenicek^{1 2} Florian Vogt³ Jan Peters^{1 2 4 5}

Abstract

Reinforcement learning has achieved significant milestones, but sample efficiency remains a bottleneck for real-world applications. Recently, CrossQ has demonstrated state-of-the-art sample efficiency with a low update-to-data (UTD) ratio of 1. In this work, we explore CrossQ’s scaling behavior with higher UTD ratios. We identify challenges in the training dynamics, which are emphasized by higher UTD ratios. To address these, we integrate weight normalization into the CrossQ framework, a solution that stabilizes training, has been shown to prevent potential loss of plasticity and keeps the effective learning rate constant. Our proposed approach reliably scales with increasing UTD ratios, achieving competitive performance across 25 challenging continuous control tasks on the DeepMind Control Suite and Myosuite benchmarks, notably the complex `dog` and `humanoid` environments. This work eliminates the need for drastic interventions, such as network resets, and offers a simple yet robust pathway for improving sample efficiency and scalability in model-free reinforcement learning.

1. Introduction

Reinforcement Learning (RL) has shown great successes in recent years, achieving breakthroughs in diverse areas. Despite these advancements, a fundamental challenge that remains in RL is enhancing the sample efficiency of algorithms. Indeed, in real-world applications, such as robotics, collecting large amounts of data can be time-consuming, costly, and sometimes impractical due to physical constraints or safety concerns. Thus, addressing this is crucial to make RL

¹Intelligent Autonomous Systems Group, Technical University of Darmstadt, Germany ²hessian.AI, Darmstadt, Germany ³Department of Computer Science, University of Freiburg, Germany ⁴German Research Center for Artificial Intelligence (DFKI) ⁵Robotics Institute Germany (RIG). Correspondence to: Daniel Palenicek <daniel.palenicek@tu-darmstadt.de>.

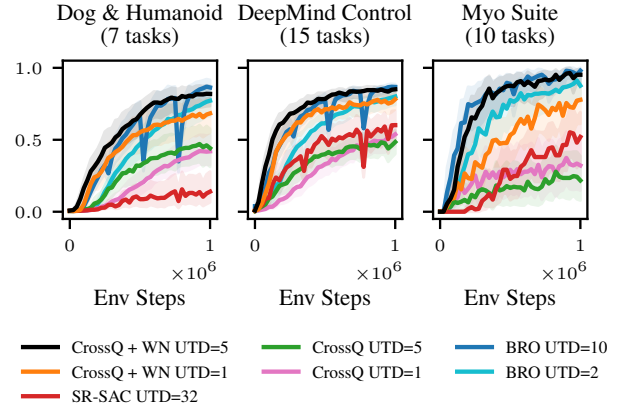


Figure 1. *CrossQ + WN UTD=5 is competitive to BRO UTD=10.* In comparison, our proposed CrossQ + WN is a simple algorithm that does not require extra exploration policies or full parameter resets. We present results for 25 complex continuous control tasks from the DMC and MyoSuite benchmarking suites. 1.0 marks the maximum score achievable on the respective benchmarks (DMC return up to 1000 / Myosuite up to 100% success rate).

methods more accessible and scalable.

Different approaches have been explored to address the problem of low sample efficiency in RL. Model-based RL, on the one hand, attempts to increase sample efficiency by learning dynamic models that reduce the need for collecting real data, a process often expensive and time-consuming (Sutton, 1990; Janner et al., 2019; Feinberg et al., 2018; Heess et al., 2015). Model-free RL approaches, on the other hand, have explored increasing the number of gradient updates on the available data, referred to as the update-to-data (UTD) ratio (Nikishin et al., 2022; D’Oro et al., 2022), modifying network architectures (Bhatt et al., 2024), or both (Chen et al., 2021; Hiraoka et al., 2021; Hussing et al., 2024; Nauman et al., 2024).

In this work, we build upon CrossQ (Bhatt et al., 2024), a recent model-free RL algorithm that recently showed state-of-the-art sample efficiency on the `MuJoCo` (Todorov et al., 2012) continuous control benchmarking tasks. Notably, the authors achieved this by carefully utilizing Batch Normal-

ization (BN, Ioffe (2015)) within the actor-critic architecture. A technique previously thought not to work in RL, as famously reported by Hiraoka et al. (2021) and others. The insight that Bhatt et al. (2024) offered is that one needs to carefully consider the different state-action distributions within the Bellman equation and handle them correctly to succeed. This novelty allowed CrossQ at a low UTD of 1 to outperform the then state-of-the-art algorithms that scaled their UTD ratios up to 20. Even though higher UTD ratios are more computationally expensive, they allow for larger policy improvements using the same amount of data.

This naturally raises the question: *How can we extend the sample efficiency benefits of CrossQ and BN to the high UTD training regime?* Which we address in this manuscript.

Contributions. In this work, we show that the vanilla CrossQ algorithm is brittle to tune on DeepMind Control (DMC) and Myosuite environments and can fail to scale reliably with increased compute. To address these limitations, we propose the addition of weight normalization (WN), which we show to be a simple yet effective enhancement that stabilizes CrossQ. We motivate the combined use of WN and BN on insights from the continual learning and loss of plasticity literature and connections to the effective learning rate. Our experiments show that incorporating WN not only improves the stability of CrossQ but also allows us to scale its UTD, thereby significantly enhancing sample efficiency.

2. Preliminaries

This section briefly outlines the required background knowledge for this paper.

Reinforcement learning. A Markov Decision Process (MDP) (Puterman, 2014) is a tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \mu_0, \gamma \rangle$, with state space $\mathcal{S} \subseteq \mathbb{R}^n$, action space $\mathcal{A} \subseteq \mathbb{R}^m$, transition probability $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$, the reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, initial state distribution μ_0 and discount factor γ . We define the RL problem according to Sutton & Barto (2018). A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ is a behavior plan, which maps a state s to a distribution over actions a . The discounted cumulative return is defined as

$$\mathcal{R}(s, a) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t),$$

where $s_0 = s$ and $a_0 = a$. Further, $s_{t+1} \sim \mathcal{P}(\cdot | s_t, a_t)$ and $a_t \sim \pi(\cdot | s_t)$. The Q-function of a policy π is the expected discounted return $Q^\pi(s, a) = \mathbb{E}_{\pi, \mathcal{P}}[\mathcal{R}(s, a)]$.

The goal of an RL agent is to find an optimal policy π^* that maximizes the expected return from the initial state distribution

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{s \sim \mu_0} [Q^\pi(s, a)].$$

Soft Actor-Critic (SAC, Haarnoja et al. (2018)) addresses this optimization problem by jointly learning neural network representations for the Q-function and the policy. The policy network is optimized to maximize the Q-values, while the Q-function is optimized to minimize the Bellman error

$$\mathcal{L} = \mathbb{E}_{\mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - \left(r(s_t, a_t) + \gamma \mathbb{E}_{\mathcal{P}}[V(s_{t+1})] \right) \right)^2 \right],$$

where the value function is computed by taking an expectation over the learned Q function

$$V(s_{t+1}) = \mathbb{E}_{\mathcal{P}, \pi_\theta} [Q_{\bar{\theta}}(s_{t+1}, a_{t+1})]. \quad (1)$$

To stabilize the Q-function learning, Haarnoja et al. (2018) found it necessary to use a target Q-network in the computation of the value function instead of the regular Q-network. The target Q-network is structurally equal to the regular Q-network, and its parameters $\bar{\theta}$ are obtained via Polyak Averaging over the learned parameters θ . While this scheme ensures stability during training by explicitly delaying value function updates, it also arguably slows down online learning (Plappert et al., 2018; Kim et al., 2019; Morales, 2020).

Instead of relying on target networks, CrossQ (Bhatt et al., 2024) addresses training stability issues by introducing Batch Normalization (BN, Ioffe (2015)) in its Q-function and achieves substantial improvements in sample and computational efficiency over SAC. A central challenge when using BN in Q networks is distribution mismatch: during training, the Q-function is optimized with samples s_t, a_t from the replay buffer. However, when the Q-function is evaluated to compute the target values (Equation (1)), it receives actions sampled from the current policy $a_{t+1} \sim \pi_\theta(\cdot | s_{t+1})$. Those samples have no guarantee of lying within the training distribution of the Q-function. BN is known to struggle with out-of-distribution samples, as such, training can become unstable if the distribution mismatch is not correctly accounted for (Bhatt et al., 2024). To deal with this issue, CrossQ removes the separate target Q-function and evaluates both Q values during the critic update in a single forward pass, which causes the BN layers to compute shared statistics over the samples from the replay buffer and the current policy. This scheme effectively tackles distribution mismatch problems, ensuring that all inputs and intermediate activations are effectively forced to lie within the training distribution.

Normalization techniques in RL. Normalization techniques are widely recognized for improving the training of neural networks, as they generally accelerate training and improve generalization (Huang et al., 2023). There are many ways of introducing different types of normalizations into the RL framework. Most commonly, authors have used Layer Normalization (LN) within the network architectures to stabilize training (Hiraoka et al., 2021; Lyle et al., 2024). Recently, CrossQ has been the first algorithm to successfully

use BN layers in RL (Bhatt et al., 2024). The addition of BN leads to substantial gains in sample efficiency. In contrast to LN, however, one needs to carefully consider the different state-action distributions within the critic loss when integrating BN. In a different line of work, Hussing et al. (2024) proposed the integration of unit ball normalization and projected the output features of the penultimate layer onto the unit ball in order to reduce Q-function overestimation.

Increasing update-to-data ratios. Although scaling up the UTD ratio is an intuitive approach to increase the sample efficiency, in practice, it comes with several challenges. Nikishin et al. (2022) demonstrated that overfitting on early training data can inhibit the agent from learning anything later in the training. The authors dub this phenomenon the primacy bias. To address the primacy bias, they suggest to periodically reset the network parameters while retraining the replay buffer. Many works that followed have adapted this intervention (D’Oro et al., 2022; Nauman et al., 2024). While often effective, regularly resetting is a very drastic intervention and by design induces regular drops in performance. Since the agent has to start learning from scratch repeatedly, it is also not very computing efficient. Finally, the exact reasons why parameter resets work well in practice are not yet well understood (Li et al., 2023). Instead of resetting there have also been other types of regularization that allowed practitioners to train stably with high UTD ratios. Janner et al. (2019) generate additional modeled data, by virtually increasing the UTD. In REDQ, Chen et al. (2021) leverage ensembles of Q-functions, while Hiraoka et al. (2021) use dropout and LN to effectively scale to higher UTD ratios.

3. CrossQ fails to scale up stably

Bhatt et al. (2024) demonstrated CrossQ’s state-of-the-art sample efficiency on the MuJoCo task suite (Todorov et al., 2012), while at the same time also being very computationally efficient. However, on the more extensive DMC and Myosuite task suites, we find that CrossQ requires tuning. We further find that it works on some, but not all, environments stably and reliably.

Mixed performance of CrossQ. Figure 2 shows CrossQ training performance on a subset of DMC tasks. Namely, the dog-stand, dog-trot and humanoid-walk, selected for their varying difficulty levels to demonstrate a wide range of behaviors, including both successful learning and challenges encountered during training. The figure compares a SAC baseline with standard hyperparameters against tuned CrossQ agents with UTD ratios of 1 and 5, where the hyperparameters were identified through a grid search over learning rates and network sizes, as detailed in Table 1. The first row of the figure shows the IQM training

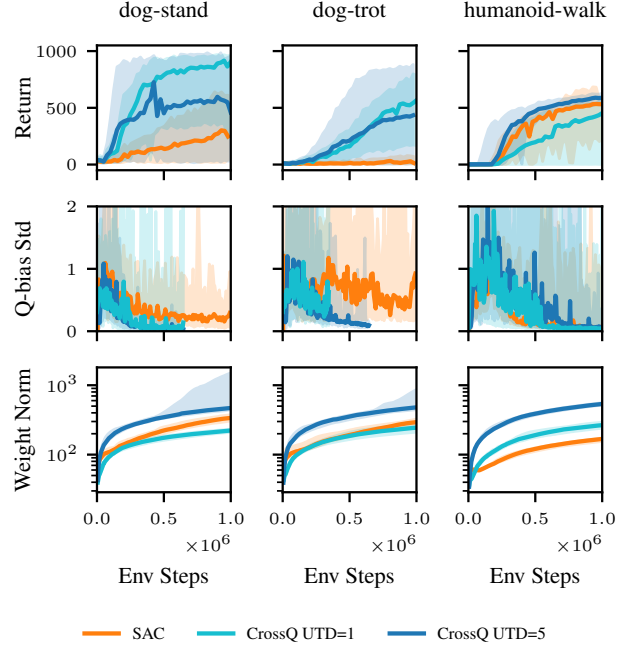


Figure 2. *Q-bias and weight norms.* CrossQ critic weight norms increase significantly with increasing UTD ratios.

performance and 95% confidence intervals for each agent across 10 seeds. Here, we identify three different training behaviors. On dog-stand CrossQ trains stably at UTD= 1, but increasing the UTD to 5 introduces instabilities and decreases performance. On dog-trot, both UTD ratios perform very similarly. Finally, on humanoid-walk, UTD=5 outperforms UTD=1, although it merely manages to catch up to the SAC baseline in this case. Overall, for all CrossQ runs we notice very large confidence intervals.

Q-function bias analysis. The second row in Figure 2 shows the standard deviation of the normalized Q-function bias. This bias measures how much the Q-function is overestimating or underestimating the true expected return of the current policy.

To compute the normalized Q-function bias, we follow the protocol of Chen et al. (2021). We gather 5 trajectories in the environment using the current policy and use each trajectory’s first 350 state-action pairs to calculate the bias. For this, we compare the cumulative discounted rewards for each state-action pair with its Q-value predicted by the Q-function. This bias is then normalized using the cumulative discounted rewards. The mean over these normalized Q-function biases measures the expected bias. Even if the mean is high, as long as the bias is consistent, the selected actions of the policy do not change and, therefore, it is not a problem (Van Hasselt et al., 2016). If the standard deviation

is high, the change in bias is high, which might hinder learning. Thus, following the work of [Chen et al. \(2021\)](#); [Bhatt et al. \(2024\)](#), we focus on the standard deviation.

We find large fluctuations for the Q-function bias standard deviation with all three agents across environments. And even on `dog-stand`, where CrossQ UTD=5 does not learn reliably, it maintains a small Q-function bias standard deviation. From these mixed results, we conclude that we cannot directly link the standard deviation of the Q-function bias to the learning performance. While this does not mean that for larger UTD ratios, the Q-bias would not explode. Rather, it means that, in our case, the Q-bias is not at fault, and the root cause for unstable training lies elsewhere.

Growing network parameter norms. The third row of Figure 2 displays the sum over the L2 norms of the dense layers in the critic network. This includes three dense layers, each with a hidden dimension of 512.

All three baselines exhibit growing network weights over the course of training. We find that the effect is particularly pronounced for CrossQ with increasing UTD ratios. We further find that in the second training phase, the spread of network weight norms increases for the `dog` runs with large confidence intervals. This is visualized by the large spread of the shaded areas, which show the 95% inter percentile ranges for the weight norms.

Growing network weights have been linked to a loss of plasticity, a phenomenon where networks become increasingly resistant to parameter update, which can lead to premature convergence ([Elsayed et al., 2024](#)). Additionally, the growing magnitudes pose a challenge for optimization, connected to the issue of growing activations, which has recently been analyzed by [Hussing et al. \(2024\)](#). Further, growing network weights decrease the effective learning rate when the networks contain normalization layers ([Van Hasselt et al., 2019](#); [Lyle et al., 2024](#)).

In summary, the scaling results for vanilla CrossQ are mixed. While increasing UTD ratios is known to yield increased sample efficiency, if careful regularization is used ([Janner et al., 2019](#); [Chen et al., 2021](#); [Nikishin et al., 2022](#)), CrossQ alone with BN cannot benefit from it. We notice that with increasing UTD ratios, CrossQ’s weight layer norms grow significantly faster and overall larger. This observation motivates us to further study the weight norms in CrossQ to increase UTD ratios.

4. Combining batch normalization and weight normalization for scaling up

Inspired by the combined insights of [Van Hasselt et al. \(2019\)](#) and [Lyle et al. \(2024\)](#), we propose to integrate CrossQ with Weight Normalization (WN) as a means of

counteracting the rapid growth of weight norms we observe with increasing update-to-data (UTD) ratios.

Our approach is based on the following reasoning: Due to the use of BN in CrossQ, the critic network exhibits scale invariance, as previously noted by [Van Laarhoven \(2017\)](#).

Theorem 4.1 ([Van Laarhoven \(2017\)](#)). *Let $f(\mathbf{X}; \mathbf{w}, b, \gamma, \beta)$ be a function, with inputs $\mathbf{X}$ and parameters $\mathbf{w}$ and b and γ and β batch normalization parameters. When f is normalized with batch normalization, f becomes scale-invariant with respect to its parameters, i.e.,*

$$f(\mathbf{X}; c\mathbf{w}, cb, \gamma, \beta) = f(\mathbf{X}; \mathbf{w}, b, \gamma, \beta),$$

with scaling factor $c > 0$.

Proof. Appendix A

This property allows us to introduce WN as a mechanism to regulate the growth of weight norms in CrossQ without affecting the critic’s outputs. Further, it can be shown, that for such a scale invariant function, the gradient scales inversely proportionally to the scaling factor $c > 0$.

Theorem 4.2 ([Van Laarhoven \(2017\)](#)). *Let $f(\mathbf{X}; \mathbf{w}, b, \gamma, \beta)$ be a scale-invariant function. Then, the gradients of f scale inversely proportional to the scaling factor $c \in \mathbb{R}$ of its parameters $\mathbf{w}$,*

$$\nabla f(\mathbf{X}; c\mathbf{w}, cb, \gamma, \beta) = \nabla f(\mathbf{X}; \mathbf{w}, b, \gamma, \beta)/c.$$

Proof. Appendix B

Recently, [Lyle et al. \(2024\)](#) demonstrated that the combination of LN and WN can help mitigate loss of plasticity. Since the gradient scale is inversely proportional to c , keeping norms constant helps to maintain a stable effective learning rate (ELR, [Van Hasselt et al. \(2019\)](#)), further enhancing training stability.

We conjecture that maintaining a stable ELR could also be beneficial when increasing the UTD ratios in continuous control RL. As the UTD ratio increases, the networks are updated more frequently with each environment interaction. Empirically, we find that the network norms tend to grow quicker with increased UTD ratios (Figure 2), which in turn decreases the ELR even quicker and could be the case for instabilities and low training performance. As such, we empirically investigate the effectiveness of combining CrossQ with WN with increasing UTD ratios.

Implementation details. We apply WN to the first two linear layers, ensuring that their weights remain unit norm after each gradient step by projecting them onto the unit ball, similar to [Lyle et al. \(2024\)](#). Since this constraint effectively stabilizes the ELR in these layers, we found it beneficial to slightly reduce the learning rate from $1e\text{U}3$ to $1e\text{U}4$. While

we could employ a learning rate schedule (Lyle et al., 2024) we did not investigate this here. Additionally, we impose weight decay on all parameters that remain unbounded—specifically the final dense output layer. In practice, we use AdamW (Loshchilov, 2017) with a decay of 0 (which falls back to vanilla Adam (Kingma, 2014)) for the normalized intermediate dense layers and $1e\text{U}2$ otherwise. We chose a maximum UTD of 5 for our experiments due to our computational budget and good strong in sample efficiency.

Target networks. CrossQ famously removed the target networks from the actor-critic framework and showed that using BN training remains stable even without them (Bhatt et al., 2024). While we find this to be true in many cases, we find that especially in DMC, the re-integration of target networks can help stabilize training overall (see Section 5.4). However, not surprisingly, we find that the integration of target networks with BN requires careful consideration of the different state-action distributions between the s, a and $s', a' \sim \pi(s')$ exactly as proposed by Bhatt et al. (2024). To satisfy this, we keep the joined forward pass through both the critic network as well as the target critic network. We evaluate both networks in *training mode*, i.e., they calculate the joined state-action batch statistics on the current batches. As is common, we use Polyak-averaging with a $\tau = 0.005$ from the critic network to the target network.

5. Experiments

To evaluate the effectiveness of our proposed CrossQ + WN method, we conduct a comprehensive set of experiments on the DeepMind Control Suite (Tassa et al., 2018) and MyoSuite (Caggiano et al., 2022) benchmarks. Our primary goal is to investigate the scalability of CrossQ + WN with increasing UTD ratios and to assess the stabilizing effects of combining CrossQ with WN. We compare our approach to several baselines, including the recent BRO (Nauman et al., 2024), CrossQ (Bhatt et al., 2024) and SR-SAC (D’Oro et al., 2022) a version of SAC (Haarnoja et al., 2018) with high UTD ratios and network resets.

5.1. Experimental setup

Our implementation is based on the SAC implementation of `jaxrl` codebase (Kostrikov, 2021). We implement CrossQ following the author’s original codebase and add the architectural modifications introduced by (Bhatt et al., 2024), incorporating batch normalization in the actor and critic networks. We extend this approach by introducing WN to regulate the growth of weight norms and prevent loss of plasticity and add target networks. We perform a grid search to focus on learning rate selection and layer width.

We evaluate 25 diverse continuous control tasks, 15 from DMC and 10 from MyoSuite. These tasks vary significantly

in complexity, requiring different levels of fine motor control and policy adaptation with high dimensional state spaces up to $\mathbb{R}^{223}$.

Each experiment is run for 1 million environment steps and across 10 random seeds to ensure statistical robustness. We evaluate agents every 25,000 environment steps for 5 trajectories. As proposed by Agarwal et al. (2021), we report the interquartile mean (IQM) and 95% stratified bootstrap confidence intervals (CIs) of the return, if not otherwise stated.

For the BRO baseline results, for computational reasons, we take the official evaluation data the authors provide. The official BRO codebase is also based on `jaxrl`, and the authors followed the same evaluation protocol, making it a fair comparison.

5.2. Weight normalization allows CrossQ to scale effectively

We provide empirical evidence for our hypothesis that controlling the weight norm and, thereby, the ELR can stabilize training. We show that through the addition of WN, CrossQ + WN shows stable training and can stably scale with increasing UTD ratios.

Figure 3 shows per environment results of our experiments encompassing all 25 tasks evaluated across 10 seeds each. Based on that, Figure 1 shows aggregated performance over all environments from Figure 3 per task suite, with a separate aggregation for the most complex `dog` and `humanoid` environments.

These results show that CrossQ + WN UTD=5 is competitive to the BRO baseline on both DMC and Myosuite, especially on the more complex `dog` and `humanoid` tasks. Notably, CrossQ + WN UTD=5 uses only half the UTD of BRO and does not require any parameter resets and no additional exploration policy. Further, it uses $\sim 90\%$ fewer network parameters—BRO reports $\sim 5M$, while our proposed CrossQ + WN uses only $\sim 600k$ (these numbers vary slightly per environment, depending on the state and action dimensionalities).

In contrast, vanilla CrossQ UTD=1 exhibits much slower learning on most tasks and, in some environments, fails to learn performant policies. Moreover, the instability of vanilla CrossQ at UTD=5 is particularly notable, as it does not reliably converge across environments.

These findings highlight the necessity of incorporating additional normalization techniques to sustain effective training at higher UTD ratios. This leads us to conclude that CrossQ benefits from the addition of WN, which results in stable training and scales well with higher UTD ratios. The resulting algorithm can match or outperform state-of-the-art

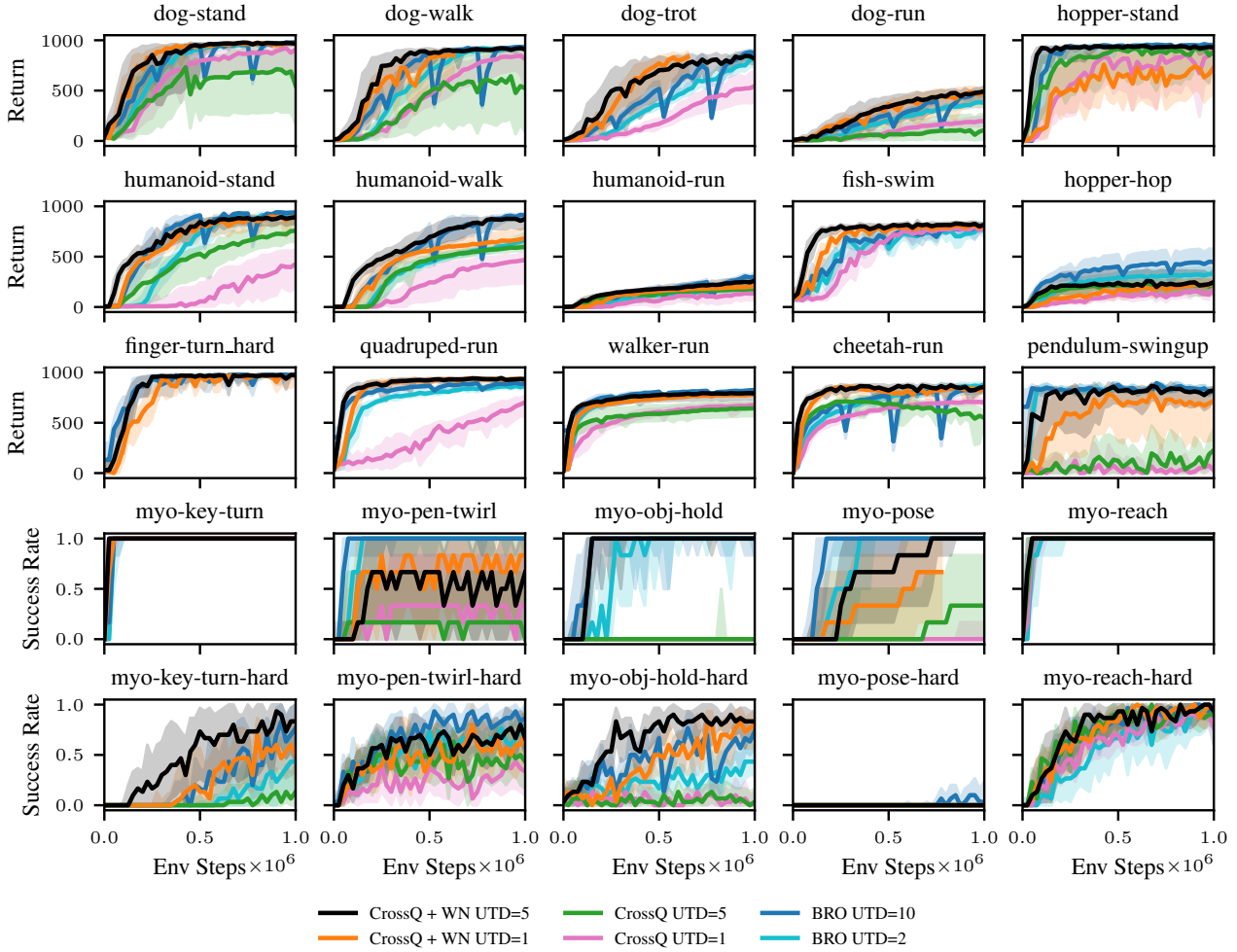


Figure 3. *CrossQ* WN + UTD=5 against baselines. We compare our proposed *CrossQ* + WN UTD=5 against two baselines, BRO (Nauman et al., 2024) and SR-SAC UTD=32. Results are reported on all 15 DMC and 10 Myosuite tasks. We plot the IQM and 95% CIs over 10 random random seeds. Our proposed approach proves competitive to BRO and outperforms the *CrossQ* baseline. We want to note that our approach achieves this performance without requiring any parameter resetting or additional exploration policies.

baselines on the continuous control DMC and Myosuite benchmarks while being much simpler algorithmically.

5.3. Stable scaling of *CrossQ* + WN with UTD ratios

To visualize the stable scaling behavior of *CrossQ* + WN we ablate across three different UTD ratios $\in \{1, 2, 5\}$. Figure 4 shows training curves aggregated over all 15 DMC tasks. As expected, *CrossQ* + WN shows reliable scaling behavior, with the learning curves ordered in increasing order accordance to their respective UTD ratio.

5.4. Hyperparameter ablation studies

We also ablate the different hyperparameters of *CrossQ* + WN UTD=5, by changing each one at a time. Figure 5 shows aggregated results of the final performances of each ablation. We will briefly discuss each ablation individually.

Removing weight normalization. Not performing weight normalization results in the biggest drop in performance across all our ablations. This loss is most drastic on the Myosuite tasks and often results in no meaningful learning. Showing that, as hypothesized, the inclusion of WN into the *CrossQ* framework yields great improvements in terms of sample efficiency and training stability, especially for larger UTD ratios.

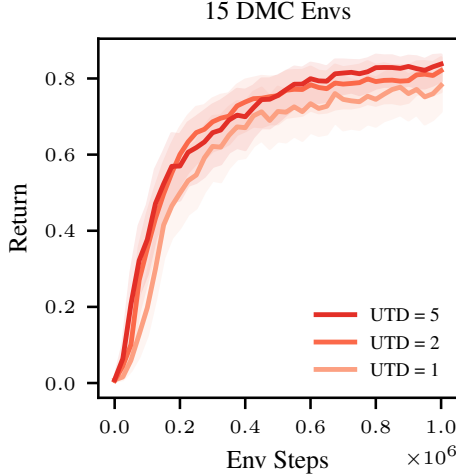


Figure 4. *CrossQ* WN UTD scaling behavior. We plot the IQM return and 95% confidence intervals for different UTD ratios $\in \{1, 2, 5\}$. The results are aggregated over 15 DMC environments and 10 random seeds each according to Agarwal et al. (2021). The sample efficiency scales reliably with increasing UTD ratios.

Update-to-data ratio. As expected, decreasing the UTD ratio decreases the performance of CrossQ + WN, as demonstrated in the previous section. Aggregated over all DMC environments, this effect is the smallest, since this aggregation includes easier environments as well. Looking at the harder dog and humanoid tasks, as well as Myosuite, the effect is more pronounced. However, lower UTDs are already reasonably competitive in overall performance.

Target networks. Ablating the target networks shows that on Myosuite, there is no significant difference between using a target network and or no target network. Results on DMC differ significantly. There, removing target networks leads to a significant drop in performance, nearly as large as removing weight normalization. This finding is interesting, as it suggests that CrossQ + WN without target networks is not inherently unstable. But there are situations where the inclusion of target networks is required. Further investigating the role and necessity of target networks in RL is an interesting direction for future research.

6. Related work

RL has demonstrated remarkable success across various domains, yet sample efficiency remains a significant challenge, especially in real-world applications where data collection is expensive or impractical. Various approaches have been explored to address this issue, including model-based RL, UTD ratio scaling, and architectural modifications.

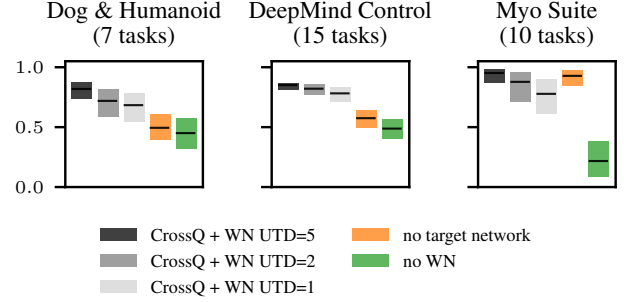


Figure 5. *Hyperparameter ablations.* We ablate CrossQ + WN with respect to the WN, target networks and different UTD.

Model-based RL methods enhance sample efficiency by constructing predictive models of the environment to reduce reliance on real data collection (Sutton, 1990; Janner et al., 2019; Feinberg et al., 2018; Heess et al., 2015). However, such methods introduce additional complexity, computational overhead, and potential biases due to model inaccuracies.

Update-to-data ratio scaling. Model-free RL methods, including those utilizing higher UTD ratios, focus on increasing the number of gradient updates per collected sample to maximize learning from available data. High UTD training introduces several challenges, such as overfitting to early training data, a phenomenon known as primacy bias (Nikishin et al., 2022). This can be counteracted by periodically resetting the network parameters (Nikishin et al., 2022; D’Oro et al., 2022). However, network resets introduce abrupt performance drops. Alternative approaches use techniques such as Q-function ensembles (Chen et al., 2021; Hiraoka et al., 2021) and architectural changes (Nauman et al., 2024).

Normalization techniques in RL. Normalization techniques have long been recognized for their impact on neural network training. LN (Ba et al., 2016) and other architectural modifications have been used to stabilize learning in RL (Hiraoka et al., 2021; Nauman et al., 2024). Yet BN has only recently been successfully applied in this context (Bhatt et al., 2024), challenging previous findings, where BN in critics caused training to fail (Hiraoka et al., 2021). WN has been shown to keep ELRs stable and prevent loss of plasticity (Lyle et al., 2024), when combined with LN, making it a promising candidate for integration into existing RL frameworks.

7. Limitations & future work

In this work, we only consider continuous state-action benchmarking tasks. While our proposed CrossQ + WN performs competitively on these tasks, its performance on discrete state-action spaces or vision-based tasks remains unexplored. We plan to investigate this in future work.

8. Conclusion

In this work, we have addressed the instability and scalability limitations of CrossQ in RL by integrating WN. Our empirical results demonstrate that WN effectively stabilizes training and allows CrossQ to scale reliably with higher UTD ratios. The proposed CrossQ + WN approach achieves competitive or superior performance compared to state-of-the-art baselines across a diverse set of 25 complex continuous control tasks from the DMC and Myosuite benchmarks. These tasks include complex and high-dimensional humanoid and dog environments. This extension preserves simplicity while enhancing robustness and scalability by eliminating the need for drastic interventions such as network resets.

Acknowledgments

We wish to thank Joe Watson for helpful discussions and advice during the project. We would also like to thank Tim Schneider, Cristiana de Farias, João Carvalho and Theo Gruner for proofreading and constructive criticism on the manuscript. This research was funded by the research cluster “Third Wave of AI”, funded by the excellence program of the Hessian Ministry of Higher Education, Science, Research and the Arts, hessian.AI.

References

- Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A. C., and Bellemare, M. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems*, 2021.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Bhatt, A., Palenicek, D., Belousov, B., Argus, M., Amiranashvili, A., Brox, T., and Peters, J. CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *International conference on learning representations*, 2024.
- Caggiano, V., Wang, H., Durandau, G., Sartori, M., and Kumar, V. Myosuite—a contact-rich simulation suite for musculoskeletal motor control. *arXiv preprint arXiv:2205.13600*, 2022.
- Chen, X., Wang, C., Zhou, Z., and Ross, K. Randomized ensembled double Q-learning: Learning fast without a model. In *International conference on learning representations*, 2021.
- D’Oro, P., Schwarzer, M., Nikishin, E., Bacon, P.-L., Bellemare, M. G., and Courville, A. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *International conference on learning representations*, 2022.
- Elsayed, M., Lan, Q., Lyle, C., and Mahmood, A. R. Weight clipping for deep continual and reinforcement learning. In *Reinforcement Learning Conference*, 2024.
- Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., and Levine, S. Model-based value estimation for efficient model-free reinforcement learning. In *International Conference on Machine Learning*, 2018.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., and Levine, S. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.
- Heess, N., Wayne, G., Silver, D., Lillicrap, T., Erez, T., and Tassa, Y. Learning continuous control policies by stochastic value gradients. In *Advances in neural information processing systems*, 2015.
- Hiraoka, T., Imagawa, T., Hashimoto, T., Onishi, T., and Tsuruoka, Y. Dropout q-functions for doubly efficient reinforcement learning. In *International conference on learning representations*, 2021.
- Huang, L., Qin, J., Zhou, Y., Zhu, F., Liu, L., and Shao, L. Normalization techniques in training dnn: Methodology, analysis and application. *IEEE transactions on pattern analysis and machine intelligence*, 2023.
- Hussing, M., Voelcker, C., Gilitschenski, I., Farahmand, A.-m., and Eaton, E. Dissecting deep rl with high update ratios: Combatting value overestimation and divergence. *arXiv preprint arXiv:2403.05996*, 2024.
- Ioffe, S. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- Janner, M., Fu, J., Zhang, M., and Levine, S. When to trust your model: Model-based policy optimization. In *Advances in neural information processing systems*, 2019.
- Kim, S., Asadi, K., Littman, M., and Konidaris, G. Deepmellow: Removing the need for a target network in deep q-learning. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*,

A. Proof Scale Invariance

Proof of Theorem 4.1.

$$\begin{aligned}
 f(\mathbf{X}; c\mathbf{w}, cb, \gamma, \beta) &= \frac{g(c\mathbf{X}\mathbf{w} + cb) \cup \mu(g(c\mathbf{X}\mathbf{w} + cb))}{\sigma(g(c\mathbf{X}\mathbf{w} + cb))} \gamma + \beta \\
 &= \frac{cg(\mathbf{X}\mathbf{w} + b) \cup c\mu(g(\mathbf{X}\mathbf{w} + b))}{|c|\sigma(g(\mathbf{X}\mathbf{w} + b))} \gamma + \beta \\
 &= \frac{g(\mathbf{X}\mathbf{w} + b) \cup \mu(g(\mathbf{X}\mathbf{w} + b))}{\sigma(g(\mathbf{X}\mathbf{w} + b))} \gamma + \beta = f(\mathbf{X}; \mathbf{w}, b, \gamma, \beta)
 \end{aligned}$$

B. Proof Inverse Proportional Gradients

To show that the gradients scale inversely proportional to the parameter norm, we can first write

$$\begin{aligned}
 f(\mathbf{X}; c\mathbf{w}, cb, \gamma, \beta) &= \frac{g(c\mathbf{X}\mathbf{w} + cb) \cup \mu(g(c\mathbf{X}\mathbf{w} + cb))}{\sigma(g(c\mathbf{X}\mathbf{w} + cb))} \gamma + \beta \\
 &= \frac{g(c\mathbf{X}\mathbf{w} + cb)}{\sigma(g(c\mathbf{X}\mathbf{w} + cb))} \gamma \cup \frac{\mu(g(c\mathbf{X}\mathbf{w} + cb))}{\sigma(g(c\mathbf{X}\mathbf{w} + cb))} \gamma + \beta.
 \end{aligned}$$

As the gradient of the weights is not backpropagated through the mean and standard deviation, we have

$$\nabla_{\mathbf{w}} f(\mathbf{X}; c\mathbf{w}, cb, \gamma, \beta) = \frac{g'(c\mathbf{X}\mathbf{w} + cb)X}{|c|\sigma(g(\mathbf{X}\mathbf{w} + b))} \gamma.$$

The gradient of the bias can be computed analogously

$$\nabla_b f(\mathbf{X}; c\mathbf{w}, cb, \gamma, \beta) = \frac{g'(c\mathbf{X}\mathbf{w} + cb)}{|c|\sigma(g(\mathbf{X}\mathbf{w} + b))} \gamma.$$

C. Hyperparameters

Table 1 gives an overview of the hyperparameters that were used for each algorithm that was considered in this work.

Table 1. Hyperparameters

Hyperparameter	CrossQ	CrossQ + WN	SAC	SR-SAC	BRO
Critic learning rate	0.0001	0.0001	0.0003	0.0003	0.0003
Critic hidden dim	512	512	256	256	512
Actor learning rate	0.0001	0.0001	0.0003	0.0003	0.0003
Actor hidden dim	256	256	256	256	256
Initial temperature	1.0	1.0	1.0	1.0	1.0
Temperature learning rate	0.0001	0.0001	0.0003	0.0003	0.0003
Target entropy	$ A $	$ A $	$ A $	$ A $	$ A $
Target network momentum	0.005	0.005	0.005	0.005	0.005
UTD	1,5	1,5	1	32	10
Number of critics	2	2	2	2	1
Action repeat	2	2	2	2	1
Discount	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.99 (Myo)
Optimizer	Adam	AdamW	Adam	Adam	AdamW
Optimizer momentum (β_1, β_2)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)
Policy delay	3	3	1	1	1
Warmup transitions	5000	5000	10000	10000	10000
AdamW weight decay critic	0.0	0.01	0.0	0.0	0.0001
AdamW weight decay actor	0.0	0.01	0.0	0.0	0.0001
AdamW weight decay temperature	0.0	0.0	0.0	0.0	0.0
Batch Normalization momentum	0.99	0.99	N/A	N/A	N/A
Reset Interval of networks	N/A	N/A	N/A	every 80k steps	at 15k, 50k, 250k, 500k and 750k steps
Batch Size	256	256	256	256	128

D. Individual training curves for ablation results

Here, we provide detailed individual training curves for each ablation experiment conducted in our study. Figure 6 shows experiments with no WN, no target network, CrossQ with WN and UTD=1, and CrossQ with WN and UTD=5.

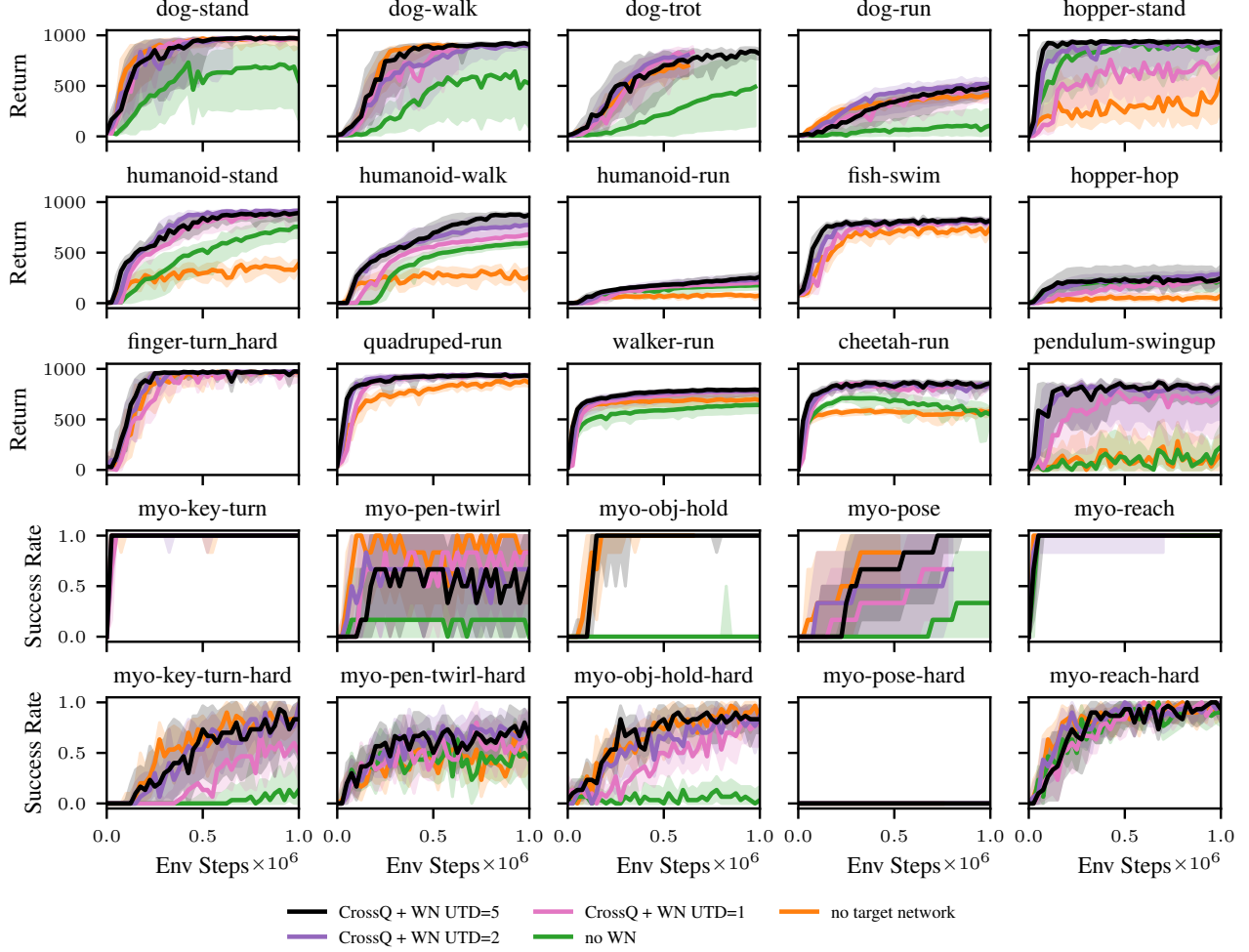


Figure 6. Individual training curves for ablations

On the importance of initialization and momentum in deep learning

Ilya Sutskever¹
James Martens
George Dahl
Geoffrey Hinton

ILYASU@GOOGLE.COM
JMARTENS@CS.TORONTO.EDU
GDAHL@CS.TORONTO.EDU
HINTON@CS.TORONTO.EDU

Abstract

Deep and recurrent neural networks (DNNs and RNNs respectively) are powerful models that were considered to be almost impossible to train using stochastic gradient descent with momentum. In this paper, we show that when stochastic gradient descent with momentum uses a well-designed random initialization and a particular type of slowly increasing schedule for the momentum parameter, it can train both DNNs and RNNs (on datasets with long-term dependencies) to levels of performance that were previously achievable only with Hessian-Free optimization. We find that both the initialization and the momentum are crucial since poorly initialized networks cannot be trained with momentum and well-initialized networks perform markedly worse when the momentum is absent or poorly tuned.

Our success training these models suggests that previous attempts to train deep and recurrent neural networks from random initializations have likely failed due to poor initialization schemes. Furthermore, carefully tuned momentum methods suffice for dealing with the curvature issues in deep and recurrent network training objectives without the need for sophisticated second-order methods.

1. Introduction

Deep and recurrent neural networks (DNNs and RNNs, respectively) are powerful models that achieve high performance on difficult pattern recognition problems in vision, and speech (Krizhevsky et al., 2012; Hinton et al., 2012; Dahl et al., 2012; Graves, 2012).

Although their representational power is appealing, the difficulty of training DNNs has prevented their

widespread use until fairly recently. DNNs became the subject of renewed attention following the work of Hinton et al. (2006) who introduced the idea of greedy layerwise pre-training. This approach has since branched into a family of methods (Bengio et al., 2007), all of which train the layers of the DNN in a sequence using an auxiliary objective and then “fine-tune” the entire network with standard optimization methods such as stochastic gradient descent (SGD). More recently, Martens (2010) attracted considerable attention by showing that a type of truncated-Newton method called Hessian-free Optimization (HF) is capable of training DNNs from certain random initializations without the use of pre-training, and can achieve lower errors for the various auto-encoding tasks considered by Hinton & Salakhutdinov (2006).

Recurrent neural networks (RNNs), the temporal analogue of DNNs, are highly expressive sequence models that can model complex sequence relationships. They can be viewed as very deep neural networks that have a “layer” for each time-step with parameter sharing across the layers and, for this reason, they are considered to be even harder to train than DNNs. Recently, Martens & Sutskever (2011) showed that the HF method of Martens (2010) could effectively train RNNs on artificial problems that exhibit very long-range dependencies (Hochreiter & Schmidhuber, 1997). Without resorting to special types of memory units, these problems were considered to be impossibly difficult for first-order optimization methods due to the well known vanishing gradient problem (Bengio et al., 1994). Sutskever et al. (2011) and later Mikolov et al. (2012) then applied HF to train RNNs to perform character-level language modeling and achieved excellent results.

Recently, several results have appeared to challenge the commonly held belief that simpler first-order methods are incapable of learning deep models from random initializations. The work of Glorot & Bengio (2010), Mohamed et al. (2012), and Krizhevsky et al. (2012) reported little difficulty training neural networks with depths up to 8 from certain well-chosen

Proceedings of the 30th International Conference on Machine Learning, Atlanta, Georgia, USA, 2013. JMLR: W&CP volume 28. Copyright 2013 by the author(s).

¹Work was done while the author was at the University of Toronto.

random initializations. Notably, Chapelle & Erhan (2011) used the random initialization of Glorot & Bengio (2010) and SGD to train the 11-layer autoencoder of Hinton & Salakhutdinov (2006), and were able to surpass the results reported by Hinton & Salakhutdinov (2006). While these results still fall short of those reported in Martens (2010) for the same tasks, they indicate that learning deep networks is not nearly as hard as was previously believed.

The first contribution of this paper is a much more thorough investigation of the difficulty of training deep and temporal networks than has been previously done. In particular, we study the effectiveness of SGD when combined with well-chosen initialization schemes and various forms of momentum-based acceleration. We show that while a definite performance gap seems to exist between plain SGD and HF on certain deep and temporal learning problems, this gap can be eliminated or nearly eliminated (depending on the problem) by careful use of classical momentum methods or Nesterov’s accelerated gradient. In particular, we show how certain carefully designed schedules for the constant of momentum μ , which are inspired by various theoretical convergence-rate theorems (Nesterov, 1983; 2003), produce results that even surpass those reported by Martens (2010) on certain deep-autencoder training tasks. For the long-term dependency RNN tasks examined in Martens & Sutskever (2011), which first appeared in Hochreiter & Schmidhuber (1997), we obtain results that fall just short of those reported in that work, where a considerably more complex approach was used.

Our results are particularly surprising given that momentum and its use within neural network optimization has been studied extensively before, such as in the work of Orr (1996), and it was never found to have such an important role in deep learning. One explanation is that previous theoretical analyses and practical benchmarking focused on local convergence in the stochastic setting, which is more of an estimation problem than an optimization one (Bottou & LeCun, 2004). In deep learning problems this final phase of learning is not nearly as long or important as the initial “transient phase” (Darken & Moody, 1993), where a better argument can be made for the beneficial effects of momentum.

In addition to the inappropriate focus on purely local convergence rates, we believe that the use of poorly designed standard random initializations, such as those in Hinton & Salakhutdinov (2006), and suboptimal meta-parameter schedules (for the momentum constant in particular) has hampered the discovery of the true effectiveness of first-order momentum methods in deep learning. We carefully avoid both of these pitfalls in our experiments and provide a simple to understand and easy to use framework for deep learning that

is surprisingly effective and can be naturally combined with techniques such as those in Raiko et al. (2011).

We will also discuss the links between classical momentum and Nesterov’s accelerated gradient method (which has been the subject of much recent study in convex optimization theory), arguing that the latter can be viewed as a simple modification of the former which increases stability, and can sometimes provide a distinct improvement in performance we demonstrated in our experiments. We perform a theoretical analysis which makes clear the precise difference in local behavior of these two algorithms. Additionally, we show how HF employs what can be viewed as a type of “momentum” through its use of special initializations to conjugate gradient that are computed from the update at the previous time-step. We use this property to develop a more momentum-like version of HF which combines some of the advantages of both methods to further improve on the results of Martens (2010).

2. Momentum and Nesterov’s Accelerated Gradient

The momentum method (Polyak, 1964), which we refer to as classical momentum (CM), is a technique for accelerating gradient descent that accumulates a velocity vector in directions of persistent reduction in the objective across iterations. Given an objective function $f(\theta)$ to be minimized, classical momentum is given by:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (2)$$

where $\varepsilon > 0$ is the learning rate, $\mu \in [0, 1]$ is the momentum coefficient, and $\nabla f(\theta_t)$ is the gradient at θ_t .

Since directions d of low-curvature have, by definition, slower local change in their rate of reduction (i.e., $d^\top \nabla f$), they will tend to persist across iterations and be amplified by CM. Second-order methods also amplify steps in low-curvature directions, but instead of accumulating changes they reweight the update along each eigen-direction of the curvature matrix by the inverse of the associated curvature. And just as second-order methods enjoy improved local convergence rates, Polyak (1964) showed that CM can considerably accelerate convergence to a local minimum, requiring $\sqrt{R}$ -times fewer iterations than steepest descent to reach the same level of accuracy, where R is the condition number of the curvature at the minimum and μ is set to $(\sqrt{R} - 1)/(\sqrt{R} + 1)$.

Nesterov’s Accelerated Gradient (abbrv. NAG; Nesterov, 1983) has been the subject of much recent attention by the convex optimization community (e.g., Cotter et al., 2011; Lan, 2010). Like momentum, NAG is a first-order optimization method with better convergence rate guarantee than gradient descent in

certain situations. In particular, for general smooth (non-strongly) convex functions and a deterministic gradient, NAG achieves a global convergence rate of $O(1/T^2)$ (versus the $O(1/T)$ of gradient descent), with constant proportional to the Lipschitz coefficient of the derivative and the squared Euclidean distance to the solution. While NAG is not typically thought of as a type of momentum, it indeed turns out to be closely related to classical momentum, differing only in the precise update of the velocity vector v , the significance of which we will discuss in the next sub-section. Specifically, as shown in the appendix, the NAG update may be rewritten as:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t + \mu v_t) \quad (3)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (4)$$

While the classical convergence theories for both methods rely on noiseless gradient estimates (i.e., not stochastic), with some care in practice they are both applicable to the stochastic setting. However, the theory predicts that any advantages in terms of asymptotic local rate of convergence will be lost (Orr, 1996; Wiegerinck et al., 1999), a result also confirmed in experiments (LeCun et al., 1998). For these reasons, interest in momentum methods diminished after they had received substantial attention in the 90's. And because of this apparent incompatibility with stochastic optimization, some authors even discourage using momentum or downplay its potential advantages (LeCun et al., 1998).

However, while local convergence is all that matters in terms of asymptotic convergence rates (and on certain very simple/shallow neural network optimization problems it may even dominate the total learning time), in practice, the "transient phase" of convergence (Darken & Moody, 1993), which occurs before fine local convergence sets in, seems to matter a lot more for optimizing deep neural networks. In this transient phase of learning, directions of reduction in the objective tend to persist across many successive gradient estimates and are not completely swamped by noise.

Although the transient phase of learning is most noticeable in training deep learning models, it is still noticeable in convex objectives. The convergence rate of stochastic gradient descent on smooth convex functions is given by $O(L/T + \sigma/\sqrt{T})$, where σ is the variance in the gradient estimate and L is the Lipschitz coefficient of ∇f . In contrast, the convergence rate of an accelerated gradient method of Lan (2010) (which is related to but different from NAG, in that it combines Nesterov style momentum with dual averaging) is $O(L/T^2 + \sigma/\sqrt{T})$. Thus, for convex objectives, momentum-based methods will outperform SGD in the early or transient stages of the optimization where L/T is the dominant term. However, the two methods will be equally effective during the final stages

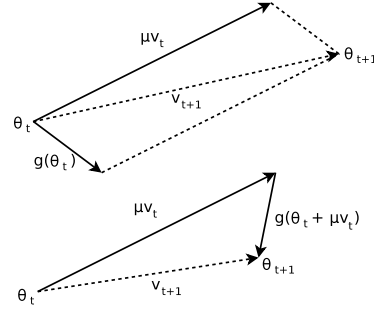


Figure 1. **(Top)** Classical Momentum **(Bottom)** Nesterov Accelerated Gradient

of the optimization where $\sigma/\sqrt{T}$ is the dominant term (i.e., when the optimization problem resembles an estimation one).

2.1. The Relationship between CM and NAG

From Eqs. 1-4 we see that both CM and NAG compute the new velocity by applying a gradient-based correction to the previous velocity vector (which is decayed), and then add the velocity to θ_t . But while CM computes the gradient update from the current position θ_t , NAG first performs a partial update to θ_t , computing $\theta_t + \mu v_t$, which is similar to θ_{t+1} , but missing the as yet unknown correction. This benign-looking difference seems to allow NAG to change v in a quicker and more responsive way, letting it behave more stably than CM in many situations, especially for higher values of μ .

Indeed, consider the situation where the addition of μv_t results in an immediate undesirable increase in the objective f . The gradient correction to the velocity v_t is computed at position $\theta_t + \mu v_t$ and if μv_t is indeed a poor update, then $\nabla f(\theta_t + \mu v_t)$ will point back towards θ_t more strongly than $\nabla f(\theta_t)$ does, thus providing a larger and more timely correction to v_t than CM. See fig. 1 for a diagram which illustrates this phenomenon geometrically. While each iteration of NAG may only be slightly more effective than CM at correcting a large and inappropriate velocity, this difference in effectiveness may compound as the algorithms iterate. To demonstrate this compounding, we applied both NAG and CM to a two-dimensional oblong quadratic objective, both with the same momentum and learning rate constants (see fig. 2 in the appendix). While the optimization path taken by CM exhibits large oscillations along the high-curvature vertical direction, NAG is able to avoid these oscillations almost entirely, confirming the intuition that it is much more effective than CM at decelerating over the course of multiple iterations, thus making NAG more tolerant of large values of μ compared to CM.

In order to make these intuitions more rigorous and

help quantify precisely the way in which CM and NAG differ, we analyzed the behavior of each method when applied to a positive definite quadratic objective $q(x) = x^\top Ax/2 + b^\top x$. We can think of CM and NAG as operating independently over the different eigendirections of A . NAG operates along any one of these directions equivalently to CM, except with an effective value of μ that is given by $\mu(1 - \lambda\varepsilon)$, where λ is the associated eigenvalue/curvature.

The first step of this argument is to reparameterize $q(x)$ in terms of the coefficients of x under the basis of eigenvectors of A . Note that since $A = U^\top DU$ for a diagonal D and orthonormal U (as A is symmetric), we can reparameterize $q(x)$ by the matrix transform U and optimize $y = Ux$ using the objective $p(y) \equiv q(x) = q(U^\top y) = y^\top U (U^\top DU) U^\top y/2 + b^\top U^\top y = y^\top Dy/2 + c^\top y$, where $c = Ub$. We can further rewrite p as $p(y) = \sum_{i=1}^n [p]_i([y]_i)$, where $[p]_i(t) = \lambda_i t^2/2 + [c]_i t$ and $\lambda_i > 0$ are the diagonal entries of D (and thus the eigenvalues of A) and correspond to the curvature along the associated eigenvector directions. As shown in the appendix (Proposition 6.1), both CM and NAG, being first-order methods, are “invariant” to these kinds of reparameterizations by orthonormal transformations such as U . Thus when analyzing the behavior of either algorithm applied to $q(x)$, we can instead apply them to $p(y)$, and transform the resulting sequence of iterates back to the default parameterization (via multiplication by $U^{-1} = U^\top$).

Theorem 2.1. *Let $p(y) = \sum_{i=1}^n [p]_i([y]_i)$ such that $[p]_i(t) = \lambda_i t^2/2 + c_i t$. Let ε be arbitrary and fixed. Denote by $CM_x(\mu, p, y, v)$ and $CM_v(\mu, p, y, v)$ the parameter vector and the velocity vector respectively, obtained by applying one step of CM (i.e., Eq. 1 and then Eq. 2) to the function p at point y , with velocity v , momentum coefficient μ , and learning rate ε . Define NAG_x and NAG_v analogously. Then the following holds for $z \in \{x, v\}$:*

$$CM_z(\mu, p, y, v) = \begin{bmatrix} CM_z(\mu, [p]_1, [y]_1, [v]_1) \\ \vdots \\ CM_z(\mu, [p]_n, [y]_n, [v]_n) \end{bmatrix}$$

$$NAG_z(\mu, p, y, v) = \begin{bmatrix} CM_z(\mu(1 - \lambda_1\varepsilon), [p]_1, [y]_1, [v]_1) \\ \vdots \\ CM_z(\mu(1 - \lambda_n\varepsilon), [p]_n, [y]_n, [v]_n) \end{bmatrix}$$

Proof. See the appendix. $\square$

The theorem has several implications. First, CM and NAG become equivalent when ε is small (when $\varepsilon\lambda \ll 1$ for every eigenvalue λ of A), so NAG and CM are distinct only when ε is reasonably large. When ε is relatively large, NAG uses smaller effective momentum for the high-curvature eigen-directions, which prevents

oscillations (or divergence) and thus allows the use of a larger μ than is possible with CM for a given ε .

3. Deep Autoencoders

The aim of our experiments is three-fold. First, to investigate the attainable performance of stochastic momentum methods on deep autoencoders starting from well-designed random initializations; second, to explore the importance and effect of the schedule for the momentum parameter μ assuming an optimal fixed choice of the learning rate ε ; and third, to compare the performance of NAG versus CM.

For our experiments with feed-forward nets, we focused on training the three deep autoencoder problems described in Hinton & Salakhutdinov (2006) (see sec. A.2 for details). The task of the neural network autoencoder is to reconstruct its own input subject to the constraint that one of its hidden layers is of low-dimension. This “bottleneck” layer acts as a low-dimensional code for the original input, similar to other dimensionality reduction techniques like Principle Component Analysis (PCA). These autoencoders are some of the deepest neural networks with published results, ranging between 7 and 11 layers, and have become a standard benchmarking problem (e.g., Martens, 2010; Glorot & Bengio, 2010; Chapelle & Erhan, 2011; Raiko et al., 2011). See the appendix for more details.

Because the focus of this study is on optimization, we only report training errors in our experiments. Test error depends strongly on the amount of overfitting in these problems, which in turn depends on the type and amount of regularization used during training. While regularization is an issue of vital importance when designing systems of practical utility, it is outside the scope of our discussion. And while it could be objected that the gains achieved using better optimization methods are only due to more exact fitting of the training set in a manner that does not generalize, this is simply not the case in these problems, where under-trained solutions are known to perform poorly on both the training and test sets (underfitting).

The networks we trained used the standard sigmoid nonlinearity and were initialized using the “sparse initialization” technique (SI) of Martens (2010) that is described in sec. 3.1. Each trial consists of 750,000 parameter updates on minibatches of size 200. No regularization is used. The schedule for μ was given by the following formula:

$$\mu_t = \min(1 - 2^{-1 - \log_2(\lfloor t/250 \rfloor + 1)}, \mu_{\max}) \quad (5)$$

where $\mu_{\max}$ was chosen from $\{0.999, 0.995, 0.99, 0.9, 0\}$. This schedule was motivated by Nesterov (1983) who advocates using what amounts to $\mu_t = 1 - 3/(t + 5)$ after some manipulation

On the importance of initialization and momentum in deep learning

task	$0_{(\text{SGD})}$	0.9N	0.99N	0.995N	0.999N	0.9M	0.99M	0.995M	0.999M	SGD _C	HF [†]	HF*
Curves	0.48	0.16	0.096	0.091	0.074	0.15	0.10	0.10	0.10	0.16	0.058	0.11
Mnist	2.1	1.0	0.73	0.75	0.80	1.0	0.77	0.84	0.90	0.9	0.69	1.40
Faces	36.4	14.2	8.5	7.8	7.7	15.3	8.7	8.3	9.3	NA	7.5	12.0

Table 1. The table reports the squared errors on the problems for each combination of $\mu_{\max}$ and a momentum type (NAG, CM). When $\mu_{\max}$ is 0 the choice of NAG vs CM is of no consequence so the training errors are presented in a single column. For each choice of $\mu_{\max}$, the highest-performing learning rate is used. The column SGD_C lists the results of Chapelle & Erhan (2011) who used 1.7M SGD steps and tanh networks. The column $\text{HF}^\dagger$ lists the results of HF without L2 regularization, as described in sec. 5; and the column HF^* lists the results of Martens (2010).

problem	before	after
Curves	0.096	0.074
Mnist	1.20	0.73
Faces	10.83	7.7

Table 2. The effect of low-momentum finetuning for NAG. The table shows the training squared errors before and after the momentum coefficient is reduced. During the primary (“transient”) phase of learning we used the optimal momentum and learning rates.

(see appendix), and by Nesterov (2003) who advocates a constant μ_t that depends on (essentially) the condition number. The constant μ_t achieves exponential convergence on strongly convex functions, while the $1 - 3/(t+5)$ schedule is appropriate when the function is not strongly convex. The schedule of Eq. 5 blends these proposals. For each choice of $\mu_{\max}$, we report the learning rate that achieved the best training error. Given the schedule for μ , the learning rate ε was chosen from $\{0.05, 0.01, 0.005, 0.001, 0.0005, 0.0001\}$ in order to achieve the lowest final error training error after our fixed number of updates.

Table 1 summarizes the results of these experiments. It shows that NAG achieves the lowest published results on this set of problems, including those of Martens (2010). It also shows that larger values of $\mu_{\max}$ tend to achieve better performance and that NAG usually outperforms CM, especially when $\mu_{\max}$ is 0.995 and 0.999. Most surprising and importantly, the results demonstrate that NAG can achieve results that are comparable with some of the best HF results for training deep autoencoders. Note that the previously published results on HF used L2 regularization, so they cannot be directly compared. However, the table also includes experiments we performed with an improved version of HF (see sec. 2.1) where weight decay was removed towards the end of training.

We found it beneficial to reduce μ to 0.9 (unless μ is 0, in which case it is unchanged) during the final 1000 parameter updates of the optimization without reducing the learning rate, as shown in Table 2. It appears that reducing the momentum coefficient allows for finer convergence to take place whereas otherwise the overly aggressive nature of CM or NAG

would prevent this. This phase shift between optimization that favors fast accelerated motion along the error surface (the “transient phase”) followed by more careful optimization-as-estimation phase seems consistent with the picture presented by Darken & Moody (1993). However, while asymptotically it is the second phase which must eventually dominate computation time, in practice it seems that for deeper networks in particular, the first phase dominates overall computation time as long as the second phase is cut off before the remaining potential gains become either insignificant or entirely dominated by overfitting (or both).

It may be tempting then to use lower values of μ from the outset, or to reduce it immediately when progress in reducing the error appears to slow down. However, in our experiments we found that doing this was detrimental in terms of the final errors we could achieve, and that despite appearing to not make much progress, or even becoming significantly non-monotonic, the optimizers were doing something apparently useful over these extended periods of time at higher values of μ .

A speculative explanation as to why we see this behavior is as follows. While a large value of μ allows the momentum methods to make useful progress along slowly-changing directions of low-curvature, this may not immediately result in a significant reduction in error, due to the failure of these methods to converge in the more turbulent high-curvature directions (which is especially hard when μ is large). Nevertheless, this progress in low-curvature directions takes the optimizers to new regions of the parameter space that are characterized by closer proximity to the optimum (in the case of a convex objective), or just higher-quality local minimia (in the case of non-convex optimization). Thus, while it is important to adopt a more careful scheme that allows fine convergence to take place along the high-curvature directions, this must be done with care. Reducing μ and moving to this fine convergence regime too early may make it difficult for the optimization to make significant progress along the low-curvature directions, since without the benefit of momentum-based acceleration, first-order methods are notoriously bad at this (which is what motivated the use of second-order methods like HF for deep learning).

SI scale multiplier	0.25	0.5	1	2	4
error	16	16	0.074	0.083	0.35

Table 3. The table reports the training squared error that is attained by changing the scale of the initialization.

3.1. Random Initializations

The results in the previous section were obtained with standard logistic sigmoid neural networks that were initialized with the sparse initialization technique (SI) described in Martens (2010). In this scheme, each random unit is connected to 15 randomly chosen units in the previous layer, whose weights are drawn from a unit Gaussian, and the biases are set to zero. The intuitive justification is that the total amount of input to each unit will not depend on the size of the previous layer and hence they will not as easily saturate. Meanwhile, because the inputs to each unit are not all randomly weighted blends of the outputs of many 100s or 1000s of units in the previous layer, they will tend to be qualitatively more “diverse” in their response to inputs. When using tanh units, we transform the weights to simulate sigmoid units by setting the biases to 0.5 and rescaling the weights by 0.25.

We investigated the performance of the optimization as a function of the scale constant used in SI (which defaults to 1 for sigmoid units). We found that SI works reasonably well if it is rescaled by a factor of 2, but leads to noticeable (but not severe) slow down when scaled by a factor of 3. When we used the factor 1/2 or 5 we did not achieve sensible results.

4. Recurrent Neural Networks

Echo-State Networks (ESNs) is a family of RNNs with an unusually simple training method: their hidden-to-output connections are learned from data, but their recurrent connections are fixed to a random draw from a specific distribution and are not learned. Despite their simplicity, ESNs with many hidden units (or with units with explicit temporal integration, like the LSTM) have achieved high performance on tasks with long range dependencies (?). In this section, we investigate the effectiveness of momentum-based methods with ESN-inspired initialization at training RNNs with conventional size and standard (i.e., non-integrating) neurons. We find that momentum-accelerated SGD can successfully train such RNNs on various artificial datasets exhibiting considerable long-range temporal dependencies. This is unexpected because RNNs were believed to be almost impossible to successfully train on such datasets with first-order methods, due to various difficulties such as vanishing/exploding gradients (Bengio et al., 1994). While we found that the use of momentum significantly improved performance and robustness, we obtained nontrivial results even with standard SGD, provided that the learning rate was set low enough.

connection type	sparsity	scale
in-to-hid (add,mul)	dense	$0.001 \cdot N(0, 1)$
in-to-hid (mem)	dense	$0.1 \cdot N(0, 1)$
hid-to-hid	15 fan-in	spectral radius of 1.1
hid-to-out	dense	$0.1 \cdot N(0, 1)$
hid-bias	dense	0
out-bias	dense	average of outputs

Table 4. The RNN initialization used in the experiments. The scale of the vis-hid connections is problem dependent.

Each task involved optimizing the parameters of a randomly initialized RNN with 100 standard tanh hidden units (the same model used by Martens & Sutskever (2011)). The tasks were designed by Hochreiter & Schmidhuber (1997), and are referred to as training “problems”. See sec. A.3 of the appendix for details.

4.1. ESN-based Initialization

As argued by Jaeger & Haas (2004), the spectral radius of the hidden-to-hidden matrix has a profound effect on the dynamics of the RNN’s hidden state (with a tanh nonlinearity). When it is smaller than 1, the dynamics will have a tendency to quickly “forget” whatever input signal they may have been exposed to. When it is much larger than 1, the dynamics become oscillatory and chaotic, allowing it to generate responses that are varied for different input histories. While this allows information to be retained over many time steps, it can also lead to severe exploding gradients that make gradient-based learning much more difficult. However, when the spectral radius is only slightly greater than 1, the dynamics remain oscillatory and chaotic while the gradient are no longer exploding (and if they do explode, then only “slightly so”), so learning may be possible with a spectral radius of this order. This suggests that a spectral radius of around 1.1 may be effective.

To achieve robust results, we also found it is essential to carefully set the initial scale of the input-to-hidden connections. When training RNNs to solve those tasks that possess many random and irrelevant distractor inputs, we found that having the scale of these connections set too high at the start led to relevant information in the hidden state being too easily “overwritten” by the many irrelevant signals, which ultimately led the optimizer to converge towards an extremely poor local minimum where useful information was never relayed over long distances. Conversely, we found that if this scale was set too low, it led to significantly slower learning. Having experimented with multiple scales we found that a Gaussian draw with a standard deviation of 0.001 achieved a good balance between these concerns. However, unlike the value of 1.1 for the spectral radius of the dynamics matrix, which worked well on all tasks, we found that good choices for initial scale of the input-to-hidden weights depended a lot on the particular characteristics of the particular task (such

as its dimensionality or the input variance). Indeed, for tasks that do not have many irrelevant inputs, a larger scale of the input-to-hidden weights (namely, 0.1) worked better, because the aforementioned disadvantage of large input-to-hidden weights does not apply. See table 4 for a summary of the initializations used in the experiments. Finally, we found centering (mean subtraction) of both the inputs and the outputs to be important to reliably solve all of the training problems. See the appendix for more details.

4.2. Experimental Results

We conducted experiments to determine the efficacy of our initializations, the effect of momentum, and to compare NAG with CM. Every learning trial used the aforementioned initialization, 50,000 parameter updates and on minibatches of 100 sequences, and the following schedule for the momentum coefficient μ : $\mu = 0.9$ for the first 1000 parameter, after which $\mu = \mu_0$, where μ_0 can take the following values $\{0, 0.9, 0.98, 0.995\}$. For each μ_0 , we use the empirically best learning rate chosen from $\{10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}\}$.

The results are presented in Table 5, which are the average loss over 4 different random seeds. Instead of reporting the loss being minimized (which is the squared error or cross entropy), we use a more interpretable zero-one loss, as is standard practice with these problems. For the bit memorization, we report the fraction of timesteps that are computed incorrectly. And for the addition and the multiplication problems, we report the fraction of cases where the RNN the error in the final output prediction exceeded 0.04.

Our results show that despite the considerable long-range dependencies present in training data for these problems, RNNs can be successfully and robustly trained to solve them, through the use of the initialization discussed in sec. 4.1, momentum of the NAG type, a large μ_0 , and a particularly small learning rate (as compared with feedforward networks). Our results also suggest that with larger values of μ_0 achieve better results with NAG but not with CM, possibly due to NAG’s tolerance of larger μ_0 ’s (as discussed in sec. 2).

Although we were able to achieve surprisingly good training performance on these problems using a sufficiently strong momentum, the results of Martens & Sutskever (2011) appear to be moderately better and more robust. They achieved lower error rates and their initialization was chosen with less care, although the initializations are in many ways similar to ours. Notably, Martens & Sutskever (2011) were able to solve these problems without centering, while we had to use centering to solve the multiplication problem (the other problems are already centered). This suggests that the initialization proposed here, together with the method of Martens & Sutskever (2011), could achieve

even better performance. But the main achievement of these results is a demonstration of the ability of momentum methods to cope with long-range temporal dependency training tasks to a level which seems sufficient for most practical purposes. Moreover, our approach seems to be more tolerant of smaller minibatches, and is considerably simpler than the particular version of HF proposed in Martens & Sutskever (2011), which used a specialized update damping technique whose benefits seemed mostly limited to training RNNs to solve these kinds of extreme temporal dependency problems.

5. Momentum and HF

Truncated Newton methods, that include the HF method of Martens (2010) as a particular example, work by optimizing a local quadratic model of the objective via the linear conjugate gradient algorithm (CG), which is a first-order method. While HF, like all truncated-Newton methods, takes steps computed using partially converged calls to CG, it is naturally accelerated along at least some directions of lower curvature compared to the gradient. It can even be shown (Martens & Sutskever, 2012) that CG will tend to favor convergence to the exact solution to the quadratic sub-problem first along higher curvature directions (with a bias towards those which are more clustered together in their curvature-scalars/eigenvalues).

While CG accumulates information as it iterates which allows it to be optimal in a much stronger sense than any other first-order method (like NAG), once it is terminated, this information is lost. Thus, standard truncated Newton methods can be thought of as persisting information which accelerates convergence (of the current quadratic) only over the number of iterations CG performs. By contrast, momentum methods persist information that can inform new updates across an arbitrary number of iterations.

One key difference between standard truncated Newton methods and HF is the use of “hot-started” calls to CG, which use as their initial solution the one found at the previous call to CG. While this solution was computed using old gradient and curvature information from a previous point in parameter space and possibly a different set of training data, it may be well-converged along certain eigen-directions of the new quadratic, despite being very poorly converged along others (perhaps worse than the default initial solution of $\vec{0}$). However, to the extent to which the new local quadratic model resembles the old one, and in particular in the more difficult to optimize directions of low-curvature (which will arguably be more likely to persist across nearby locations in parameter space), the previous solution will be a preferable starting point to 0, and may even allow for gradually increasing levels of convergence along certain directions which persist

On the importance of initialization and momentum in deep learning

problem	biases	0	0.9N	0.98N	0.995N	0.9M	0.98M	0.995M
add $T = 80$	0.82	0.39	0.02	0.21	0.00025	0.43	0.62	0.036
mul $T = 80$	0.84	0.48	0.36	0.22	0.0013	0.029	0.025	0.37
mem-5 $T = 200$	2.5	1.27	1.02	0.96	0.63	1.12	1.09	0.92
mem-20 $T = 80$	8.0	5.37	2.77	0.0144	0.00005	1.75	0.0017	0.053

Table 5. Each column reports the errors (zero-one losses; sec. 4.2) on different problems for each combination of μ_0 and momentum type (NAG, CM), averaged over 4 different random seeds. The “biases” column lists the error attainable by learning the output biases and ignoring the hidden state. This is the error of an RNN that failed to “establish communication” between its inputs and targets. For each μ_0 , we used the fixed learning rate that gave the best performance.

in the local quadratic models across many updates.

The connection between HF and momentum methods can be made more concrete by noticing that a single step of CG is effectively a gradient update taken from the current point, plus the previous update reapplied, just as with NAG, and that if CG terminated after just 1 step, HF becomes equivalent to NAG, except that it uses a special formula based on the curvature matrix for the learning rate instead of a fixed constant. The most effective implementations of HF even employ a “decay” constant (Martens & Sutskever, 2012) which acts analogously to the momentum constant μ . Thus, in this sense, the CG initializations used by HF allow us to view it as a hybrid of NAG and an exact second-order method, with the number of CG iterations used to compute each update effectively acting as a dial between the two extremes.

Inspired by the surprising success of momentum-based methods for deep learning problems, we experimented with making HF behave even more like NAG than it already does. The resulting approach performed surprisingly well (see Table 1). For a more detailed account of these experiments, see sec. A.6 of the appendix.

If viewed on the basis of each CG step (instead of each update to parameters θ), HF can be thought of as a peculiar type of first-order method which approximates the objective as a series of quadratics only so that it can make use of the powerful first-order CG method. So apart from any potential benefit to global convergence from its tendency to prefer certain directions of movement in parameter space over others, perhaps the main theoretical benefit to using HF over a first-order method like NAG is its use of CG, which, while itself a first-order method, is well known to have strongly optimal convergence properties for quadratics, and can take advantage of clustered eigenvalues to accelerate convergence (see Martens & Sutskever (2012) for a detailed account of this well-known phenomenon). However, it is known that in the worst case that CG, when run in batch mode, will converge asymptotically no faster than NAG (also run in batch mode) for certain specially designed quadratics with very evenly distributed eigenvalues/curvatures. Thus it is worth asking whether the quadratics which arise during the optimization of neural networks by HF are such that CG has a distinct advantage in optimizing

them over NAG, or if they are closer to the aforementioned worst-case examples. To examine this question we took a quadratic generated during the middle of a typical run of HF on the curves dataset and compared the convergence rate of CG, initialized from zero, to NAG (also initialized from zero). Figure 5 in the appendix presents the results of this experiment. While this experiment indicates some potential advantages to HF, the closeness of the performance of NAG and HF suggests that these results might be explained by the solutions leaving the area of trust in the quadratics before any extra speed kicks in, or more subtly, that the faithfulness of approximation goes down just enough as CG iterates to offset the benefit of the acceleration it provides.

6. Discussion

Martens (2010) and Martens & Sutskever (2011) demonstrated the effectiveness of the HF method as a tool for performing optimizations for which previous attempts to apply simpler first-order methods had failed. While some recent work (Chapelle & Erhan, 2011; Glorot & Bengio, 2010) suggested that first-order methods can actually achieve some success on these kinds of problems when used in conjunction with good initializations, their results still fell short of those reported for HF. In this paper we have completed this picture and demonstrated conclusively that a large part of the remaining performance gap that is not addressed by using a well-designed random initialization is in fact addressed by careful use of momentum-based acceleration (possibly of the Nesterov type). We showed that careful attention must be paid to the momentum constant μ , as predicted by the theory for local and convex optimization.

Momentum-accelerated SGD, despite being a first-order approach, is capable of accelerating directions of low-curvature just like an approximate Newton method such as HF. Our experiments support the idea that this is important, as we observed that the use of stronger momentum (as determined by μ) had a dramatic effect on optimization performance, particularly for the RNNs. Moreover, we showed that HF can be viewed as a first-order method, and as a generalization of NAG in particular, and that it already derives some of its benefits through a momentum-like mechanism.

Understanding and Improving Layer Normalization

Jingjing Xu¹, Xu Sun^{1,2*}, Zhiyuan Zhang¹, Guangxiang Zhao², Junyang Lin¹

¹ MOE Key Lab of Computational Linguistics, School of EECS, Peking University

² Center for Data Science, Peking University

{jingjingxu, xusun, zzy1210, zhaoguangxiang, linjunyang}@pku.edu.cn

Abstract

Layer normalization (LayerNorm) is a technique to normalize the distributions of intermediate layers. It enables smoother gradients, faster training, and better generalization accuracy. However, it is still unclear where the effectiveness stems from. In this paper, our main contribution is to take a step further in understanding LayerNorm. Many of previous studies believe that the success of LayerNorm comes from forward normalization. Unlike them, we find that the derivatives of the mean and variance are more important than forward normalization by re-centering and re-scaling backward gradients. Furthermore, we find that the parameters of LayerNorm, including the bias and gain, increase the risk of over-fitting and do not work in most cases. Experiments show that a simple version of LayerNorm (LayerNorm-simple) without the bias and gain outperforms LayerNorm on four datasets. It obtains the state-of-the-art performance on En-Vi machine translation. To address the over-fitting problem, we propose a new normalization method, Adaptive Normalization (AdaNorm), by replacing the bias and gain with a new transformation function. Experiments show that AdaNorm demonstrates better results than LayerNorm on seven out of eight datasets.

1 Introduction

Neural network training has long been a focus in Deep Learning research area. One of the prominent progress is the application of normalization methods. Initially, Ioffe and Szegedy [2015] introduce the concept of normalizing layers with the proposed Batch Normalization (BatchNorm). It is widely believed that by controlling the mean and variance of layer inputs across mini-batches, BatchNorm stabilizes the distribution and improves training efficiency. Following this work, Lei Ba et al. [2016] point out its limitation in Recurrent Neural Networks (RNN) and propose Layer Normalization (LayerNorm) that is performed across the neurons in a layer. LayerNorm is adaptive to RNN and self-attention-based models. A typical example is its application in the state-of-the-art framework, Transformer [Vaswani et al., 2017]. LayerNorm enables faster training of Transformer and is irreplaceable in this framework.

Despite its great success, it is still unclear why LayerNorm is so effective. The widely accepted explanation is that forward normalization brings distribution stability [Ioffe and Szegedy, 2015, Lei Ba et al., 2016]. Recent studies show that the effects of BatchNorm are not related to the stability of input distribution [Zhang et al., 2017, Santurkar et al., 2018]. They also propose that the reason why BatchNorm is effective is that normalization smooths the optimization landscape. However, it is still unclear whether these theories can explain the success of LayerNorm.

The main contribution of this paper is to explore how LayerNorm works. Through a series of analyses, we find that the derivatives of the mean and variance are important by re-centering and re-scaling

*Corresponding author.

backward gradients. Furthermore, it is beyond our expectation that the bias and gain do not work in most cases. The details of our findings are illustrated below.

The derivatives of the mean and variance are more important to LayerNorm than forward normalization. Many of the previous studies believe that the forward normalization is the only decisive factor to LayerNorm. It makes the input distribution more stable, thus brings better convergence. Unlike them, our experimental results show that forward normalization has little to do with the effectiveness and the derivatives of the mean and variance play a significant role in LayerNorm. To illustrate how these derivatives work, we propose DetachNorm, which adds an additional detaching operation to LayerNorm to change the mean and variance from variables to constants. It preserves the re-centering and re-scaling fact but cuts off the derivative of the mean and variance with respect to the input. DetachNorm performs worse than LayerNorm on six out of eight datasets. This proves that the derivatives of the mean and variance are useful to LayerNorm. Furthermore, to investigate the reason for the above observation, we analyze the gradients in LayerNorm and DetachNorm, and find that the derivatives of means re-center gradients and the derivatives of variances re-scale gradients.

The parameters of LayerNorm, including the bias and gain, increase the risk of over-fitting and do not work in most cases. The bias and gain are applied for affine transformation on normalized vectors. They are expected to enhance the expressive power by re-shaping the distribution. To evaluate their effects on results, we build a simple version of LayerNorm (LayerNorm-simple) by removing the bias and gain. Our experimental results show that LayerNorm-simple achieves better results than LayerNorm on four datasets. It even achieves the state-of-the-art performance on En-Vi machine translation. By comparing loss curves of LayerNorm with and without the bias and gain, we find that the bias and gain cause over-fitting. We speculate the reason of over-fitting is mainly that the bias and gain are learned from the training set and cannot adjust themselves towards different input distributions when testing.

Motivated by this assumption, we propose a novel normalization method, Adaptive Normalization (AdaNorm). AdaNorm replaces the bias and gain with a new transformation function. This function adaptively adjusts scaling weights based on input values. We evaluate AdaNorm and LayerNorm on eight datasets, covering tasks of machine translation, language modeling, text classification, image classification, and dependency parsing. Results show that AdaNorm achieves better results on seven datasets.

2 Preliminaries

In this section, we first review the algorithm of LayerNorm and then introduce the datasets and models used in the following analysis sections.

2.1 LayerNorm Algorithm

Let $\mathbf{x} = (x_1, x_2, \dots, x_H)$ be the vector representation of an input of size H to normalization layers. LayerNorm re-centers and re-scales input $\mathbf{x}$ as

$$\mathbf{h} = \mathbf{g} \odot N(\mathbf{x}) + \mathbf{b}, \quad N(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma}, \quad \mu = \frac{1}{H} \sum_{i=1}^H x_i, \quad \sigma = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2} \quad (1)$$

where $\mathbf{h}$ is the output of a LayerNorm layer. $\odot$ is a dot production operation. μ and σ are the mean and standard deviation of input. Bias $\mathbf{b}$ and gain $\mathbf{g}$ are parameters with the same dimension H .

2.2 Experimental Setup

To investigate how LayerNorm works, we conduct a series of experiments in this paper. Since LayerNorm is a default setting in Transformer [Vaswani et al., 2017] and Transformer-XL [Dai et al., 2019], which have shown state-of-the-art results on a variety of tasks (e.g., machine translation), we primarily consider normalization on Transformer and Transformer-XL networks. Also, to avoid the impact of model architecture, we evaluate the effects of normalization on feed-forward neural networks and convolutional neural networks. Here list the datasets and models. More details can be found at the Appendix.

Machine translation includes three widely-used datasets, WMT English-German (En-De), IWSLT 14 German-English (De-En) [Cettolo et al., 2014] and IWSLT 15 English-Vietnamese (En-Vi) [Cettolo et al., 2015]. For all dataset, we use the setting of PreNorm where normalization is applied before each layer. We re-implement Transformer with the released code of Fairseq [Ott et al., 2019]². The evaluation metric is BLEU [Papineni et al., 2002].

For En-De dataset, we use the same dataset splits and the same compound splitting following previous work [Vaswani et al., 2017]. BPE is used to get vocabularies. We use the shared embedding setting and the vocabulary size is 32,765. We use “transformer_wmt_en_de_big_t2t” as our basic model. The dropout rate is 0.3. The learning rate is 0.001. The training batch size is 4,096 tokens. We use optimizer Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.98$. The number of warmup steps is 4K.

The De-En dataset is provided by the IWSLT 2014 Evaluation Campaign. We use the same dataset splits following previous work [Ott et al., 2019, Ranzato et al., 2016, Wiseman and Rush, 2016]. It contains 153K sentences for training, 7K sentences for validation, and 7K sentences for testing. BPE is used to get vocabularies. We use the shared embedding setting and the vocabulary size is 10,149. We use “transformer_iwslt_de_en” as our basic model. The dropout rate is 0.3. The attention dropout rate is 0.1. The activation dropout is 0.1. The initialization learning rate is 1e-07 and the learning rate is 0.0015. The training batch size is 4,096 tokens. We update gradients for every 2 steps. The number of warmup steps is 8K.

The En-Vi dataset contains 133K training sentence pairs provided by the IWSLT 2015 Evaluation Campaign. We use TED tst2012 (1,553 sentences) as the validation set and TED tst2013 (1,268 sentences) as the test set. BPE is used to get input and output vocabularies. The English and Vietnamese vocabulary sizes are 7,669 and 6,669 respectively. The dropout rate is 0.1. The learning rate is 0.001. The training batch size is 4,096 tokens. The number of warmup steps is 8K. We use “transformer_wmt_en_de” as our basic model. We use optimizer Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.98$.

Language modeling includes a large dataset, Enwiki8³ that contains 100M bytes of unprocessed Wikipedia text. We implement a 12-layer Transformer-XL model. The dimension of each layer is 512. Multi-head attention contains 8 heads and the dimension of each head is 64. The dropout rate is 0.1. The batch size is 22. We use optimizer Adam with a learning rate 0.00025. We use the average number of Bits-Per-Character (BPC) as the evaluation metric [Al-Rfou et al., 2018, Dai et al., 2019].

Text classification includes two sentence classification datasets: RT [Pang and Lee, 2005], and SST5 [Socher et al., 2013]. RT is a binary sentiment classification dataset from online movie reviews. We randomly divide all examples into 8,608 for training, 964 for validation, and 1,089 for testing. SST5 is a single sentence classification dataset built on movie reviews. We run experiments on a five label set. We build a Transformer model with a 4-layer encoder. The batch size is 4,096 tokens. The word embedding dimension is 128 and the hidden dimension is 128. The dropout rate is 0.2. We use optimizer Adam with $\beta_1 = 0.9$, $\beta_2 = 0.998$. Normalization is applied before each layer. Accuracy is the evaluation metric.

Image classification includes a widely-used dataset, MNIST [LeCun et al., 1998]. It consists of 55,000 training images, 5,000 validation images, and additional 10,000 testing images. We implement a 3-layer convolutional neural network for classification. The first 2D-convolution layer has 1 in-channel, 20 out-channels. The second 2D-convolution layer has 20 in-channels, 50 out-channels. We flatten the output of the second 2D-convolution layer and send it to a linear layer. The batch size is 32. We use optimizer Adam with a learning rate of 0.001. We apply LayerNorm before the activation in every linear layer. We train the model for 20 epochs. Normalization is applied before each layer. Accuracy is the evaluation metric.

Dependency parsing includes a dataset, English Penn TreeBank (PTB) [Marcus et al., 1993]. We follow the standard split of the corpus with sections 2-21 as the training set (39,832 sentences, 1,900,056 transition examples), section 22 as the validation set (1,700 sentences, 80,234 transition examples), and section 23 as the testing set (2,416 sentences, 113,368 transition examples). We implement a MLP-based parser following the work [Chen and Manning, 2014]. The dimension of the hidden state is 512, the batch size is 1,024, the dropout rate is 0.2. We use optimizer Adam and initialize the learning rate to 0.001. We apply normalization before activation in every linear layer.

²<https://github.com/pytorch/fairseq>

³<http://mattmahoney.net/dc/text.html>

Following the work [Chen and Manning, 2014], we use Unlabeled Attachment Score (UAS) as the evaluation metric.

3 Understanding LayerNorm

To investigate how LayerNorm facilitates training, we conduct ablation studies to observe each part’s contribution to the performance. In this section, we analyse the effects of the bias and gain, forward normalization, and backward normalization.

Table 1: The bias and gain do not work on six out of eight datasets. “w/o Norm” is a naive model without LayerNorm. “LayerNorm-simple” is a variant of LayerNorm that drops the bias and gain. “(+)” means higher is better. “(-)” means lower is better.

Models	Machine Translation			Language Modeling	Classification			Parsing
	En-De(+)	De-En(+)	En-Vi(+)	Enwiki8(-)	RT(+)	SST5(+)	MNIST(+)	PTB(+)
Model Layers	12	12	12	12	4	4	3	3
w/o Norm	Diverge	34.0	28.4	1.04	76.85	38.55	99.14	88.31
LayerNorm	28.3	35.5	31.2	1.07	77.21	39.23	99.13	89.12
LayerNorm-simple	28.4	35.5	31.6	1.07	76.66	40.54	99.09	89.19

3.1 The Effect of the Bias and Gain in LayerNorm

The bias and gain do not work in most cases. From Table 1 it can be found that LayerNorm is an effective approach. It brings large performance improvements on six out of eight datasets compared with the naive baseline without LayerNorm (“w/o Norm”). By comparing LayerNorm and LayerNorm-simple, we find that dropping the bias and gain (“LayerNorm-simple”) does not decrease the performance on six datasets. Surprisingly, LayerNorm-simple outperforms LayerNorm on four datasets, even with a 0.4 BLEU improvement on En-Vi and a 1.31 ACC improvement on SST-5. Also, it needs to notice that 31.6 achieved by LayerNorm-simple is the state-of-the-art result on En-Vi machine translation.

Furthermore, we find that the bias and gain increase the risk of over-fitting. Initially, considering that input information may be lost when normalizing input distributions, the bias and gain are designed for affine transformation on normalized vectors to enhance the expressive power. However, since the bias and gain are learned from the training set and they ignore the input distributions of the testing data, the risk of over-fitting may increase in LayerNorm. It is verified by convergence curves in Figure 1. LayerNorm achieves lower training loss (or BPC) but higher validation loss (or BPC) than LayerNorm-simple on En-Vi, Enwiki8. These results indicate that current affine transformation mechanism has a potential risk of over-fitting and needs to be further improved.

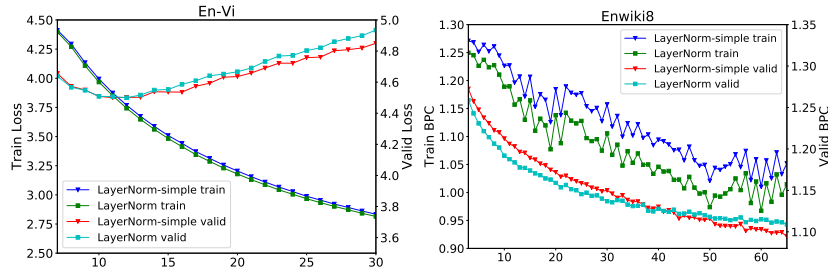


Figure 1: Convergence curves of LayerNorm and LayerNorm-simple on En-Vi, Enwiki8. Lower is better. The bias and gain increase the risk of over-fitting.

3.2 The Effect of Forward Normalization

For easier analysis, we only consider LayerNorm without the bias and gain here. Let $\mathbf{y} = (y_1, y_2, \dots, y_H)$ be the normalized vector, the calculation process of LayerNorm without the bias and

Table 2: The derivatives of the mean and variance matter. “w/o Norm” is the naive model without normalization. “DetachNorm” is a variant of “LayerNorm-simple”. It detaches the derivatives of the mean and variance. “(+)” means higher is better. “(-)” means lower is better. The top table shows the effect of forward normalization. The bottom table shows the effect of the derivatives of the mean and variance.

Models	Machine Translation			Language Modeling	Classification			Parsing
	En-De	De-En(+)	En-Vi(+)	Enwiki8(-)	RT(+)	SST5(+)	MNIST(+)	PTB(+)
Model Layers	12	12	12	12	4	4	3	3
w/o Norm	Diverge	34.0	28.4	1.04	76.85	38.55	99.14	88.31
DetachNorm	Diverge	33.9	27.7	1.12	76.40	40.04	99.10	89.79
Improvement	–	-0.1	-0.7	-0.08	-0.45	1.49	-0.04	1.48

Models	Machine Translation			Language Modeling	Classification			Parsing
	En-De	De-En(+)	En-Vi(+)	Enwiki8(-)	RT(+)	SST5(+)	MNIST(+)	PTB(+)
Model Layers	12	12	12	12	4	4	3	3
DetachNorm	Diverge	33.9	27.7	1.12	76.40	40.04	99.10	89.79
LayerNorm-simple	28.4	35.5	31.6	1.07	76.66	40.54	99.09	89.19
Improvement	–	1.6	3.9	0.05	0.26	0.50	-0.01	-0.60

gain can be written as

$$\mathbf{y} = \frac{\mathbf{x} - \mu}{\sigma}, \quad \mu = \frac{1}{H} \sum_{i=1}^H x_i, \quad \sigma = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2} \quad (2)$$

where $\mathbf{x} = (x_1, x_2, \dots, x_H)$ is the input vector and H is the dimension of $\mathbf{x}$. μ and σ are the mean and standard deviation of $x_1, x_2, \dots, x_H$. Then, suppose $\bar{y}$ and D_y are the mean and variance of $y_1, y_2, \dots, y_H$. It is easy to verify

$$\bar{y} = \frac{1}{H} \sum_{i=1}^H y_i = \frac{1}{H} \sum_{i=1}^H \frac{(x_i - \mu)}{\sigma} = 0, \quad D_y = \frac{1}{H} \sum_{i=1}^H \frac{(x_i - \mu)^2}{\sigma^2} = 1. \quad (3)$$

Eq. (3) shows that normalization re-centers and re-scales input vector $\mathbf{x}$. By now, a widely accepted belief is that the effectiveness of LayerNorm comes from steady layer distributions brought by forward normalization (Lei Ba et al., 2016). To evaluate whether forward normalization explains the effectiveness of LayerNorm, we need to separate the effect on forward layer inputs and that on backward gradients. In this paper, we design a new method, called DetachNorm. The difference between LayerNorm and DetachNorm is that DetachNorm detaches the derivatives of the mean and variance⁴. Detaching derivatives means treating the mean and variance as changeable constants, rather than variables, which do not require gradients in backward propagation. The calculation of DetachNorm can be written as

$$\mathbf{y} = \frac{\mathbf{x} - \hat{\mu}}{\hat{\sigma}}, \quad \hat{\mu} = \theta(\mu), \quad \hat{\sigma} = \theta(\sigma) \quad (4)$$

where μ and σ are the mean and standard deviation of input $\mathbf{x}$, as calculated in Eq. (2). The function $\theta(\cdot)$ can be seen as a special copy function, which copies the values of μ and σ into constants $\hat{\mu}$ and $\hat{\sigma}$. In all, DetachNorm keeps the same forward normalization fact as LayerNorm does, but cuts off the derivatives of the mean and variance.

Since DetachNorm keeps the same re-centering and re-scaling way in forward propagation as LayerNorm-simple does, the gap between DetachNorm and “w/o Norm” shows the effect of forward normalization. As we can see, DetachNorm perform worse than “w/o Norm”, showing that forward normalization has little to do with the success of LayerNorm.

Furthermore, the only difference between DetachNorm and LayerNorm-simple lies in that DetachNorm detaches the derivatives of the mean and variance. As shown in Table 2, DetachNorm performs

⁴In our implementation, we detach the derivative of standard deviation, the square root of variance.

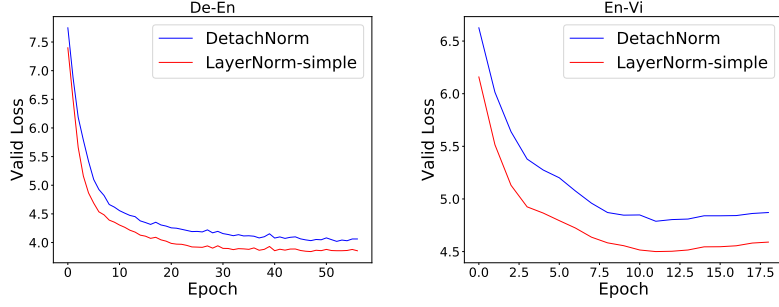


Figure 2: Convergence curves of LayerNorm-simple and DetachNorm on two translation datasets.

worse than LayerNorm-simple on six datasets. It is mainly because that DetachNorm converges to much worse local optima compared with LayerNorm-simple, as shown in Figure 2. The gap between DetachNorm and LayerNorm-simple shows the effectiveness of the derivatives of the mean and variance. By comparing the achieved improvements, we find that the derivatives of the mean and variance bring higher improvements than forward normalization does.

These results demonstrate that the derivatives of the mean and variance play a significant role. In addition, the extremely worse results of DetachNorm on En-De, De-En and En-Vi indicate that the derivatives of the mean and variance may be more important for deeper models. In the following section, we will give a detailed analysis of why and how the derivatives of the mean and variance contribute to the performance.

3.3 The Effect of the Derivatives of the Mean and Variance

To understand how the derivatives of the mean and variance work, we analyze the gradients of LayerNorm-simple and DetachNorm. According to the chain rule, the gradient of $\mathbf{x}$ is⁵

$$\frac{\partial \ell}{\partial \mathbf{x}} \leftarrow \frac{d\mathbf{y}}{d\mathbf{x}} \frac{\partial \ell}{\partial \mathbf{y}} \quad (5)$$

where ℓ is the loss function, $\mathbf{x}$ is the input vector and $\mathbf{y}$ is the normalized vector. We here analyze the effect of detaching the derivatives of the mean and variance on backward gradients. Our results are summarized in the following theorem, whose proof is listed in the Appendix.

Theorem 1. Given $\frac{\partial \ell}{\partial \mathbf{y}} = (g_1, g_2, \dots, g_H)^T$, let $\bar{g}$ and D_g be the mean and variance of $g_1, g_2, \dots, g_H$. For the case of detaching the derivatives of μ and σ , suppose $\frac{\partial \ell}{\partial \mathbf{x}} = (a_1, a_2, \dots, a_H)^T$ is the gradient of $\mathbf{x}$ with mean $\bar{a}$ and variance D_a . We have $\bar{a} = \bar{g}/\sigma$ and $D_a = D_g/(\sigma^2)$.

(1) For the case of standard LayerNorm-simple, suppose $\frac{\partial \ell}{\partial \mathbf{x}} = (b_1, b_2, \dots, b_H)^T$ is the gradient of $\mathbf{x}$ with mean $\bar{b}$ and variance D_b .

$$\text{We have } \bar{b} = 0 \text{ and } D_b \leq D_g/(\sigma^2).$$

(2) For the case of detaching the derivative of μ , suppose $\frac{\partial \ell}{\partial \mathbf{x}} = (c_1, c_2, \dots, c_H)^T$ is the gradient of $\mathbf{x}$ with mean $\bar{c}$ and variance D_c .

$$\text{We have } \bar{c} = \bar{g}/\sigma \text{ and } D_c \leq D_g/(\sigma^2).$$

(3) For the case of detaching the derivative of σ , suppose $\frac{\partial \ell}{\partial \mathbf{x}} = (d_1, d_2, \dots, d_H)^T$ is the gradient of $\mathbf{x}$ with mean $\bar{d}$ and variance D_d .

$$\text{We have } \bar{d} = 0 \text{ and } D_d = D_g/(\sigma^2).$$

By comparing the case of detaching the derivative of μ with that of LayerNorm-simple in Theorem 1, we find that the derivative of μ re-centers $\frac{\partial \ell}{\partial \mathbf{x}}$ to zero. By comparing the case of detaching the

⁵When calculating the gradient, we adopt the denominator layout.

derivative of σ with of LayerNorm-simple, we find that the derivative of σ reduces the variance of $\frac{\partial \ell}{\partial \mathbf{x}}$, which can be seen a kind of re-scaling. We refer to gradient re-centering and re-scaling as gradient normalization.

To further evaluate the effect of gradient normalization on model performance, we test the derivatives of the mean and variance separately. Table 3 shows that detaching the derivative of variance decreases the performance significantly on deeper networks. Therefore, it is necessary to control the variance of gradients for deeper networks.

In conclusion, LayerNorm normalizes forward layer inputs and backward gradients. The derivatives of the mean and variance play more important roles than forward normalization in LayerNorm. Furthermore, unlike previous work [Santurkar et al., 2018] only noticing that normalization smooths gradients, this paper provides deeper insight about how normalization impacts backward gradients.

Table 3: The derivative of variance is more important than that of mean for deeper networks. “(+)” means higher is better. “(-)” means lower is better.

Models	Machine Translation			Language Model	Classification			Parsing
	En-De(+)	De-En(+)	En-Vi(+)	Enwiki8(-)	RT(+)	SST5(+)	MNIST(+)	PTB(+)
Model Layers	12	12	12	12	4	4	3	3
LayerNorm-simple	28.4	35.5	31.6	1.07	76.66	40.54	99.09	89.19
Detach Mean	28.3	35.6	31.3	1.07	75.02	40.99	99.25	89.45
Detach Variance	Diverge	34.2	29.8	1.10	77.04	41.74	99.10	89.80

4 AdaNorm

AdaNorm adopts a new transformation function which can adaptively control scaling weights towards different inputs.⁶

4.1 AdaNorm Algorithm

Formally, let $\mathbf{y} = N(\mathbf{x}) = (\mathbf{x} - \mu) / \sigma$ be the normalized vector where μ and σ are the mean and variance of the input $\mathbf{x} = (x_1, x_2, \dots, x_H)$. We use $\phi(\mathbf{y})$, a function with respect to input $\mathbf{x}$, to replace the bias and gain with the following equation:

$$\mathbf{z} = \phi(\mathbf{y}) \odot \mathbf{y} = \phi(N(\mathbf{x})) \odot N(\mathbf{x}) \quad (6)$$

where $\mathbf{z} = (z_1, z_2, \dots, z_H)$ is the output of AdaNorm and $\odot$ is a dot product operation. Unlike the bias and gain being fixed in LayerNorm, $\phi(\mathbf{y})$ can adaptively adjust scaling weights based on inputs. To keep the stability of training, we expect that $\phi(\cdot)$ has some features. First, $\phi(\cdot)$ must be differentiable. Second, we expect that the average scaling weight is fixed, namely the average of $\phi(\mathbf{y})$ is a constant C where $C > 0$. Third, we expect that the average of $\mathbf{z}$ is bounded, which can avoid the problem of exploding loss. Namely, we require that there exists a constant M such that $|\frac{1}{H} \sum_{i=1}^H z_i| < M$. Theorem 2 proves that there exists a unique solution which can satisfy these requirements. The proof is listed in the Appendix.

Theorem 2. Suppose $\phi(y_i)$ is derivable, $\forall \mathbf{y}$, $\frac{1}{H} \sum_{i=1}^H \phi(y_i) = C > 0$, and $\exists M$, s.t. $|\frac{1}{H} \sum_{i=1}^H z_i| < M$ ($M > 0$), where H is the hidden size. There exists only one solution:

$$\phi(y_i) = C(1 - ky_i)$$

which can satisfy these requirements.

Since $1 - ky_i < 0$ will undesirably change the direction of vector, we expect that $\phi(y_i) > 0$ holds, which means $y_i < 1/k$ must hold. Due to the symmetry of y_i , $|y_i| < 1/k$ is required to hold too.

⁶Our code is released at <https://github.com/lancopku/AdaNorm>

Based on Chebyshev’s Inequality, we have

$$P(|y_i| < 1/k) = P(|y_i - E(y_i)| < 1/k) \geq 1 - \frac{D_y}{(1/k)^2} = 1 - k^2 D_y \quad (7)$$

where D_y is the variance of $\mathbf{y} = (y_1, y_2, \dots, y_H)$ and H is the dimension of $\mathbf{y}$. Based on Eq. (3), we can verify $D_y = 1$. If we expect that $|y_i| < 1/k$ holds with a probability higher than 99%, $k = 1/10$ should be choose based on Eq. (7). Namely, we choose

$$\phi(y_i) = C(1 - \frac{y_i}{10}). \quad (8)$$

Given an input vector $\mathbf{x}$, the complete calculation process of AdaNorm is

$$\mathbf{z} = C(1 - k\mathbf{y}) \odot \mathbf{y}, \quad \mathbf{y} = \frac{\mathbf{x} - \mu}{\sigma}, \quad \mu = \frac{1}{H} \sum_{i=1}^H x_i, \quad \sigma = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2} \quad (9)$$

where C is a hyper-parameter. $\odot$ is a dot product operation. k is recommended to set as $1/10$. To prevent the introduced term $C(1 - k\mathbf{y})$ dismissing the feature of gradient re-centering and re-scaling, we detach the gradient of $C(1 - k\mathbf{y})$ and only treat it as a changeable constant in implementation.

Table 4: Results of LayerNorm and AdaNorm. “(+)” means higher is better. “(-)” means lower is better. AdaNorm outperforms LayerNorm on seven datasets.

Models	Machine Translation			Language Model	Classification			Parsing
	En-De(+)	De-En(+)	En-Vi(+)	Enwiki8(-)	RT(+)	SST5(+)	MNIST(+)	PTB(+)
w/o Norm	Diverge	34.0	28.4	1.04	76.85	38.55	99.14	88.31
LayerNorm	28.3	35.5	31.2	1.07	77.21	39.23	99.13	89.12
LayerNorm-simple	28.4	35.5	31.6	1.07	76.66	40.54	99.09	89.19
AdaNorm	28.5	35.6	31.4	1.07	77.50	40.54	99.35	89.23

4.2 Comparison between AdaNorm and LayerNorm

The comparison between LayerNorm and AdaNorm is shown in Table 4. AdaNorm outperforms LayerNorm on seven datasets, with 0.2 BLEU on En-De, 0.1 BLEU on De-En, 0.2 BLEU on En-Vi, 0.29 ACC on RT, 1.31 ACC on SST, 0.22 ACC on MNIST, and 0.11 F1 on PTB. Unlike LayerNorm-simple only performing well on bigger models, AdaNorm achieves more balanced results. Figure 3 shows the loss curves of LayerNorm and AdaNorm on the validation set of En-Vi, PTB, and De-En. Compared to AdaNorm, LayerNorm has lower training loss but higher validation loss. Lower validation loss proves that AdaNorm has better convergence.

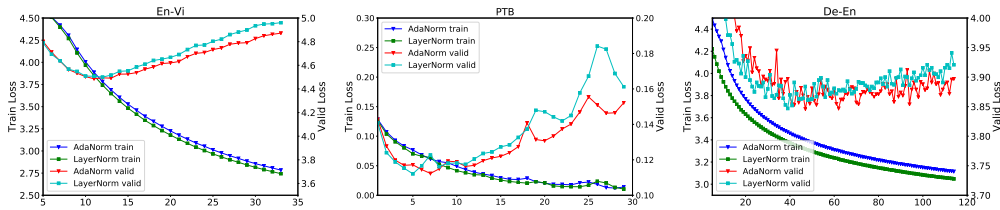


Figure 3: Loss curves of LayerNorm and AdaNorm on En-Vi, PTB, and De-En.

5 Related Work

Deep neural networks have outperformed shallow models in a variety of fields, such as natural language processing [Sutskever et al., 2014, Bahdanau et al., 2015, Devlin et al., 2018], computer vision [He et al., 2016, Huang et al., 2017], etc. The improvement mainly comes from the stronger

⁷For AdaNorm implementation, Kaiming initialization and the setting of pre-norm are recommended.

expressive power of deep layers. However, with the increase of depth, the network training process becomes complicated and requires advanced architectural techniques. One of the important techniques of such advances is normalization.

Currently, it is widely accepted that normalization layers assist training by smoothing gradients, enabling large learning rates, accelerating convergence, and improving generalization results [Zhang et al., 2019]. First introduced by Ioffe and Szegedy [2015], BatchNorm fixes layer distributions to reduce ICS (Internal Covariate Shift), a phenomenon that the upper layers need to continuously adapt to the new distributions of lower layers. Following this work, several normalization methods have been proposed, like instance normalization [Ulyanov et al., 2016] and group normalization [Wu and He, 2018]. In addition, there are several studies exploring better activation functions [Klambauer et al., 2017] or initialization methods [Zhang et al., 2019] to avoid the dependency on normalization layers.

LayerNorm is proposed to expand BatchNorm into RNN. LayerNorm normalizes the mean and variance of all summed inputs to the neurons in one layer. Unlike BatchNorm that depends on the size of mini-batch, LayerNorm has fewer limitations. LayerNorm is adaptive to RNN and self-attention-based models. It has been applied to the state-of-the-art frameworks such as Transformer [Vaswani et al., 2017], BERT [Devlin et al., 2018], and Transformer-XL [Dai et al., 2019]. LayerNorm brings better performance and is irreplaceable in these frameworks.

Despite the good performance, it is still unclear how layer normalization works. Ioffe and Szegedy [2015] claim that the effectiveness of BatchNorm comes from reducing ICS. It has been a popular belief about BatchNorm [Santurkar et al., 2018]. However, some recent studies point out that the success of BatchNorm relates to the smoother gradients and has little to do with reducing ICS [Santurkar et al., 2018; Bjorck et al., 2018]. Although these studies provide a pioneering perspective to understand BatchNorm, there still remain some unanswered questions, such as how BatchNorm helps smooth gradients. Also, there are little work studying whether these theories can explain the success of LayerNorm. In this paper, we take a further step to a better understanding of LayerNorm.

6 Conclusion

In this paper, we investigate how layer normalization works. Based on a series of experiments and theoretical analysis, we summarize some interesting conclusions. We find that the derivatives of the mean and variance are important to the success of LayerNorm by re-centering and re-scaling backward gradients. Furthermore, experiments show that the bias and gain increase the risk of over-fitting and do not work in most cases. To address this problem, we propose a normalization method AdaNorm. It replaces the bias and gain in LayerNorm with a new adaptive transformation function that can update scaling weights based on input values. Experiments show that AdaNorm outperforms LayerNorm on seven datasets. In the future work, we would like to explore more alternatives to LayerNorm from the perspective of gradient normalization.

Acknowledgments

We thank all reviewers for providing the thoughtful and constructive suggestions. This work was supported in part by National Natural Science Foundation of China (No. 61673028).

References

- R. Al-Rfou, D. Choe, N. Constant, M. Guo, and L. Jones. Character-level language modeling with deeper self-attention. *CoRR*, abs/1808.04444, 2018.
- D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- N. Bjorck, C. P. Gomes, B. Selman, and K. Q. Weinberger. Understanding batch normalization. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information*

7 Appendix

7.1 Experimental Settings

7.1.1 Neural Machine Translation

For neural machine translation tasks, we re-implement Transformer with the released code of Fairseq [Ott et al., 2019]⁸.

IWSLT 2015 English-Vietnamese Translation It contains 133K training sentence pairs provided by the IWSLT 2015 Evaluation Campaign. Following the pre-processing steps in the work of Raffel et al. [2017], we use TED tst2012 (1,553 sentences) as the validation set and TED tst2013 (1,268 sentences) as the test set. BPE is used to get input and output vocabularies. The English and Vietnamese vocabulary sizes are 7,669 and 6,669 respectively. The dropout rate is 0.1. The learning rate is 0.001. The training batch size is 4,096 tokens. The number of warmup steps is 8K. We use “transformer_wmt_en_de” as our basic model. The setting of PreNorm is adopted. We use optimizer Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.98$. For AdaNorm, the hyper-parameter C is set to 1. We average the last 10 checkpoints for evaluation and set the beam size to 5.

IWSLT 2014 German-English Translation It is provided by the IWSLT 2014 Evaluation Campaign. We use the same dataset splits following previous work [Ott et al., 2019, Ranzato et al., 2016, Wiseman and Rush, 2016]. It contains 153K sentences for training, 7K sentences for validation, and 7K sentences for testing. BPE is used to get vocabularies. We use the shared embedding setting and the vocabulary size is 10,149. We use “transformer_iwslt_de_en” as our basic model. The setting of PreNorm is adopted. The dropout rate is 0.3. The attention dropout rate is 0.1. The activation dropout is 0.1. The initialization learning rate is 1e-07 and the learning rate is 0.0015. The training batch size is 4,096 tokens. We use optimizer Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.98$. We update the gradients for every 2 steps. The number of warmup steps is 8K. For AdaNorm, the hyper-parameter C is set to 2. We average the last 10 checkpoints for evaluation and set the beam size to 5.

WMT English-German Translation Following previous work [Vaswani et al., 2017], we use the same dataset splits and the same compound splitting. The pre-processing code is provided by Fairseq. BPE is used to get vocabularies. We use the shared embedding setting and the vocabulary size is 32,765. We use “transformer_wmt_en_de_big_t2t” as our basic model. The setting of PreNorm is adopted. The dropout rate is 0.3. The learning rate is 0.001. The training batch size is 4,096 tokens. We use optimizer Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.98$. The number of warmup steps is 4K. For AdaNorm, the hyper-parameter C is set to 2. We average the last 10 checkpoints for evaluation and set the beam size to 4.

7.1.2 Language Modeling

Enwiki-8⁹ This is a character-level language model dataset with 100M bytes. We use the same preprocessed dataset as in the work [Chung et al., 2017]. We use the code provided by Transformer-XL¹⁰. We use the default hyper-parameters in the code. The model contains 12 decoder layers and the dimension of each layer is 512. Multi-head attention contains 8 heads and the dimension of each head is 64. The dropout rate is 0.1. The batch size is 22. We use optimizer Adam with a learning rate 0.0025. For AdaNorm, the hyper-parameter C is set to 1. We choose the best checkpoint on the validation set to evaluate the result on the test set.

7.1.3 Classification

RT The rating inference dataset [Pang and Lee, 2005] is a binary sentiment classification dataset from online movie reviews. Due to the lack of the standard split, we randomly divide all examples into 8,608 for training, 964 for validation, and 1,089 for testing. We implement a 4-layer Transformer encoder. The setting of PreNorm is adopted. The batch size is 4,096 tokens. The word embedding dimension is 128, the hidden dimension is 128. The dropout rate is 0.2. The optimization method is Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.998$. For AdaNorm, the hyper-parameter C is set to 0.3.

⁸<https://github.com/pytorch/fairseq>

⁹<http://www.mattmahoney.net/dc/text.html>

¹⁰<https://github.com/kimiyoung/transformer-xl>

SST The Stanford sentiment treebank [Socher et al., 2013] is a single-sentence classification dataset built on movie reviews. We run experiments on a five label set. It provides the standard split, with 8,544 for training, 1,101 for validation, and 2,210 for testing. We use the same model structure in RT. For AdaNorm, the hyper-parameter C is set to 0.3. The rest of parameters are set exactly the same as in RT settings.

MNIST Image Recognition The MNIST handwritten digit dataset [LeCun et al., 1998] consists of 55,000 training images, 5,000 validation images, and additional 10,000 testing images. This task aims to recognize the numerical digit (0-9) of each image. We implement a CNN based classifier. The first 2D-convolution layer has 1 in-channel, 20 out-channels. The second 2D-convolution layer has 20 in-channels, 50 out-channels. We flatten the output of the second 2D-convolution layer and send it to a linear layer. The batch size is 32. We use Adam optimizer with a learning rate of 0.001. We apply LayerNorm before activation in every linear layer. When applying AdaNorm, we set hyper-parameter C to 2. We train the model for 20 epochs. We choose the best checkpoint on the validation set for evaluation.

7.1.4 Dependency Parsing

Transition-based Dependency Parsing Following previous work, we use English Penn TreeBank (PTB) [Marcus et al., 1993] for experiments. We follow the standard split of the corpus with sections 2-21 as the training set (39,832 sentences, 1,900,056 transition examples), section 22 as the validation set (1,700 sentences, 80,234 transition examples), and section 23 as the testing set (2,416 sentences, 113,368 transition examples). We implement a MLP-based parser following the work [Chen and Manning, 2014]. The dimension of the hidden state is 512, the batch size is 1,024, the dropout rate is 0.2. We use optimizer Adam and initialize the learning rate to 0.001. We apply LayerNorm before activation in every linear layer. When applying AdaNorm, we set hyper-parameter C to 1. We train 20 epochs on the training set. We evaluate the model on the development set every epoch and find the best checkpoint to evaluate the test results.

7.2 Proof of Theorem 1

Proof. Define $\mathbf{1}_H = (1, 1, \dots, 1)^\top$. It is easy to verify

$$\begin{aligned}\mathbf{y}^\top \mathbf{y} &= \sum_{i=1}^H y_i^2 = \sum_{i=1}^H \left(\frac{x_i - \mu}{\sigma} \right)^2 = H \\ \mathbf{1}_H^\top \mathbf{1}_H &= \sum_{i=1}^H 1^2 = H \\ \mathbf{y}^\top \mathbf{1}_H &= \sum_{i=1}^H y_i = \sum_{i=1}^H \frac{x_i - \mu}{\sigma} = 0\end{aligned}\tag{10}$$

The forward propagation

$$\mathbf{y} = \frac{\mathbf{x} - \mu \mathbf{1}_H}{\sigma}\tag{11}$$

Calculating the gradient in backward propagation

$$\begin{aligned}\frac{\partial \mathbf{y}}{\partial \mathbf{x}} &= \frac{\partial((\mathbf{I}\mathbf{x} - \mu \mathbf{1}_H)/\sigma)}{\partial \mathbf{x}} = \frac{1}{\sigma} \mathbf{I}^\top = \frac{1}{\sigma} \mathbf{I} \\ \frac{\partial \mathbf{y}}{\partial \mu} &= \frac{\partial((\mathbf{I}\mathbf{x} - \mu \mathbf{1}_H)/\sigma)}{\partial \mu} = -\frac{1}{\sigma} \mathbf{1}_H^\top \\ \frac{\partial \mathbf{y}}{\partial \sigma} &= \frac{\partial((\mathbf{I}\mathbf{x} - \mu \mathbf{1}_H)/\sigma)}{\partial \sigma} = -\frac{(\mathbf{I}\mathbf{x} - \mu \mathbf{1}_H)^\top}{\sigma^2} = -\frac{1}{\sigma} \mathbf{y}^\top \\ \frac{\partial \mu}{\partial x_i} &= \frac{\sum_{i=1}^H x_i / H}{\partial x_i} = \frac{1}{H} \implies \frac{\partial \mu}{\partial \mathbf{x}} = \frac{\mathbf{1}_H}{H} \\ \frac{\partial \sigma}{\partial x_i} &= \frac{\partial \sqrt{(\sum_{i=1}^H x_i^2 - H\mu^2)/H}}{\partial x_i} = \frac{1}{2\sigma} \frac{\partial(\sigma^2)}{\partial x_i} \\ &= \frac{1}{2\sigma} \frac{\partial((\sum_{i=1}^H x_i^2 - H\mu^2)/H)}{\partial x_i} = \frac{1}{2\sigma} (2x_i/H - 2\mu \frac{\partial \mu}{\partial x_i}) \\ &= \frac{x_i - \mu}{H\sigma} = \frac{y_i}{H} \implies \frac{\partial \sigma}{\partial \mathbf{x}} = \frac{\mathbf{y}}{H}\end{aligned}\tag{12}$$

To conclude

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{1}{\sigma} \mathbf{I} \quad \frac{\partial \mathbf{y}}{\partial \mu} = -\frac{1}{\sigma} \mathbf{1}_H^\top \quad \frac{\partial \mathbf{y}}{\partial \sigma} = -\frac{1}{\sigma} \mathbf{y}^\top \quad \frac{\partial \mu}{\partial \mathbf{x}} = \frac{\mathbf{1}_H}{H} \quad \frac{\partial \sigma}{\partial \mathbf{x}} = \frac{\mathbf{y}}{H}\tag{13}$$

If we detach the gradient of μ and σ , in backward propagation

$$\frac{\partial \ell}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \ell}{\partial \mathbf{y}} = \frac{1}{\sigma} \frac{\partial \ell}{\partial \mathbf{y}}\tag{14}$$

namely

$$a_i = \frac{g_i}{\sigma}\tag{15}$$

Calculating $\bar{a}$ and D_a

$$\begin{aligned}H\bar{a} &= \sum_{i=1}^H a_i = \sum_{i=1}^H \frac{g_i}{\sigma} = \frac{H\bar{g}}{\sigma} \\ HD_a &= \sum_{i=1}^H (a_i - \bar{a})^2 = \sum_{i=1}^H \left(\frac{g_i - \bar{g}}{\sigma} \right)^2 = \frac{HD_g}{\sigma^2}\end{aligned}\tag{16}$$

To conclude, $\bar{a} = \bar{g}/\sigma$ and $D_a = D_g/(\sigma^2)$.

Proof of (1)

(1) In standard layernorm, we do not detach the gradients of μ and σ , in backward propagation

$$\frac{\partial \ell}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} + \frac{\partial \mu}{\partial \mathbf{x}} \frac{\partial \mathbf{y}}{\partial \mu} + \frac{\partial \sigma}{\partial \mathbf{x}} \frac{\partial \mathbf{y}}{\partial \sigma} \right) \frac{\partial \ell}{\partial \mathbf{y}} = \frac{1}{\sigma} \left(I - \frac{\mathbf{y}\mathbf{y}^T}{H} - \frac{\mathbf{1}_H \mathbf{1}_H^T}{H} \right) \frac{\partial \ell}{\partial \mathbf{y}} \quad (17)$$

Define $W_1 = \frac{1}{\sigma} \left(I - \frac{\mathbf{y}\mathbf{y}^T}{H} - \frac{\mathbf{1}_H \mathbf{1}_H^T}{H} \right)$, we can verify that

$$\mathbf{1}_H^T W_1 = \mathbf{1}_H^T \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T + \mathbf{y}\mathbf{y}^T}{H} \right) = \frac{1}{\sigma} (\mathbf{1}_H - \frac{\mathbf{1}_H^T \mathbf{1}_H}{H} \mathbf{1}_H - \frac{\mathbf{1}_H^T \mathbf{y}}{H} \mathbf{y}^T) = \frac{\mathbf{1}_H - \mathbf{1}_H - 0}{\sigma} = 0 \quad (18)$$

Therefore,

$$H\bar{b} = \sum_{i=1}^H b_i = \mathbf{1}_H^T (b_1, b_2, \dots, b_H)^T = \mathbf{1}_H^T W_1 (g_1, g_2, \dots, g_H)^T = 0 \quad (19)$$

For any vector $\mathbf{u}$ vertical to $\mathbf{1}_H$ and $\mathbf{y}$ ($\mathbf{1}_H$ is vertical to $\mathbf{y}$), we have

$$\begin{aligned} W_1 \mathbf{u} &= \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T + \mathbf{y}\mathbf{y}^T}{H} \right) \mathbf{u} = \frac{1}{\sigma} (\mathbf{u} - \mathbf{1}_H \frac{\mathbf{1}_H^T \mathbf{u}}{H} - \mathbf{y} \frac{\mathbf{y}^T \mathbf{u}}{H}) = \frac{1}{\sigma} \mathbf{u} \\ W_1 \mathbf{y} &= \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T + \mathbf{y}\mathbf{y}^T}{H} \right) \mathbf{y} = \frac{1}{\sigma} (\mathbf{y} - \mathbf{1}_H \frac{\mathbf{1}_H^T \mathbf{y}}{H} - \mathbf{y} \frac{\mathbf{y}^T \mathbf{y}}{H}) = \frac{\mathbf{y} - 0 - \mathbf{y}}{\sigma} = 0 \\ W_1 \mathbf{1}_H &= \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T + \mathbf{y}\mathbf{y}^T}{H} \right) \mathbf{1}_H = \frac{1}{\sigma} (\mathbf{1}_H - \mathbf{1}_H \frac{\mathbf{1}_H^T \mathbf{1}_H}{H} - \mathbf{y} \frac{\mathbf{y}^T \mathbf{1}_H}{H}) = \frac{\mathbf{1}_H - \mathbf{1}_H - 0}{\sigma} = 0 \end{aligned} \quad (20)$$

We expand $\mathbf{1}_H$ and $\mathbf{y}$ to a standard orthogonal basis $\mathbf{u}_1 = \mathbf{1}_H/\sqrt{H}$, $\mathbf{u}_2 = \mathbf{y}/\sqrt{H}$, $\mathbf{u}_3, \dots, \mathbf{u}_H$, then for any vector $\mathbf{v} = \sum_{i=1}^H \lambda_i \mathbf{u}_i$, we have

$$\begin{aligned} W_1 \mathbf{v} &= \sum_{i=1}^H \lambda_i W_1 \mathbf{u}_i = W_1 \mathbf{1}_H/\sqrt{H} + W_1 \mathbf{y}/\sqrt{H} + \sum_{i=3}^H \lambda_i W_1 \mathbf{u}_i = \frac{1}{\sigma} \sum_{i=3}^H \lambda_i \mathbf{u}_i \\ \|W_1 \mathbf{v}\|^2 &= \frac{1}{\sigma^2} \sum_{i=3}^H \lambda_i^2 \leq \frac{1}{\sigma^2} \sum_{i=1}^H \lambda_i^2 = \frac{1}{\sigma^2} \|\mathbf{v}\|^2 \end{aligned} \quad (21)$$

Therefore,

$$\begin{aligned} HD_b &= \sum_{i=1}^H (b_i - \bar{b})^2 \\ &= \sum_{i=1}^H b_i^2 \\ &= \|(b_1, b_2, \dots, b_H)^T\|^2 \\ &= \|W_1 (g_1, g_2, \dots, g_H)^T\|^2 \\ &= \|W_1 (g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T + W_1 \bar{g} \mathbf{1}_H\|^2 \\ &= \|W_1 (g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T\|^2 \\ &\leq \frac{1}{\sigma^2} \|(g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T\|^2 \\ &= \frac{1}{\sigma^2} \sum_{i=1}^H (g_i - \bar{g})^2 \quad (\text{Ineq. 21}) \\ &= \frac{1}{\sigma^2} HD_g \\ &= HD_a \end{aligned} \quad (22)$$

□

To conclude, $\bar{b} = 0$ and $D_b \leq D_a = D_g/(\sigma^2)$.

Proof of (2) (2) If we detach the gradients of μ , in backward propagation

$$\frac{\partial \ell}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} + \frac{\partial \sigma}{\partial \mathbf{x}} \frac{\partial \mathbf{y}}{\partial \sigma} \right) \frac{\partial \ell}{\partial \mathbf{y}} = \frac{1}{\sigma} \left(I - \frac{\mathbf{y}\mathbf{y}^T}{H} \right) \frac{\partial \ell}{\partial \mathbf{y}} \quad (23)$$

Define $W_2 = \frac{1}{\sigma} \left(I - \frac{\mathbf{y}\mathbf{y}^T}{H} \right)$, then

$$\mathbf{1}_H^T W_2 = \mathbf{1}_H^T \frac{1}{\sigma} \left(I - \frac{\mathbf{y}\mathbf{y}^T}{H} \right) = \frac{1}{\sigma} \left(\mathbf{1}_H^T - \frac{\mathbf{1}_H^T \mathbf{y}}{H} \mathbf{y}^T \right) = \frac{\mathbf{1}_H^T - 0}{\sigma} = \frac{\mathbf{1}_H^T}{\sigma} \quad (24)$$

Therefore,

$$H\bar{c} = \sum_{i=1}^H c_i = \mathbf{1}_H^T (c_1, \dots, c_H)^T = \mathbf{1}_H^T W_2 (g_1, \dots, g_H)^T = \frac{\mathbf{1}_H^T (g_1, \dots, g_H)^T}{\sigma} = \sum_{i=1}^H g_i = \frac{H\bar{g}}{\sigma} \quad (25)$$

Consider

$$\begin{aligned} (I - \mathbf{1}_H \mathbf{1}_H^T / H) W_2 &= (I - \mathbf{1}_H \mathbf{1}_H^T / H) \frac{1}{\sigma} (I - \mathbf{y}\mathbf{y}^T / H) \\ &= \frac{1}{\sigma} (I - \mathbf{1}_H \mathbf{1}_H^T / H - \mathbf{y}\mathbf{y}^T / H + \mathbf{1}_H \mathbf{1}_H^T \mathbf{y}\mathbf{y}^T / (H^2)) \\ &= \frac{1}{\sigma} (I - \mathbf{1}_H \mathbf{1}_H^T / H - \mathbf{y}\mathbf{y}^T / H + 0) \\ &= W_1 \end{aligned} \quad (26)$$

Therefore,

$$\begin{aligned} HD_c &= \sum_{i=1}^H (c_i - \bar{c})^2 \\ &= \|(c_1 - \bar{c}, c_2 - \bar{c}, \dots, c_H - \bar{c})^T\|^2 \\ &= \|(I - \mathbf{1}_H \mathbf{1}_H^T / H) (c_1, c_2, \dots, c_H)^T\|^2 \\ &= \|(I - \mathbf{1}_H \mathbf{1}_H^T / H) W_2 (g_1, g_2, \dots, g_H)^T\|^2 \\ &= \|W_1 (g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T\|^2 \\ &\leq \frac{1}{\sigma^2} \|(g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T\|^2 \\ &= \frac{1}{\sigma^2} \sum_{i=1}^H (g_i - \bar{g})^2 \quad (\text{Ineq. 21}) \\ &= \frac{1}{\sigma^2} HD_g \quad \square \\ &= HD_a \end{aligned} \quad (27)$$

To conclude, $\bar{c} = \bar{a} = \bar{g}/\sigma$, $D_c \leq D_a = D_g/(\sigma)^2$.

Proof of (3)

(3) If we detach the gradient of σ , in backward propagation

$$\frac{\partial \ell}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} + \frac{\partial \mu}{\partial \mathbf{x}} \frac{\partial \mathbf{y}}{\partial \mu} \right) \frac{\partial \ell}{\partial \mathbf{y}} = \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T}{H} \right) \frac{\partial \ell}{\partial \mathbf{y}} \quad (28)$$

Define $W_3 = \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T}{H} \right)$, we can verify that

$$\mathbf{1}_H^T W_3 = \mathbf{1}_H^T \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T}{H} \right) = \frac{1}{\sigma} \left(\mathbf{1}_H^T - \frac{\mathbf{1}_H^T \mathbf{1}_H}{H} \mathbf{1}_H^T \right) = \frac{\mathbf{1}_H^T - \mathbf{1}_H^T}{\sigma} = 0 \quad (29)$$

Therefore,

$$H\bar{d} = \sum_{i=1}^H d_i = \mathbf{1}_H^T (d_1, d_2, \dots, d_H)^T = \mathbf{1}_H^T W_3 (g_1, g_2, \dots, g_H)^T = 0 \quad (30)$$

For any vector $\mathbf{u}$ vertical to $\mathbf{1}_H$

$$\begin{aligned} W_3 \mathbf{1}_H &= \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T}{H} \right) \mathbf{1}_H = \frac{1}{\sigma} \left(\mathbf{1}_H - \frac{\mathbf{1}_H \mathbf{1}_H^T \mathbf{1}_H}{H} \right) = \frac{\mathbf{1}_H - \mathbf{1}_H}{\sigma} = 0 \\ W_3 \mathbf{u} &= \frac{1}{\sigma} \left(I - \frac{\mathbf{1}_H \mathbf{1}_H^T}{H} \right) \mathbf{u} = \frac{1}{\sigma} \left(\mathbf{u} - \frac{\mathbf{1}_H \mathbf{1}_H^T \mathbf{u}}{H} \right) = \frac{\mathbf{u} - 0}{\sigma} = \frac{\mathbf{u}}{\sigma} \end{aligned} \quad (31)$$

Note that $(g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g}) \mathbf{1}_H = \sum_{i=1}^H (g_i - \bar{g}) = 0$, namely $(g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T$ is vertical to $\mathbf{1}_H$ and $W_3 (g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T = (g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T$, therefore

$$\begin{aligned} HD_d &= \sum_{i=1}^H (d_i - \bar{d})^2 \\ &= \sum_{i=1}^H (d_i)^2 \\ &= \|(d_1, d_2, \dots, d_H)^T\|^2 \\ &= \|W_3 (g_1, g_2, \dots, g_H)^T\|^2 \\ &= \|W_3 (g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T + W_3 \bar{g} \mathbf{1}_H\|^2 \\ &= \|W_3 (g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T\|^2 \\ &= \|(g_1 - \bar{g}, g_2 - \bar{g}, \dots, g_H - \bar{g})^T\|^2 \\ &= \frac{1}{\sigma^2} \sum_{i=1}^H (g_i - \bar{g})^2 \\ &= \frac{1}{\sigma^2} HD_g \\ &= HD_a \end{aligned} \quad (32)$$

To conclude $\bar{d} = 0$ and $D_d = D_a = D_g/(\sigma^2)$.

□

7.3 Proof of Theorem 2

Proof. Assume $d\mathbf{y} = (dy_1, dy_2, \dots, dy_H)$. Because ϕ is derivable, assume $\mathbf{v} = (\phi'(y_1), \phi'(y_2), \dots, \phi'(y_H))$. It is easy to verify

$$\begin{aligned} \sum_{i=1}^H \phi(y_i) &= HC \\ \sum_{i=1}^H y_i &= 0 \\ \sum_{i=1}^H y_i^2 &= H \end{aligned} \tag{33}$$

Differential on both sides of following three equations

$$\mathbf{v}^T d\mathbf{y} = \sum_{i=1}^H \phi'(y_i) dy_i = 0 \tag{34}$$

$$\mathbf{1}_H^T d\mathbf{y} = \sum_{i=1}^H dy_i = 0 \tag{35}$$

$$\mathbf{y}^T d\mathbf{y} = \sum_{i=1}^H y_i dy_i = 0 \tag{36}$$

In H -dim Euclidean space, note that $\mathbf{y}^T \mathbf{1}_H = \sum_{i=1}^H y_i = 0$, namely $\mathbf{1}_H$ and $\mathbf{y}$ are vertical. We expand $\mathbf{1}_H$ and $\mathbf{y}$ to an orthogonal basis $\mathbf{u}_1 = \mathbf{1}_H, \mathbf{u}_2 = \mathbf{y}, \mathbf{u}_3, \dots, \mathbf{u}_H$. Suppose $d\mathbf{y} = \sum_{i=1}^H \alpha_i \mathbf{u}_i$ and $\mathbf{v} = \sum_{i=1}^H \beta_i \mathbf{u}_i$, we have

$$\begin{aligned} \mathbf{v}^T d\mathbf{y} &= \sum_{i=1}^H \alpha_i \beta_i \|\mathbf{u}_i\|^2 = 0 \\ \mathbf{1}_H^T d\mathbf{y} &= \alpha_1 \|\mathbf{u}_1\|^2 = 0 \\ \mathbf{y}^T d\mathbf{y} &= \alpha_2 \|\mathbf{u}_2\|^2 = 0 \end{aligned} \tag{37}$$

Accoring to Eq. 37, $\sum_{i=3}^H \alpha_i \beta_i \|\mathbf{u}_i\|^2 = 0$. Because it holds in spite of $\alpha_i, (i > 2), \beta_i = 0, (i > 2)$. Therefore, $\mathbf{v} = \beta_1 \mathbf{1}_H + \beta_2 \mathbf{y}$. Namely

$$\begin{aligned} \phi'(y_i) &= \beta_1 + \beta_2 y_i \\ \phi(y_i) &= C_1 + C_2 y_i + C_3 y_i^2 \end{aligned} \tag{38}$$

Consider z_i

$$\begin{aligned}
M &> \left| \frac{1}{H} \sum_{i=1}^H z_i \right| \\
&= \left| \frac{1}{H} \sum_{i=1}^H \phi(y_i) y_i \right| \\
&= \left| \frac{1}{H} \sum_{i=1}^H C_1 y_i + C_2 y_i^2 + C_3 y_i^3 \right| \\
&= \left| \frac{1}{H} (C_1 \sum_{i=1}^H y_i + C_2 \sum_{i=1}^H y_i^2 + C_3 \sum_{i=1}^H y_i^3) \right| \\
&= \left| \frac{1}{H} (C_2 H + C_3 \sum_{i=1}^H y_i^3) \right| \\
&= \left| C_2 + \frac{C_3}{H} \sum_{i=1}^H y_i^3 \right| \\
&\geq \left| \frac{C_3}{H} \sum_{i=1}^H y_i^3 \right| - |C_2|
\end{aligned} \tag{39}$$

If we set $y_1 = 2\sqrt{H/6}$, $y_2 = y_3 = -\sqrt{H/6}$, $y_i = 0$ ($i > 3$), then $\sum_{i=1}^H y_i = 0$ and $\sum_{i=1}^H y_i^2 = \frac{4H}{6} + \frac{H}{6} + \frac{4H}{6} = H$ hold. Under this circumstances

$$\begin{aligned}
M &> \left| \frac{C_3}{H} \sum_{i=1}^H y_i^3 \right| - |C_2| \\
&= \left| \frac{C_3}{H} ((2\sqrt{H/6})^3 + (-\sqrt{H/6})^3 + (-\sqrt{H/6})^3) \right| - |C_2| \\
&= \left| \frac{C_3}{H} \left(\frac{4H}{3} - \frac{H}{6} - \frac{H}{6} \right) \sqrt{H/6} \right| - |C_2| \\
&= |C_3 \sqrt{H/6}| - |C_2|
\end{aligned} \tag{40}$$

Therefore $|C_3| < \frac{|C_2|+M}{\sqrt{H/6}}$ holds for any H , when H approaches infinity, we have $|C_3| = 0$.

$\sum_{i=1}^H \phi(y_i) = C_1 H + C_2 (\sum_{i=1}^H y_i) = C_1 H = CH$, therefore $C_1 = C$. Let $k = -C_2/C$, therefore $\phi(y_i) = C(1 - ky_i)$, then

$$\begin{aligned}
M &> \left| \frac{1}{H} \sum_{i=1}^H z_i \right| \\
&= \left| \frac{1}{H} \sum_{i=1}^H \phi(y_i) y_i \right| \\
&= \left| \frac{1}{H} \sum_{i=1}^H C_1 y_i + C_2 y_i^2 + C_3 y_i^3 \right| \\
&= \left| \frac{1}{H} (C \sum_{i=1}^H y_i - k \sum_{i=1}^H y_i^2) \right| \\
&= |k|
\end{aligned} \tag{41}$$

Namely $M > \left| \frac{1}{H} \sum_{i=1}^H z_i \right|$ can hold if $M > |k|$. To conclude $\phi(y_i) = C(1 - ky_i)$ is the only solution. $\square$

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*

University of Amsterdam, OpenAI
dpkingma@openai.com

Jimmy Lei Ba*

University of Toronto
jimmy@psi.utoronto.ca

ABSTRACT

We introduce *Adam*, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal rescaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters have intuitive interpretations and typically require little tuning. Some connections to related algorithms, on which *Adam* was inspired, are discussed. We also analyze the theoretical convergence properties of the algorithm and provide a regret bound on the convergence rate that is comparable to the best known results under the online convex optimization framework. Empirical results demonstrate that Adam works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss *AdaMax*, a variant of *Adam* based on the infinity norm.

1 INTRODUCTION

Stochastic gradient-based optimization is of core practical importance in many fields of science and engineering. Many problems in these fields can be cast as the optimization of some scalar parameterized objective function requiring maximization or minimization with respect to its parameters. If the function is differentiable w.r.t. its parameters, gradient descent is a relatively efficient optimization method, since the computation of first-order partial derivatives w.r.t. all the parameters is of the same computational complexity as just evaluating the function. Often, objective functions are stochastic. For example, many objective functions are composed of a sum of subfunctions evaluated at different subsamples of data; in this case optimization can be made more efficient by taking gradient steps w.r.t. individual subfunctions, i.e. stochastic gradient descent (SGD) or ascent. SGD proved itself as an efficient and effective optimization method that was central in many machine learning success stories, such as recent advances in deep learning (Deng et al., 2013; Krizhevsky et al., 2012; Hinton & Salakhutdinov, 2006; Hinton et al., 2012a; Graves et al., 2013). Objectives may also have other sources of noise than data subsampling, such as dropout (Hinton et al., 2012b) regularization. For all such noisy objectives, efficient stochastic optimization techniques are required. The focus of this paper is on the optimization of stochastic objectives with high-dimensional parameters spaces. In these cases, higher-order optimization methods are ill-suited, and discussion in this paper will be restricted to first-order methods.

We propose *Adam*, a method for efficient stochastic optimization that only requires first-order gradients with little memory requirement. The method computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients; the name *Adam* is derived from adaptive moment estimation. Our method is designed to combine the advantages of two recently popular methods: AdaGrad (Duchi et al., 2011), which works well with sparse gradients, and RMSProp (Tieleman & Hinton, 2012), which works well in on-line and non-stationary settings; important connections to these and other stochastic optimization methods are clarified in section 5. Some of Adam’s advantages are that the magnitudes of parameter updates are invariant to rescaling of the gradient, its stepsizes are approximately bounded by the stepsize hyperparameter, it does not require a stationary objective, it works with sparse gradients, and it naturally performs a form of step size annealing.

*Equal contribution. Author ordering determined by coin flip over a Google Hangout.

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

In section 2 we describe the algorithm and the properties of its update rule. Section 3 explains our initialization bias correction technique, and section 4 provides a theoretical analysis of Adam’s convergence in online convex programming. Empirically, our method consistently outperforms other methods for a variety of models and datasets, as shown in section 6. Overall, we show that Adam is a versatile algorithm that scales to large-scale high-dimensional machine learning problems.

2 ALGORITHM

See algorithm 1 for pseudo-code of our proposed algorithm *Adam*. Let $f(\theta)$ be a noisy objective function: a stochastic scalar function that is differentiable w.r.t. parameters θ . We are interested in minimizing the expected value of this function, $\mathbb{E}[f(\theta)]$ w.r.t. its parameters θ . With $f_1(\theta), \dots, f_T(\theta)$ we denote the realisations of the stochastic function at subsequent timesteps $1, \dots, T$. The stochasticity might come from the evaluation at random subsamples (minibatches) of datapoints, or arise from inherent function noise. With $g_t = \nabla_{\theta} f_t(\theta)$ we denote the gradient, i.e. the vector of partial derivatives of f_t , w.r.t θ evaluated at timestep t .

The algorithm updates exponential moving averages of the gradient (m_t) and the squared gradient (v_t) where the hyper-parameters $\beta_1, \beta_2 \in [0, 1)$ control the exponential decay rates of these moving averages. The moving averages themselves are estimates of the 1st moment (the mean) and the 2nd raw moment (the uncentered variance) of the gradient. However, these moving averages are initialized as (vectors of) 0’s, leading to moment estimates that are biased towards zero, especially during the initial timesteps, and especially when the decay rates are small (i.e. the β s are close to 1). The good news is that this initialization bias can be easily counteracted, resulting in bias-corrected estimates $\hat{m}_t$ and $\hat{v}_t$. See section 3 for more details.

Note that the efficiency of algorithm 1 can, at the expense of clarity, be improved upon by changing the order of computation, e.g. by replacing the last three lines in the loop with the following lines: $\alpha_t = \alpha \cdot \sqrt{1 - \beta_2^t} / (1 - \beta_1^t)$ and $\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot m_t / (\sqrt{v_t} + \epsilon)$.

2.1 ADAM’S UPDATE RULE

An important property of Adam’s update rule is its careful choice of stepsizes. Assuming $\epsilon = 0$, the effective step taken in parameter space at timestep t is $\Delta_t = \alpha \cdot \hat{m}_t / \sqrt{\hat{v}_t}$. The effective stepsize has two upper bounds: $|\Delta_t| \leq \alpha \cdot (1 - \beta_1) / \sqrt{1 - \beta_2}$ in the case $(1 - \beta_1) > \sqrt{1 - \beta_2}$, and $|\Delta_t| \leq \alpha$

otherwise. The first case only happens in the most severe case of sparsity: when a gradient has been zero at all timesteps except at the current timestep. For less sparse cases, the effective stepsize will be smaller. When $(1 - \beta_1) = \sqrt{1 - \beta_2}$ we have that $|\hat{m}_t/\sqrt{\hat{v}_t}| < 1$ therefore $|\Delta_t| < \alpha$. In more common scenarios, we will have that $\hat{m}_t/\sqrt{\hat{v}_t} \approx \pm 1$ since $|\mathbb{E}[g]/\sqrt{\mathbb{E}[g^2]}| \leq 1$. The effective magnitude of the steps taken in parameter space at each timestep are approximately bounded by the stepsize setting α , i.e., $|\Delta_t| \lesssim \alpha$. This can be understood as establishing a *trust region* around the current parameter value, beyond which the current gradient estimate does not provide sufficient information. This typically makes it relatively easy to know the right scale of α in advance. For many machine learning models, for instance, we often know in advance that good optima are with high probability within some set region in parameter space; it is not uncommon, for example, to have a prior distribution over the parameters. Since α sets (an upper bound of) the magnitude of steps in parameter space, we can often deduce the right order of magnitude of α such that optima can be reached from θ_0 within some number of iterations. With a slight abuse of terminology, we will call the ratio $\hat{m}_t/\sqrt{\hat{v}_t}$ the *signal-to-noise ratio (SNR)*. With a smaller SNR the effective stepsize Δ_t will be closer to zero. This is a desirable property, since a smaller SNR means that there is greater uncertainty about whether the direction of $\hat{m}_t$ corresponds to the direction of the true gradient. For example, the SNR value typically becomes closer to 0 towards an optimum, leading to smaller effective steps in parameter space: a form of automatic annealing. The effective stepsize Δ_t is also invariant to the scale of the gradients; rescaling the gradients g with factor c will scale $\hat{m}_t$ with a factor c and $\hat{v}_t$ with a factor c^2 , which cancel out: $(c \cdot \hat{m}_t)/(\sqrt{c^2 \cdot \hat{v}_t}) = \hat{m}_t/\sqrt{\hat{v}_t}$.

3 INITIALIZATION BIAS CORRECTION

As explained in section 2, Adam utilizes initialization bias correction terms. We will here derive the term for the second moment estimate; the derivation for the first moment estimate is completely analogous. Let g be the gradient of the stochastic objective f , and we wish to estimate its second raw moment (uncentered variance) using an exponential moving average of the squared gradient, with decay rate β_2 . Let $g_1, \dots, g_T$ be the gradients at subsequent timesteps, each a draw from an underlying gradient distribution $g_t \sim p(g_t)$. Let us initialize the exponential moving average as $v_0 = 0$ (a vector of zeros). First note that the update at timestep t of the exponential moving average $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (where g_t^2 indicates the elementwise square $g_t \odot g_t$) can be written as a function of the gradients at all previous timesteps:

$$v_t = (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} \cdot g_i^2 \quad (1)$$

We wish to know how $\mathbb{E}[v_t]$, the expected value of the exponential moving average at timestep t , relates to the true second moment $\mathbb{E}[g_t^2]$, so we can correct for the discrepancy between the two. Taking expectations of the left-hand and right-hand sides of eq. (1):

$$\mathbb{E}[v_t] = \mathbb{E} \left[(1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} \cdot g_i^2 \right] \quad (2)$$

$$= \mathbb{E}[g_t^2] \cdot (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} + \zeta \quad (3)$$

$$= \mathbb{E}[g_t^2] \cdot (1 - \beta_2^t) + \zeta \quad (4)$$

where $\zeta = 0$ if the true second moment $\mathbb{E}[g_t^2]$ is stationary; otherwise ζ can be kept small since the exponential decay rate β_1 can (and should) be chosen such that the exponential moving average assigns small weights to gradients too far in the past. What is left is the term $(1 - \beta_2^t)$ which is caused by initializing the running average with zeros. In algorithm 1 we therefore divide by this term to correct the initialization bias.

In case of sparse gradients, for a reliable estimate of the second moment one needs to average over many gradients by choosing a small value of β_2 ; however it is exactly this case of small β_2 where a lack of initialisation bias correction would lead to initial steps that are much larger.

4 CONVERGENCE ANALYSIS

We analyze the convergence of Adam using the online learning framework proposed in (Zinkevich, 2003). Given an arbitrary, unknown sequence of convex cost functions $f_1(\theta), f_2(\theta), \dots, f_T(\theta)$. At each time t , our goal is to predict the parameter θ_t and evaluate it on a previously unknown cost function f_t . Since the nature of the sequence is unknown in advance, we evaluate our algorithm using the regret, that is the sum of all the previous difference between the online prediction $f_t(\theta_t)$ and the best fixed point parameter $f_t(\theta^*)$ from a feasible set $\mathcal{X}$ for all the previous steps. Concretely, the regret is defined as:

$$R(T) = \sum_{t=1}^T [f_t(\theta_t) - f_t(\theta^*)] \quad (5)$$

where $\theta^* = \arg \min_{\theta \in \mathcal{X}} \sum_{t=1}^T f_t(\theta)$. We show Adam has $O(\sqrt{T})$ regret bound and a proof is given in the appendix. Our result is comparable to the best known bound for this general convex online learning problem. We also use some definitions simplify our notation, where $g_t \triangleq \nabla f_t(\theta_t)$ and $g_{t,i}$ as the i^{th} element. We define $g_{1:t,i} \in \mathbb{R}^t$ as a vector that contains the i^{th} dimension of the gradients over all iterations till t , $g_{1:t,i} = [g_{1,i}, g_{2,i}, \dots, g_{t,i}]$. Also, we define $\gamma \triangleq \frac{\beta_1^2}{\sqrt{\beta_2}}$. Our following theorem holds when the learning rate α_t is decaying at a rate of $t^{-\frac{1}{2}}$ and first moment running average coefficient $\beta_{1,t}$ decay exponentially with λ , that is typically close to 1, e.g. $1 - 10^{-8}$.

Theorem 4.1. *Assume that the function f_t has bounded gradients, $\|\nabla f_t(\theta)\|_2 \leq G$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in \mathbb{R}^d$ and distance between any θ_t generated by Adam is bounded, $\|\theta_n - \theta_m\|_2 \leq D$, $\|\theta_m - \theta_n\|_\infty \leq D_\infty$ for any $m, n \in \{1, \dots, T\}$, and $\beta_1, \beta_2 \in [0, 1)$ satisfy $\frac{\beta_1^2}{\sqrt{\beta_2}} < 1$. Let $\alpha_t = \frac{\alpha}{\sqrt{t}}$ and $\beta_{1,t} = \beta_1 \lambda^{t-1}$, $\lambda \in (0, 1)$. Adam achieves the following guarantee, for all $T \geq 1$.*

$$R(T) \leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^d \sqrt{T \hat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^d \|g_{1:T,i}\|_2 + \sum_{i=1}^d \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha(1-\beta_1)(1-\lambda)^2}$$

Our Theorem 4.1 implies when the data features are sparse and bounded gradients, the summation term can be much smaller than its upper bound $\sum_{i=1}^d \|g_{1:T,i}\|_2 \ll dG_\infty \sqrt{T}$ and $\sum_{i=1}^d \sqrt{T \hat{v}_{T,i}} \ll dG_\infty \sqrt{T}$, in particular if the class of function and data features are in the form of section 1.2 in (Duchi et al., 2011). Their results for the expected value $\mathbb{E}[\sum_{i=1}^d \|g_{1:T,i}\|_2]$ also apply to Adam. In particular, the adaptive method, such as Adam and Adagrad, can achieve $O(\log d \sqrt{T})$, an improvement over $O(\sqrt{dT})$ for the non-adaptive method. Decaying $\beta_{1,t}$ towards zero is important in our theoretical analysis and also matches previous empirical findings, e.g. (Sutskever et al., 2013) suggests reducing the momentum coefficient in the end of training can improve convergence.

Finally, we can show the average regret of Adam converges,

Corollary 4.2. *Assume that the function f_t has bounded gradients, $\|\nabla f_t(\theta)\|_2 \leq G$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in \mathbb{R}^d$ and distance between any θ_t generated by Adam is bounded, $\|\theta_n - \theta_m\|_2 \leq D$, $\|\theta_m - \theta_n\|_\infty \leq D_\infty$ for any $m, n \in \{1, \dots, T\}$. Adam achieves the following guarantee, for all $T \geq 1$.*

$$\frac{R(T)}{T} = O\left(\frac{1}{\sqrt{T}}\right)$$

This result can be obtained by using Theorem 4.1 and $\sum_{i=1}^d \|g_{1:T,i}\|_2 \leq dG_\infty \sqrt{T}$. Thus, $\lim_{T \rightarrow \infty} \frac{R(T)}{T} = 0$.

5 RELATED WORK

Optimization methods bearing a direct relation to Adam are RMSProp (Tieleman & Hinton, 2012; Graves, 2013) and AdaGrad (Duchi et al., 2011); these relationships are discussed below. Other stochastic optimization methods include vSGD (Schaul et al., 2012), AdaDelta (Zeiler, 2012) and the natural Newton method from Roux & Fitzgibbon (2010), all setting stepsizes by estimating curvature

from first-order information. The Sum-of-Functions Optimizer (SFO) (Sohl-Dickstein et al., 2014) is a quasi-Newton method based on minibatches, but (unlike Adam) has memory requirements linear in the number of minibatch partitions of a dataset, which is often infeasible on memory-constrained systems such as a GPU. Like natural gradient descent (NGD) (Amari, 1998), Adam employs a preconditioner that adapts to the geometry of the data, since $\hat{v}_t$ is an approximation to the diagonal of the Fisher information matrix (Pascanu & Bengio, 2013); however, Adam’s preconditioner (like AdaGrad’s) is more conservative in its adaption than vanilla NGD by preconditioning with the square root of the inverse of the diagonal Fisher information matrix approximation.

RMSPProp: An optimization method closely related to Adam is RMSPProp (Tieleman & Hinton, 2012). A version with momentum has sometimes been used (Graves, 2013). There are a few important differences between RMSPProp with momentum and Adam: RMSPProp with momentum generates its parameter updates using a momentum on the rescaled gradient, whereas Adam updates are directly estimated using a running average of first and second moment of the gradient. RMSPProp also lacks a bias-correction term; this matters most in case of a value of β_2 close to 1 (required in case of sparse gradients), since in that case not correcting the bias leads to very large stepsizes and often divergence, as we also empirically demonstrate in section 6.4.

AdaGrad: An algorithm that works well for sparse gradients is AdaGrad (Duchi et al., 2011). Its basic version updates parameters as $\theta_{t+1} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^t g_i^2}$. Note that if we choose β_2 to be infinitesimally close to 1 from below, then $\lim_{\beta_2 \rightarrow 1} \hat{v}_t = t^{-1} \cdot \sum_{i=1}^t g_i^2$. AdaGrad corresponds to a version of Adam with $\beta_1 = 0$, infinitesimal $(1 - \beta_2)$ and a replacement of α by an annealed version $\alpha_t = \alpha \cdot t^{-1/2}$, namely $\theta_t - \alpha \cdot t^{-1/2} \cdot \hat{m}_t / \sqrt{\lim_{\beta_2 \rightarrow 1} \hat{v}_t} = \theta_t - \alpha \cdot t^{-1/2} \cdot g_t / \sqrt{t^{-1} \cdot \sum_{i=1}^t g_i^2} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^t g_i^2}$. Note that this direct correspondence between Adam and Adagrad does not hold when removing the bias-correction terms; without bias correction, like in RMSPProp, a β_2 infinitesimally close to 1 would lead to infinitely large bias, and infinitely large parameter updates.

6 EXPERIMENTS

To empirically evaluate the proposed method, we investigated different popular machine learning models, including logistic regression, multilayer fully connected neural networks and deep convolutional neural networks. Using large models and datasets, we demonstrate Adam can efficiently solve practical deep learning problems.

We use the same parameter initialization when comparing different optimization algorithms. The hyper-parameters, such as learning rate and momentum, are searched over a dense grid and the results are reported using the best hyper-parameter setting.

6.1 EXPERIMENT: LOGISTIC REGRESSION

We evaluate our proposed method on L2-regularized multi-class logistic regression using the MNIST dataset. Logistic regression has a well-studied convex objective, making it suitable for comparison of different optimizers without worrying about local minimum issues. The stepsize α in our logistic regression experiments is adjusted by $1/\sqrt{t}$ decay, namely $\alpha_t = \frac{\alpha}{\sqrt{t}}$ that matches with our theoretical prediction from section 4. The logistic regression classifies the class label directly on the 784 dimension image vectors. We compare Adam to accelerated SGD with Nesterov momentum and Adagrad using minibatch size of 128. According to Figure 1, we found that the Adam yields similar convergence as SGD with momentum and both converge faster than Adagrad.

As discussed in (Duchi et al., 2011), Adagrad can efficiently deal with sparse features and gradients as one of its main theoretical results whereas SGD is low at learning rare features. Adam with $1/\sqrt{t}$ decay on its stepsize should theoretically match the performance of Adagrad. We examine the sparse feature problem using IMDB movie review dataset from (Maas et al., 2011). We pre-process the IMDB movie reviews into bag-of-words (BoW) feature vectors including the first 10,000 most frequent words. The 10,000 dimension BoW feature vector for each review is highly sparse. As suggested in (Wang & Manning, 2013), 50% dropout noise can be applied to the BoW features during

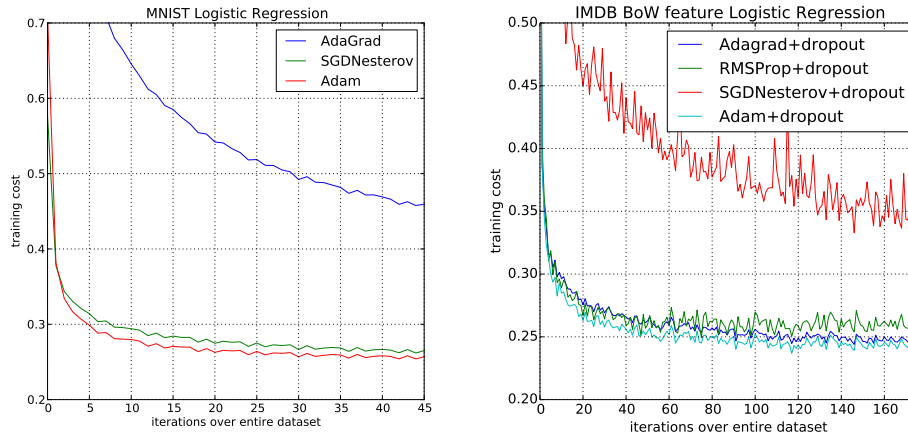


Figure 1: Logistic regression training negative log likelihood on MNIST images and IMDB movie reviews with 10,000 bag-of-words (BoW) feature vectors.

training to prevent over-fitting. In figure 1, Adagrad outperforms SGD with Nesterov momentum by a large margin both with and without dropout noise. Adam converges as fast as Adagrad. The empirical performance of Adam is consistent with our theoretical findings in sections 2 and 4. Similar to Adagrad, Adam can take advantage of sparse features and obtain faster convergence rate than normal SGD with momentum.

6.2 EXPERIMENT: MULTI-LAYER NEURAL NETWORKS

Multi-layer neural network are powerful models with non-convex objective functions. Although our convergence analysis does not apply to non-convex problems, we empirically found that Adam often outperforms other methods in such cases. In our experiments, we made model choices that are consistent with previous publications in the area; a neural network model with two fully connected hidden layers with 1000 hidden units each and ReLU activation are used for this experiment with minibatch size of 128.

First, we study different optimizers using the standard deterministic cross-entropy objective function with L_2 weight decay on the parameters to prevent over-fitting. The sum-of-functions (SFO) method (Sohl-Dickstein et al., 2014) is a recently proposed quasi-Newton method that works with minibatches of data and has shown good performance on optimization of multi-layer neural networks. We used their implementation and compared with Adam to train such models. Figure 2 shows that Adam makes faster progress in terms of both the number of iterations and wall-clock time. Due to the cost of updating curvature information, SFO is 5-10x slower per iteration compared to Adam, and has a memory requirement that is linear in the number minibatches.

Stochastic regularization methods, such as dropout, are an effective way to prevent over-fitting and often used in practice due to their simplicity. SFO assumes deterministic subfunctions, and indeed failed to converge on cost functions with stochastic regularization. We compare the effectiveness of Adam to other stochastic first order methods on multi-layer neural networks trained with dropout noise. Figure 2 shows our results; Adam shows better convergence than other methods.

6.3 EXPERIMENT: CONVOLUTIONAL NEURAL NETWORKS

Convolutional neural networks (CNNs) with several layers of convolution, pooling and non-linear units have shown considerable success in computer vision tasks. Unlike most fully connected neural nets, weight sharing in CNNs results in vastly different gradients in different layers. A smaller learning rate for the convolution layers is often used in practice when applying SGD. We show the effectiveness of Adam in deep CNNs. Our CNN architecture has three alternating stages of 5x5 convolution filters and 3x3 max pooling with stride of 2 that are followed by a fully connected layer of 1000 rectified linear hidden units (ReLU's). The input image are pre-processed by whitening, and

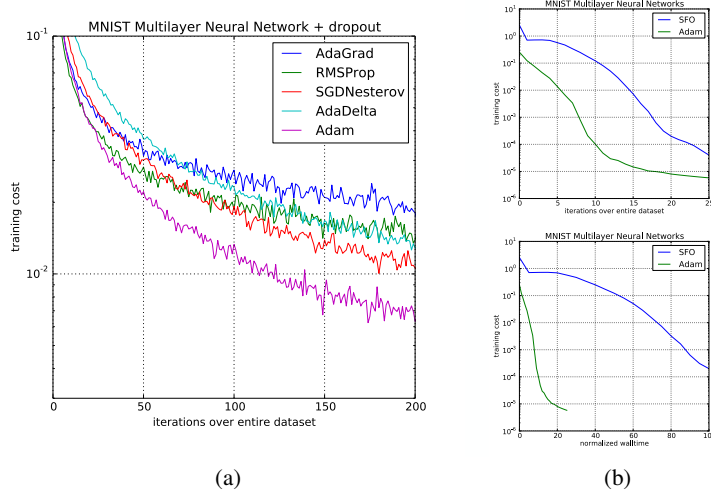


Figure 2: Training of multilayer neural networks on MNIST images. (a) Neural networks using dropout stochastic regularization. (b) Neural networks with deterministic cost function. We compare with the sum-of-functions (SFO) optimizer (Sohl-Dickstein et al., 2014)

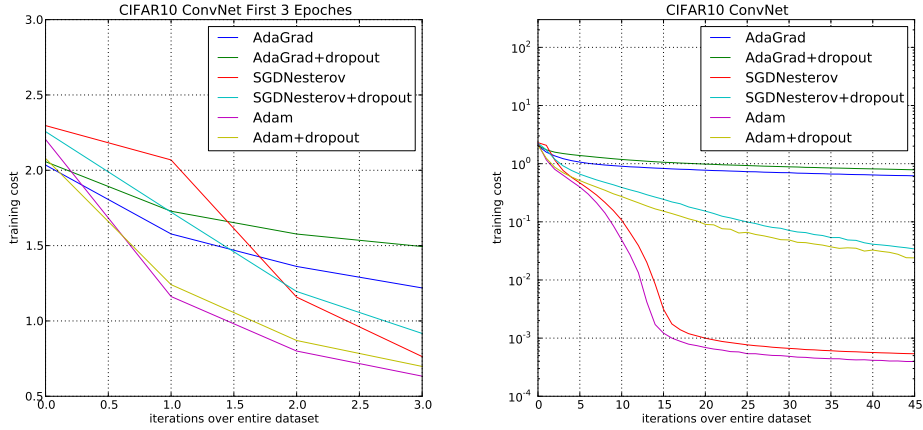


Figure 3: Convolutional neural networks training cost. (left) Training cost for the first three epochs. (right) Training cost over 45 epochs. CIFAR-10 with c64-c64-c128-1000 architecture.

dropout noise is applied to the input layer and fully connected layer. The minibatch size is also set to 128 similar to previous experiments.

Interestingly, although both Adam and Adagrad make rapid progress lowering the cost in the initial stage of the training, shown in Figure 3 (left), Adam and SGD eventually converge considerably faster than Adagrad for CNNs shown in Figure 3 (right). We notice the second moment estimate $\hat{v}_t$ vanishes to zeros after a few epochs and is dominated by the ϵ in algorithm 1. The second moment estimate is therefore a poor approximation to the geometry of the cost function in CNNs comparing to fully connected network from Section 6.2. Whereas, reducing the minibatch variance through the first moment is more important in CNNs and contributes to the speed-up. As a result, Adagrad converges much slower than others in this particular experiment. Though Adam shows marginal improvement over SGD with momentum, it adapts learning rate scale for different layers instead of hand picking manually as in SGD.

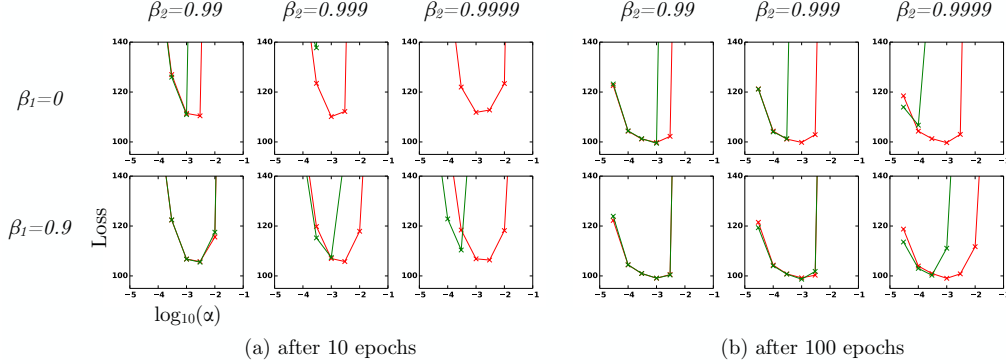


Figure 4: Effect of bias-correction terms (red line) versus no bias correction terms (green line) after 10 epochs (left) and 100 epochs (right) on the loss (y-axes) when learning a Variational Auto-Encoder (VAE) (Kingma & Welling, 2013), for different settings of stepsize α (x-axes) and hyper-parameters β_1 and β_2 .

6.4 EXPERIMENT: BIAS-CORRECTION TERM

We also empirically evaluate the effect of the bias correction terms explained in sections 2 and 3. Discussed in section 5, removal of the bias correction terms results in a version of RMSProp (Tieleman & Hinton, 2012) with momentum. We vary the β_1 and β_2 when training a variational auto-encoder (VAE) with the same architecture as in (Kingma & Welling, 2013) with a single hidden layer with 500 hidden units with softplus nonlinearities and a 50-dimensional spherical Gaussian latent variable. We iterated over a broad range of hyper-parameter choices, i.e. $\beta_1 \in [0, 0.9]$ and $\beta_2 \in [0.99, 0.999, 0.9999]$, and $\log_{10}(\alpha) \in [-5, \dots, -1]$. Values of β_2 close to 1, required for robustness to sparse gradients, results in larger initialization bias; therefore we expect the bias correction term is important in such cases of slow decay, preventing an adverse effect on optimization.

In Figure 4, values β_2 close to 1 indeed lead to instabilities in training when no bias correction term was present, especially at first few epochs of the training. The best results were achieved with small values of $(1 - \beta_2)$ and bias correction; this was more apparent towards the end of optimization when gradients tends to become sparser as hidden units specialize to specific patterns. In summary, Adam performed equal or better than RMSProp, regardless of hyper-parameter setting.

7 EXTENSIONS

7.1 ADAMAX

In Adam, the update rule for individual weights is to scale their gradients inversely proportional to a (scaled) L^2 norm of their individual current and past gradients. We can generalize the L^2 norm based update rule to a L^p norm based update rule. Such variants become numerically unstable for large p . However, in the special case where we let $p \rightarrow \infty$, a surprisingly simple and stable algorithm emerges; see algorithm 2. We'll now derive the algorithm. Let, in case of the L^p norm, the stepsize at time t be inversely proportional to $v_t^{1/p}$, where:

$$v_t = \beta_2^p v_{t-1} + (1 - \beta_2^p) |g_t|^p \quad (6)$$

$$= (1 - \beta_2^p) \sum_{i=1}^t \beta_2^{p(t-i)} \cdot |g_i|^p \quad (7)$$

Algorithm 2: *AdaMax*, a variant of Adam based on the infinity norm. See section 7.1 for details. Good default settings for the tested machine learning problems are $\alpha = 0.002$, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. With β_1^t we denote β_1 to the power t . Here, $(\alpha/(1 - \beta_1^t))$ is the learning rate with the bias-correction term for the first moment. All operations on vectors are element-wise.

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1]$: Exponential decay rates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$u_0 \leftarrow 0$ (Initialize the exponentially weighted infinity norm)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$u_t \leftarrow \max(\beta_2 \cdot u_{t-1}, |g_t|)$ (Update the exponentially weighted infinity norm)

$\theta_t \leftarrow \theta_{t-1} - (\alpha/(1 - \beta_1^t)) \cdot m_t/u_t$ (Update parameters)

end while

return θ_t (Resulting parameters)

Note that the decay term is here equivalently parameterised as β_2^p instead of β_2 . Now let $p \rightarrow \infty$, and define $u_t = \lim_{p \rightarrow \infty} (v_t)^{1/p}$, then:

$$u_t = \lim_{p \rightarrow \infty} (v_t)^{1/p} = \lim_{p \rightarrow \infty} \left((1 - \beta_2^p) \sum_{i=1}^t \beta_2^{p(t-i)} \cdot |g_i|^p \right)^{1/p} \quad (8)$$

$$= \lim_{p \rightarrow \infty} (1 - \beta_2^p)^{1/p} \left(\sum_{i=1}^t \beta_2^{p(t-i)} \cdot |g_i|^p \right)^{1/p} \quad (9)$$

$$= \lim_{p \rightarrow \infty} \left(\sum_{i=1}^t \left(\beta_2^{(t-i)} \cdot |g_i| \right)^p \right)^{1/p} \quad (10)$$

$$= \max(\beta_2^{t-1}|g_1|, \beta_2^{t-2}|g_2|, \dots, \beta_2|g_{t-1}|, |g_t|) \quad (11)$$

Which corresponds to the remarkably simple recursive formula:

$$u_t = \max(\beta_2 \cdot u_{t-1}, |g_t|) \quad (12)$$

with initial value $u_0 = 0$. Note that, conveniently enough, we don't need to correct for initialization bias in this case. Also note that the magnitude of parameter updates has a simpler bound with AdaMax than Adam, namely: $|\Delta_t| \leq \alpha$.

7.2 TEMPORAL AVERAGING

Since the last iterate is noisy due to stochastic approximation, better generalization performance is often achieved by averaging. Previously in Moulines & Bach (2011), Polyak-Ruppert averaging (Polyak & Juditsky, 1992; Ruppert, 1988) has been shown to improve the convergence of standard SGD, where $\bar{\theta}_t = \frac{1}{t} \sum_{k=1}^t \theta_k$. Alternatively, an exponential moving average over the parameters can be used, giving higher weight to more recent parameter values. This can be trivially implemented by adding one line to the inner loop of algorithms 1 and 2: $\bar{\theta}_t \leftarrow \beta_2 \cdot \bar{\theta}_{t-1} + (1 - \beta_2)\theta_t$, with $\bar{\theta}_0 = 0$. Initialization bias can again be corrected by the estimator $\hat{\theta}_t = \bar{\theta}_t/(1 - \beta_2^t)$.

8 CONCLUSION

We have introduced a simple and computationally efficient algorithm for gradient-based optimization of stochastic objective functions. Our method is aimed towards machine learning problems with

large datasets and/or high-dimensional parameter spaces. The method combines the advantages of two recently popular optimization methods: the ability of AdaGrad to deal with sparse gradients, and the ability of RMSProp to deal with non-stationary objectives. The method is straightforward to implement and requires little memory. The experiments confirm the analysis on the rate of convergence in convex problems. Overall, we found Adam to be robust and well-suited to a wide range of non-convex optimization problems in the field machine learning.

9 ACKNOWLEDGMENTS

This paper would probably not have existed without the support of Google Deepmind. We would like to give special thanks to Ivo Danihelka, and Tom Schaul for coining the name Adam. Thanks to Kai Fan from Duke University for spotting an error in the original AdaMax derivation. Experiments in this work were partly carried out on the Dutch national e-infrastructure with the support of SURF Foundation. Diederik Kingma is supported by the Google European Doctorate Fellowship in Deep Learning.

REFERENCES

- Amari, Shun-Ichi. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.
- Deng, Li, Li, Jinyu, Huang, Jui-Ting, Yao, Kaisheng, Yu, Dong, Seide, Frank, Seltzer, Michael, Zweig, Geoff, He, Xiaodong, Williams, Jason, et al. Recent advances in deep learning for speech research at microsoft. *ICASSP 2013*, 2013.
- Duchi, John, Hazan, Elad, and Singer, Yoram. Adaptive subgradient methods for online learning and stochastic optimization. *The Journal of Machine Learning Research*, 12:2121–2159, 2011.
- Graves, Alex. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- Graves, Alex, Mohamed, Abdel-rahman, and Hinton, Geoffrey. Speech recognition with deep recurrent neural networks. In *Acoustics, Speech and Signal Processing (ICASSP), 2013 IEEE International Conference on*, pp. 6645–6649. IEEE, 2013.
- Hinton, G.E. and Salakhutdinov, R.R. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- Hinton, Geoffrey, Deng, Li, Yu, Dong, Dahl, George E, Mohamed, Abdel-rahman, Jaitly, Navdeep, Senior, Andrew, Vanhoucke, Vincent, Nguyen, Patrick, Sainath, Tara N, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *Signal Processing Magazine, IEEE*, 29(6):82–97, 2012a.
- Hinton, Geoffrey E, Srivastava, Nitish, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan R. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012b.
- Kingma, Diederik P and Welling, Max. Auto-Encoding Variational Bayes. In *The 2nd International Conference on Learning Representations (ICLR)*, 2013.
- Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pp. 1097–1105, 2012.
- Maas, Andrew L, Daly, Raymond E, Pham, Peter T, Huang, Dan, Ng, Andrew Y, and Potts, Christopher. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies-Volume 1*, pp. 142–150. Association for Computational Linguistics, 2011.
- Moulines, Eric and Bach, Francis R. Non-asymptotic analysis of stochastic approximation algorithms for machine learning. In *Advances in Neural Information Processing Systems*, pp. 451–459, 2011.
- Pascanu, Razvan and Bengio, Yoshua. Revisiting natural gradient for deep networks. *arXiv preprint arXiv:1301.3584*, 2013.
- Polyak, Boris T and Juditsky, Anatoli B. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.

10 APPENDIX

10.1 CONVERGENCE PROOF

Definition 10.1. A function $f : R^d \rightarrow R$ is convex if for all $x, y \in R^d$, for all $\lambda \in [0, 1]$,

$$\lambda f(x) + (1 - \lambda)f(y) \geq f(\lambda x + (1 - \lambda)y)$$

Also, notice that a convex function can be lower bounded by a hyperplane at its tangent.

Lemma 10.2. If a function $f : R^d \rightarrow R$ is convex, then for all $x, y \in R^d$,

$$f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

The above lemma can be used to upper bound the regret and our proof for the main theorem is constructed by substituting the hyperplane with the Adam update rules.

The following two lemmas are used to support our main theorem. We also use some definitions simplify our notation, where $g_t \triangleq \nabla f_t(\theta_t)$ and $g_{t,i}$ as the i^{th} element. We define $g_{1:t,i} \in \mathbb{R}^t$ as a vector that contains the i^{th} dimension of the gradients over all iterations till t , $g_{1:t,i} = [g_{1,i}, g_{2,i}, \dots, g_{t,i}]$

Lemma 10.3. Let $g_t = \nabla f_t(\theta_t)$ and $g_{1:t}$ be defined as above and bounded, $\|g_t\|_2 \leq G$, $\|g_t\|_\infty \leq G_\infty$. Then,

$$\sum_{t=1}^T \sqrt{\frac{g_{t,i}^2}{t}} \leq 2G_\infty \|g_{1:T,i}\|_2$$

Proof. We will prove the inequality using induction over T .

The base case for $T = 1$, we have $\sqrt{g_{1,i}^2} \leq 2G_\infty \|g_{1,i}\|_2$.

For the inductive step,

$$\begin{aligned} \sum_{t=1}^T \sqrt{\frac{g_{t,i}^2}{t}} &= \sum_{t=1}^{T-1} \sqrt{\frac{g_{t,i}^2}{t}} + \sqrt{\frac{g_{T,i}^2}{T}} \\ &\leq 2G_\infty \|g_{1:T-1,i}\|_2 + \sqrt{\frac{g_{T,i}^2}{T}} \\ &= 2G_\infty \sqrt{\|g_{1:T,i}\|_2^2 - g_{T,i}^2} + \sqrt{\frac{g_{T,i}^2}{T}} \end{aligned}$$

From, $\|g_{1:T,i}\|_2^2 - g_{T,i}^2 + \frac{g_{T,i}^4}{4\|g_{1:T,i}\|_2^2} \geq \|g_{1:T,i}\|_2^2 - g_{T,i}^2$, we can take square root of both side and have,

$$\begin{aligned} \sqrt{\|g_{1:T,i}\|_2^2 - g_{T,i}^2} &\leq \|g_{1:T,i}\|_2 - \frac{g_{T,i}^2}{2\|g_{1:T,i}\|_2} \\ &\leq \|g_{1:T,i}\|_2 - \frac{g_{T,i}^2}{2\sqrt{T}G_\infty} \end{aligned}$$

Rearrange the inequality and substitute the $\sqrt{\|g_{1:T,i}\|_2^2 - g_{T,i}^2}$ term,

$$G_\infty \sqrt{\|g_{1:T,i}\|_2^2 - g_{T,i}^2} + \sqrt{\frac{g_{T,i}^2}{T}} \leq 2G_\infty \|g_{1:T,i}\|_2$$

□

Lemma 10.4. Let $\gamma \triangleq \frac{\beta_1^2}{\sqrt{\beta_2}}$. For $\beta_1, \beta_2 \in [0, 1)$ that satisfy $\frac{\beta_1^2}{\sqrt{\beta_2}} < 1$ and bounded g_t , $\|g_t\|_2 \leq G$, $\|g_t\|_\infty \leq G_\infty$, the following inequality holds

$$\sum_{t=1}^T \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} \leq \frac{2}{1-\gamma} \frac{1}{\sqrt{1-\beta_2}} \|g_{1:T,i}\|_2$$

Proof. Under the assumption, $\frac{\sqrt{1-\beta_2^t}}{(1-\beta_1^t)^2} \leq \frac{1}{(1-\beta_1)^2}$. We can expand the last term in the summation using the update rules in Algorithm 1,

$$\begin{aligned} \sum_{t=1}^T \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} &= \sum_{t=1}^{T-1} \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \frac{(\sum_{k=1}^T (1-\beta_1)\beta_1^{T-k} g_{k,i})^2}{\sqrt{T \sum_{j=1}^T (1-\beta_2)\beta_2^{T-j} g_{j,i}^2}} \\ &\leq \sum_{t=1}^{T-1} \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \sum_{k=1}^T \frac{T((1-\beta_1)\beta_1^{T-k} g_{k,i})^2}{\sqrt{T \sum_{j=1}^T (1-\beta_2)\beta_2^{T-j} g_{j,i}^2}} \\ &\leq \sum_{t=1}^{T-1} \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \sum_{k=1}^T \frac{T((1-\beta_1)\beta_1^{T-k} g_{k,i})^2}{\sqrt{T(1-\beta_2)\beta_2^{T-k} g_{k,i}^2}} \\ &\leq \sum_{t=1}^{T-1} \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \frac{(1-\beta_1)^2}{\sqrt{T(1-\beta_2)}} \sum_{k=1}^T T \left(\frac{\beta_1^2}{\sqrt{\beta_2}} \right)^{T-k} \|g_{k,i}\|_2 \\ &\leq \sum_{t=1}^{T-1} \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} + \frac{T}{\sqrt{T(1-\beta_2)}} \sum_{k=1}^T \gamma^{T-k} \|g_{k,i}\|_2 \end{aligned}$$

Similarly, we can upper bound the rest of the terms in the summation.

$$\begin{aligned} \sum_{t=1}^T \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} &\leq \sum_{t=1}^T \frac{\|g_{t,i}\|_2}{\sqrt{t(1-\beta_2)}} \sum_{j=0}^{T-t} t\gamma^j \\ &\leq \sum_{t=1}^T \frac{\|g_{t,i}\|_2}{\sqrt{t(1-\beta_2)}} \sum_{j=0}^T t\gamma^j \end{aligned}$$

For $\gamma < 1$, using the upper bound on the arithmetic-geometric series, $\sum_t t\gamma^t < \frac{1}{(1-\gamma)^2}$:

$$\sum_{t=1}^T \frac{\|g_{t,i}\|_2}{\sqrt{t(1-\beta_2)}} \sum_{j=0}^T t\gamma^j \leq \frac{1}{(1-\gamma)^2 \sqrt{1-\beta_2}} \sum_{t=1}^T \frac{\|g_{t,i}\|_2}{\sqrt{t}}$$

Apply Lemma 10.3,

$$\sum_{t=1}^T \frac{\hat{m}_{t,i}^2}{\sqrt{t\hat{v}_{t,i}}} \leq \frac{2G_\infty}{(1-\gamma)^2 \sqrt{1-\beta_2}} \|g_{1:T,i}\|_2$$

□

To simplify the notation, we define $\gamma \triangleq \frac{\beta_1^2}{\sqrt{\beta_2}}$. Intuitively, our following theorem holds when the learning rate α_t is decaying at a rate of $t^{-\frac{1}{2}}$ and first moment running average coefficient $\beta_{1,t}$ decay exponentially with λ , that is typically close to 1, e.g. $1 - 10^{-8}$.

Theorem 10.5. Assume that the function f_t has bounded gradients, $\|\nabla f_t(\theta)\|_2 \leq G$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in \mathbb{R}^d$ and distance between any θ_t generated by Adam is bounded, $\|\theta_n - \theta_m\|_2 \leq D$,

$\|\theta_m - \theta_n\|_\infty \leq D_\infty$ for any $m, n \in \{1, \dots, T\}$, and $\beta_1, \beta_2 \in [0, 1)$ satisfy $\frac{\beta_1^2}{\sqrt{\beta_2}} < 1$. Let $\alpha_t = \frac{\alpha}{\sqrt{t}}$ and $\beta_{1,t} = \beta_1 \lambda^{t-1}$, $\lambda \in (0, 1)$. Adam achieves the following guarantee, for all $T \geq 1$.

$$R(T) \leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^d \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(\beta_1+1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^d \|g_{1:T,i}\|_2 + \sum_{i=1}^d \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha(1-\beta_1)(1-\lambda)^2}$$

Proof. Using Lemma 10.2, we have,

$$f_t(\theta_t) - f_t(\theta^*) \leq g_t^T(\theta_t - \theta^*) = \sum_{i=1}^d g_{t,i}(\theta_{t,i} - \theta_{*,i}^*)$$

From the update rules presented in algorithm 1,

$$\begin{aligned} \theta_{t+1} &= \theta_t - \alpha_t \widehat{m}_t / \sqrt{\widehat{v}_t} \\ &= \theta_t - \frac{\alpha_t}{1 - \beta_1^t} \left(\frac{\beta_{1,t}}{\sqrt{\widehat{v}_t}} m_{t-1} + \frac{(1 - \beta_{1,t})}{\sqrt{\widehat{v}_t}} g_t \right) \end{aligned}$$

We focus on the i^{th} dimension of the parameter vector $\theta_t \in R^d$. Subtract the scalar $\theta_{*,i}^*$ and square both sides of the above update rule, we have,

$$(\theta_{t+1,i} - \theta_{*,i}^*)^2 = (\theta_{t,i} - \theta_{*,i}^*)^2 - \frac{2\alpha_t}{1 - \beta_1^t} \left(\frac{\beta_{1,t}}{\sqrt{\widehat{v}_{t,i}}} m_{t-1,i} + (1 - \frac{\beta_{1,t}}{\sqrt{\widehat{v}_{t,i}}} g_{t,i}) \right) (\theta_{t,i} - \theta_{*,i}^*) + \alpha_t^2 \left(\frac{\widehat{m}_{t,i}}{\sqrt{\widehat{v}_{t,i}}} \right)^2$$

We can rearrange the above equation and use Young's inequality, $ab \leq a^2/2 + b^2/2$. Also, it can be shown that $\sqrt{\widehat{v}_{t,i}} = \sqrt{\sum_{j=1}^t (1 - \beta_2) \beta_2^{t-j} g_{j,i}^2} / \sqrt{1 - \beta_2^t} \leq \|g_{1:t,i}\|_2$ and $\beta_{1,t} \leq \beta_1$. Then

$$\begin{aligned} g_{t,i}(\theta_{t,i} - \theta_{*,i}^*) &= \frac{(1 - \beta_1^t) \sqrt{\widehat{v}_{t,i}}}{2\alpha_t(1 - \beta_{1,t})} \left((\theta_{t,i} - \theta_{*,i}^*)^2 - (\theta_{t+1,i} - \theta_{*,i}^*)^2 \right) \\ &\quad + \frac{\beta_{1,t}}{(1 - \beta_{1,t})} \frac{\widehat{v}_{t-1,i}^{\frac{1}{4}}}{\sqrt{\alpha_{t-1}}} (\theta_{*,i}^* - \theta_{t,i}) \sqrt{\alpha_{t-1}} \frac{m_{t-1,i}}{\widehat{v}_{t-1,i}^{\frac{1}{4}}} + \frac{\alpha_t(1 - \beta_1^t) \sqrt{\widehat{v}_{t,i}}}{2(1 - \beta_{1,t})} \left(\frac{\widehat{m}_{t,i}}{\sqrt{\widehat{v}_{t,i}}} \right)^2 \\ &\leq \frac{1}{2\alpha_t(1 - \beta_1)} \left((\theta_{t,i} - \theta_{*,i}^*)^2 - (\theta_{t+1,i} - \theta_{*,i}^*)^2 \right) \sqrt{\widehat{v}_{t,i}} + \frac{\beta_{1,t}}{2\alpha_{t-1}(1 - \beta_{1,t})} (\theta_{*,i}^* - \theta_{t,i})^2 \sqrt{\widehat{v}_{t-1,i}} \\ &\quad + \frac{\beta_1 \alpha_{t-1}}{2(1 - \beta_1)} \frac{m_{t-1,i}^2}{\sqrt{\widehat{v}_{t-1,i}}} + \frac{\alpha_t}{2(1 - \beta_1)} \frac{\widehat{m}_{t,i}^2}{\sqrt{\widehat{v}_{t,i}}} \end{aligned}$$

We apply Lemma 10.4 to the above inequality and derive the regret bound by summing across all the dimensions for $i \in 1, \dots, d$ in the upper bound of $f_t(\theta_t) - f_t(\theta^*)$ and the sequence of convex functions for $t \in 1, \dots, T$:

$$\begin{aligned} R(T) &\leq \sum_{i=1}^d \frac{1}{2\alpha_1(1 - \beta_1)} (\theta_{1,i} - \theta_{*,i}^*)^2 \sqrt{\widehat{v}_{1,i}} + \sum_{i=1}^d \sum_{t=2}^T \frac{1}{2(1 - \beta_1)} (\theta_{t,i} - \theta_{*,i}^*)^2 \left(\frac{\sqrt{\widehat{v}_{t,i}}}{\alpha_t} - \frac{\sqrt{\widehat{v}_{t-1,i}}}{\alpha_{t-1}} \right) \\ &\quad + \frac{\beta_1 \alpha G_\infty}{(1 - \beta_1) \sqrt{1 - \beta_2} (1 - \gamma)^2} \sum_{i=1}^d \|g_{1:T,i}\|_2 + \frac{\alpha G_\infty}{(1 - \beta_1) \sqrt{1 - \beta_2} (1 - \gamma)^2} \sum_{i=1}^d \|g_{1:T,i}\|_2 \\ &\quad + \sum_{i=1}^d \sum_{t=1}^T \frac{\beta_{1,t}}{2\alpha_t(1 - \beta_{1,t})} (\theta_{*,i}^* - \theta_{t,i})^2 \sqrt{\widehat{v}_{t,i}} \end{aligned}$$

From the assumption, $\|\theta_t - \theta^*\|_2 \leq D$, $\|\theta_m - \theta_n\|_\infty \leq D_\infty$, we have:

$$\begin{aligned}
R(T) &\leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^d \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^d \|g_{1:T,i}\|_2 + \frac{D_\infty^2}{2\alpha} \sum_{i=1}^d \sum_{t=1}^t \frac{\beta_{1,t}}{(1-\beta_{1,t})} \sqrt{t\widehat{v}_{t,i}} \\
&\leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^d \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^d \|g_{1:T,i}\|_2 \\
&\quad + \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha} \sum_{i=1}^d \sum_{t=1}^t \frac{\beta_{1,t}}{(1-\beta_{1,t})} \sqrt{t}
\end{aligned}$$

We can use arithmetic geometric series upper bound for the last term:

$$\begin{aligned}
\sum_{t=1}^t \frac{\beta_{1,t}}{(1-\beta_{1,t})} \sqrt{t} &\leq \sum_{t=1}^t \frac{1}{(1-\beta_1)} \lambda^{t-1} \sqrt{t} \\
&\leq \sum_{t=1}^t \frac{1}{(1-\beta_1)} \lambda^{t-1} t \\
&\leq \frac{1}{(1-\beta_1)(1-\lambda)^2}
\end{aligned}$$

Therefore, we have the following regret bound:

$$R(T) \leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^d \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^d \|g_{1:T,i}\|_2 + \sum_{i=1}^d \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha\beta_1(1-\lambda)^2}$$

□